

第 12 章 推荐系统模块

从第 11.7 章节的分析可知，面对亿级用户与十亿级内容池的双重规模压力，工业级推荐系统必须在推荐效果、响应延迟与算力成本之间取得精细平衡。在此约束下，系统自然演化出一种高效且可扩展的架构范式——多阶段级联架构（Multi-stage Cascade Architecture）。该架构以“漏斗”（funnel）形式逐层缩小候选集规模，同时逐级提升模型复杂度与打分精度，从而在有限资源下实现高质量、低延迟的个性化推荐。

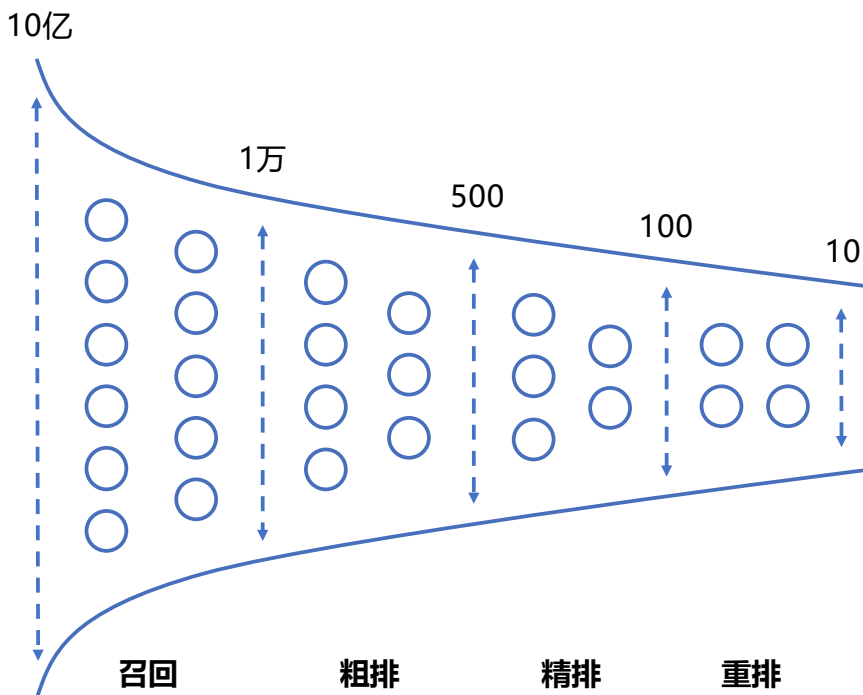


图 12.1: 推荐系统级联式架构示意图。

如图 12.1 所示，现代推荐系统通常采用一种多阶段级联架构，整体流程可划分为四个核心阶段：**召回（Retrieval）**、**粗排（Pre-Ranking）**、**精排（Ranking）**和**重排（Re-Ranking）**。这一分层设计在兼顾系统效率与推荐效果的同时，有效应对了从海量候选物品中精准筛选用户兴趣内容的挑战。

在**召回阶段**，系统需从规模庞大的物品库（例如包含数十亿甚至上百亿量级的候选物品）中快速筛选出与用户潜在兴趣相关的初步候选集。通常采用多路召回策略（如基于协同过滤、向量近邻检索、内容匹配、行为序列建模等），每一路召回各自返回若干候选物品，最终合并形成一个约 1 万量级的初始候选集合。在进入下一阶段前，系统通常会引入一个**过滤模块**，用于执行一系列业务规则和策略：例如对多路召回结果中的重复物品进行去重、剔除用户近期已交互（如点击、购买、收藏等）的物品、屏蔽存在风控或合规风险的内容，以及根据用户画像或上下文信息过滤不适宜的候选等，从而提升后续阶段的处理效率与推荐质量。

随后进入**粗排阶段**。该阶段的核心目标是在有限的计算资源下，对召回后的候选集进行初步打分与筛选。由于候选规模仍较大（通常在数千至万级），粗排模型需在效率与精度之间取得平衡，常采用轻量级但具备一定表达能力的模型结构，例如双塔模型（Two-Tower Model）或浅层神经网络。通过该模型，系统对每个候选物品计算一个粗略的相关性得分，并结合业务策略（如对不同垂类内容、运营位流量、高热/长尾物品等进行差异化加权或配额控制），确保最终传递给精排阶段的约 500 个物品在相关性、多样性与业务目标之间达到良好平衡。

在**精排阶段**，系统聚焦于对粗排筛选出的高质量候选集进行精细化建模与排序。此时，计算资源相对充裕，可部署结构更复杂、特征更丰富的深度学习模型（如深度神经网络 DNN、Wide & Deep、DeepFM、DIN 等），充分融合用户历史行为、上下文信息、物品属性、交叉特征等多维度信号，对每个候选物品进行高精度的个性化打分。精排模型的目标不仅是预测用户对单个物品的偏好程度，还需兼顾点击率（CTR）、转化率（CVR）、停留时

长、互动深度等多目标优化。最终，系统从中选出得分最高的约 100 个物品，送入下一阶段。

最后是**重排阶段**（也称“后排序”或“序列优化”）。与前几个阶段关注单点打分不同，重排更注重**整体推荐列表的序列级优化**。该阶段会综合考虑物品之间的相互影响、列表多样性、新颖性、公平性、上下文连贯性以及业务运营需求（如扶持新品、调控品类分布等），通过序列建模方法（如基于强化学习、列表级损失函数、规则引擎或轻量级 Transformer 等）对 top 100 候选进行重新排序或局部调整。最终输出一个高度优化、结构合理的推荐列表，通常仅保留 **Top 10 左右** 的物品作为最终呈现给用户的推荐结果。

综上所述，这一四阶段级联架构通过“由粗到精、层层筛选、兼顾效率与效果”的设计思想，有效支撑了大规模推荐系统的高性能与高精度需求，已成为工业界主流的推荐系统范式。

12.1 召回模块

召回模块是推荐系统流水线中的**首个关键阶段**，其核心目标是从**海量候选物品库**（通常规模达百万至亿级）中**高效筛选出一个相对较小**（通常为数百至数千）的**初步候选集**，作为后续排序 (Ranking) 和重排 (Re-Ranking) 阶段段的输入。由于全量计算用户与所有物品的相关性在工程上不可行，召回阶段并不追求极致的精准度，而是以**高效率、低延迟、高覆盖率**为设计原则，从**最粗粒度**上圈定用户可能感兴趣的物品范围。

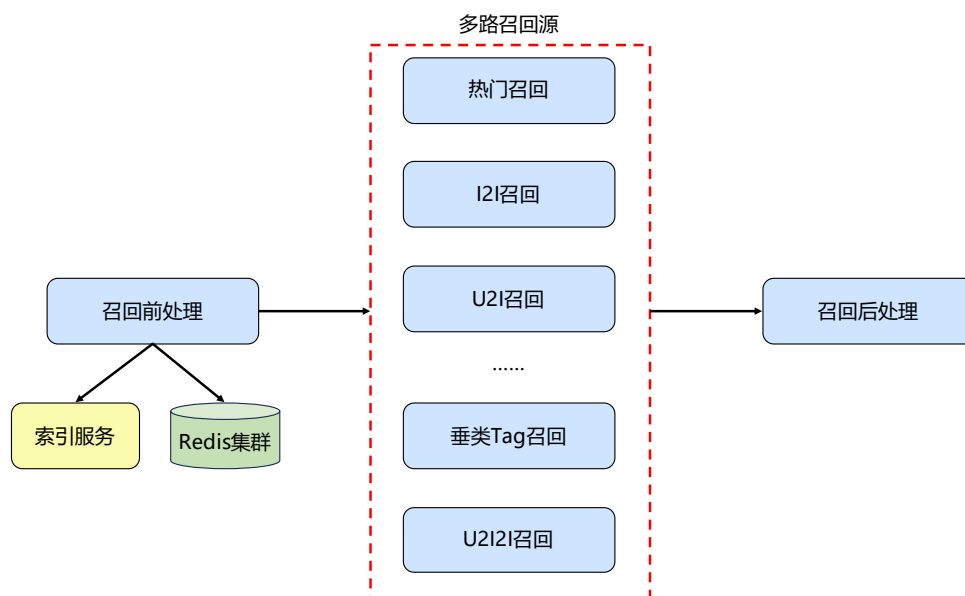


图 12.2: 召回模块内部处理流程。

在工业级推荐系统中，召回通常采用**多路召回 (Multi-channel Recall) 架构**，即并行执行多种异构的召回策略，以兼顾多样性、个性化与业务目标。如下图12.2所示，整个召回流程可分为三个主要环节：**召回前处理、多路召回执行、召回后处理**。

召回前处理 (Pre-retrieval Processing) 环节是在正式发起召回请求之前，系统需完成一系列前置准备，主要包括以下步骤：

- **构建召回所需的触发信号 (Triggers)**：这些触发信号可能来源于用户的近期行为序列 (Action List)，如点击、浏览、收藏、购买等；也可能是物品侧的一些标签、属性等字段信息。
- **获取辅助特征**：例如用户画像（性别、年龄、兴趣标签）、上下文信息（时间、地理位置、设备类型）等；
- **访问外部存储服务**：如从 Redis 集群中读取用户最近交互的物品 ID 列表，或从倒排索引服务中查询与特定标签、类目、关键词关联的物品集合。

值得注意的是，在推荐系统的索引服务体系中，通常分为两类：

- **正排索引 (Forward Index)**：以物品 ID 为键，存储其完整特征（如标题、类目、向量等），主要用于推荐模型中的特征输入或者具体推荐策略的字段依赖；

- **倒排索引 (Inverted Index)**: 以某种“触发项”(如用户 ID、物品 ID、标签、类目、关键词等) 为键, 映射到一组相关物品 ID 列表。注意这里的触发项不一定是一个字段, 也可能是多个字段, 比如“人群类别 + 物品垂类”、“国家 + 物品关键词”等。在召回阶段, 倒排索引服务被广泛使用。每个触发项 (Trigger) 所对应的物品列表, 在工程实践中常被称为“**倒排拉链 (Inverted List)**”。例如, 一个物品 A 的“相似物品”列表、一个用户标签“科技爱好者”所关联的商品集合, 都是典型的倒排拉链。

完成前处理后, 就会进行多路召回执行 (Multi-channel Retrieval Execution) 环节, 这时系统会并行发起多路召回请求, 每一路对应一种召回策略。常见的召回方式包括但不限于:

- **热门召回 (Hot/Popular Retrieval)**: 基于全局或分场景的热度统计 (如点击率、曝光量), 保障基础流量分发与冷启动覆盖;
- **I2I 召回 (Item-to-Item Retrieval)**: 基于物品协同过滤 (Item-collaborative filtering, Item CF), 根据用户当前或历史交互的物品, 召回其相似物品;
- **U2I 召回 (User-to-Item Retrieval)**: 通过用户向量与物品向量的近似最近邻 (Approximate Nearest Neighbors, ANN) 搜索实现, 通常由双塔模型生成 Embedding 后通过 Faiss、ScaNN 等库检索;
- **垂类/标签召回 (Tag/Vertical Retrieval)**: 基于用户兴趣标签或物品类目标签进行匹配, 适用于内容型或垂类平台;
- **U2U2I 召回 (User-to-User-to-Item Retrieval)**: 先找相似用户 (User-collaborative filtering, User CF), 再聚合其喜欢的物品;
- **U2I2I 召回 (User-to-Item-to-Item Retrieval)**: 结合用户历史行为与 I2I 链路, 进行两跳扩展, 增强多样性;
- **实时行为召回 (Real-time Action Retrieval)**: 基于用户当前会话的即时行为 (如搜索词、页面停留) 动态触发相关召回;
- **图神经网络召回 (Graph Neural Network Retrieval)**: 利用用户-物品交互图学习高阶关系, 生成更丰富的 embedding 用于向量召回。

由于每路召回需在大规模候选集中进行计算或检索, 并行化执行是降低端到端延迟的关键手段。现代推荐系统通常通过异步 I/O、协程或微服务编排 (如 gRPC 并行调用) 实现高效并发, 确保在数十毫秒内完成全部召回。

召回后处理 (Post-retrieval Processing) 环节主要是在多路召回的结果汇总后进行的轻量级后处理, 主要包括如下几个常见的操作:

- **去重 (Deduplication)**: 由于不同召回路可能返回相同物品 (如热门商品同时出现在热门召回和 I2I 召回中), 需对候选集进行 ID 级去重;
- **基础过滤 (Basic Filtering)**: 如剔除已下架、违规、用户已购买/屏蔽的物品, 或根据当前页面上下文 (如“猜你喜欢”vs“购物车推荐”) 应用简单规则;
- **截断 (Truncation)**: 若总候选数超过排序模型的输入上限 (如 1000 个), 可按路优先级或相关性分数进行截断。

需要强调的是, 复杂的业务规则、多样性控制、公平性约束等深度过滤逻辑, 通常下沉到后续的“过滤模块”中处理, 以保持召回模块的轻量化与高吞吐特性。

12.1.1 经典召回范式

由于多路召回执行阶段通常会同时触发数十甚至上百路异构的召回策略, 工业级推荐系统在长期实践中逐步演化出多种系统化的召回范式。这些范式可以从不同维度进行归纳, 常见的分类视角主要有以下两类:

第一, 从用户与物品之间的交互路径或关系链角度划分, 可将召回方法抽象为若干典型模式, 例如:

- **U2I (User-to-Item)**: 直接建模用户与物品的关联, 如基于用户向量的 ANN 检索;
- **I2I (Item-to-Item)**: 基于物品间的相似性进行扩展, 如 Item-CF 或语义相似商品推荐;
- **U2U2I (User-to-User-to-Item)**: 通过相似用户间接关联物品, 体现“物以类聚, 人以群分”的协同思想;
- **U2I2I (User-to-Item-to-Item)**: 结合用户历史行为与物品相似性进行两跳泛化, 在保持相关性的同时提升多样性。

这类划分方式直观反映了召回的**触发逻辑与数据流动路径**，便于理解各路召回在推荐链路中的角色，也常用于系统设计时的策略命名与调试追踪。

第二，从**技术实现与算法演进**的角度划分，则更能体现召回方法的**底层原理与发展脉络**。主流的召回范式可归纳为以下几类：

- **协同过滤召回 (Collaborative Filtering, CF)**：依赖用户-物品交互行为，包括经典的 Item-CF 与 User-CF，是早期推荐系统的核心召回手段；该方法主要是针对 User-Item 或者 Item-Item 的显示共现关系来进行推荐。
- **内容与规则召回 (Content & Rule-based Retrieval)**：基于物品属性、用户画像或人工设定的业务规则（如热门、地域、时效、运营位等），在冷启动和可控性方面具有不可替代的作用。
- **向量召回 (Embedding-based Retrieval)**：借助深度学习模型（如双塔 DSSM、序列模型、多任务学习等）将用户与物品映射至统一语义空间，再通过高效 ANN 检索（如 Faiss、ScaNN）实现大规模个性化召回，已成为当前工业界的主流范式。
- **图神经网络召回 (Graph Neural Network Retrieval)**：在用户-物品异构图上建模高阶关系，通过 GNN（如 LightGCN、PinSage）学习更丰富的结构化表示，进一步提升召回的泛化能力与关联深度。

这两种分类视角并非互斥，而是**互补共存**：前者关注“如何触发”，后者关注“如何实现”。在实际系统中，一个具体的召回路往往同时具备某种路径特征（如 I2I）和某种技术属性（如基于 Item-CF 或基于图嵌入）。理解这两种分类维度，有助于工程师在设计召回策略时既把握业务逻辑，又选对技术方案，从而构建出高效、灵活且可扩展的多路召回体系。

在本书中，我们对召回范式的分类以**技术实现与算法演进为主线**，而非仅依据用户-物品的交互路径。这种分类方式更能体现推荐系统从规则驱动到数据驱动、从浅层模型到深度学习的技术发展脉络，也便于读者系统地理解各类召回方法的核心思想、适用边界与工程落地形态。正如前文所述，尽管“U2I”、“I2I”等路径描述在工程实践中常用于命名和调试，但其背后往往依赖于不同的技术实现（例如，U2I 可由双塔模型或 GNN 等模型实现）。因此，以技术范式为纲，辅以典型触发路径为目，能够更清晰地构建召回模块的知识体系。上述四类主流召回技术的对比如下表 12.1 所示，涵盖了其核心技术、典型触发路径、优缺点及适用场景，为后续章节中具体策略的设计与选型提供参考。

表 12.1: 召回技术分类对比

召回类型	核心技术	典型的触发路径	优点	缺点	适用场景
协同过滤	Item-CF / User-CF	I2I, U2U2I	可解释、无需特征	冷启动差、稀疏敏感	行为丰富平台（电商、视频）
内容/规则	标签匹配、热度统计	Tag、热门、地域	冷启动友好、可控	个性化弱	新用户多、运营强场景
向量召回	双塔、序列模型	U2I	泛化强、支持多特征	依赖训练、黑盒	主流个性化推荐
图召回	GNN、路径游走	U2I2I、U2U2I	捕捉高阶关系	计算复杂	社交、复杂关系场景

在表 12.1 中，内容/规则召回通常不依赖模型，而是一种纯业务导向的推荐策略，在工业界应用中较为常见。一般来说，召回方法在实际落地时都包含离线部分和在线部分：离线部分主要涉及数据的统计分析或模型的训练；在线部分则通常指模型的实时推理或召回策略的即时执行。以 Tag 召回为例，这种策略导向的召回方式具体实现如下图 12.3 所示。在离线阶段，系统以用户偏好的兴趣标签（Tag）作为触发信号，通过离线计算任务预先产生每个兴趣 Tag 对应的候选物品列表。这些物品的选择可以基于物品的后验表现，例如曝光量 Top K、点赞数 Top K、评论数 Top K 等指标；同时，也可以引入一定比例的随机采样物品，以避免候选集过度集中于热门内容，从而缓解因推荐系统自身偏差（bias）导致的同质化问题。完成离线计算后，系统将每个兴趣 Tag 及其对应的物品列表以键值对（K-V）的形式存储在 Redis 缓存中。在线上阶段，系统根据用户当前的多个兴趣偏好 Tag，直接从 Redis 中查询对应的物品列表，并将多个 Tag 的召回结果进行合并，即可高效完成整个在线召回过程。

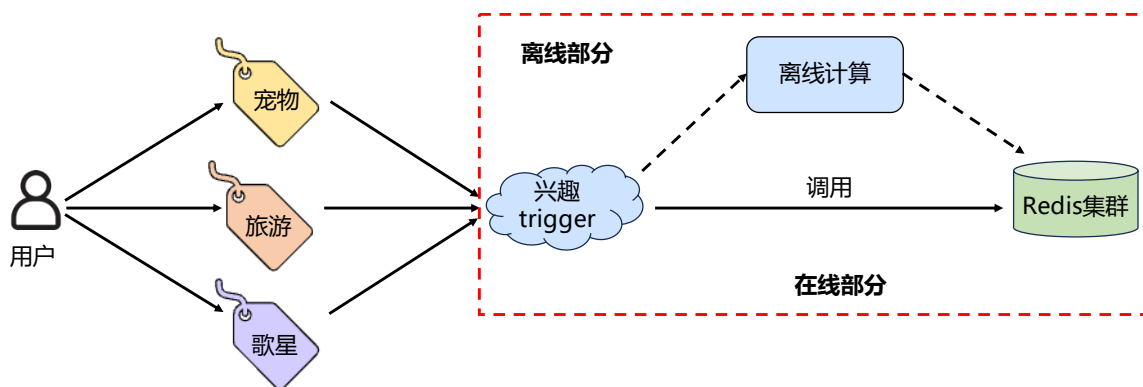


图 12.3: 用户兴趣 Tag 召回示意图。

12.1.2 向量召回 ANN 算法

最后，需要强调的是，在上述各类召回范式中，**向量召回**（Embedding-based Retrieval）已成为当前工业级推荐系统的主流技术路径。其核心思想是将用户与物品分别映射为低维稠密向量（即 Embedding），并在统一的语义空间中度量相似性。然而，面对亿级甚至十亿级的物品库，若对每个查询向量执行全量精确最近邻搜索（Exact Nearest Neighbor Search），其计算与存储开销将难以承受。为此，工业界普遍采用**近似最近邻搜索**（Approximate Nearest Neighbor Search, ANN）技术，在可接受的精度损失下，实现毫秒级、高吞吐的向量检索。

ANN 并非单一算法，而是一类高效检索框架的统称。近年来，多个高性能 ANN 库相继涌现，极大地推动了向量召回在大规模系统中的工程落地。其中最具代表性的包括：

- **Annoy**（Approximate Nearest Neighbors Oh Yeah）：由 Spotify 开源，基于随机投影树（Random Projection Trees），以轻量、易部署著称，适用于中小规模场景。代码地址为 <https://github.com/spotify/annoy>。
- **Faiss**：由 Meta（原 Facebook AI）开发，支持多种索引结构，如倒排文件（Inverted File, IVF）、乘积量化（Product Quantization, PQ）、分层可导航小世界图（Hierarchical Navigable Small World, HNSW），并提供 GPU 加速能力，功能全面，是当前最广泛使用的 ANN 库之一。代码地址为 <https://github.com/facebookresearch/faiss>。
- **ScaNN**（Scalable Nearest Neighbors）：由 Google Research 提出，通过各向异性向量量化（Anisotropic Vector Quantization）与高度优化的 CPU 指令，在同等召回率下显著优于传统乘积量化方法，尤其适合超大规模、低延迟的在线服务。代码地址为 <https://github.com/google-research/google-research/tree/master/scann>。

上述这些库之所以强大，关键在于其底层集成了多种经典的 ANN 算法。其中，**乘积量化（PQ）**与**分层可导航小世界图（HNSW）**是两类最具代表性的方法，分别从**量化压缩**与**图结构导航**两个维度，解决了高维向量检索中的效率与精度难题。

12.1.2.1 PQ 算法

乘积量化（Product Quantization, PQ）最早由 Jégou 等人在 2011 年发表于 *IEEE TPAMI* 的论文《Product Quantization for Nearest Neighbor Search》[17] 中系统提出。该方法的核心动机源于一个朴素但深刻的观察：**高维空间虽难以直接压缩，但若将其分解为若干低维子空间，则每个子空间均可独立进行高效向量量化。**

具体而言，给定一个 D 维向量 x ，PQ 将其均匀拆分为 m 个子向量 $u_1(x), \dots, u_m(x)$ ，每个维度为 $D^* = D/m$ 。随后，对每个子空间独立训练一个小型码本（通常通过 k -means 聚类获得），整体码本则定义为这 m 个子码本的笛卡尔积。尽管总码字数量可达 k^m ，但实际仅需存储 $m \cdot k$ 个中心向量，实现了指数级的存储压缩。

一个数据库向量 y 的 PQ 编码即为长度为 m 的索引序列 $[i_1, i_2, \dots, i_m]$ ，其中 i_j 表示其第 j 个子向量在对应子码本中最邻近的聚类中心索引。例如，当 $m = 8$ 、 $k = 256$ 时，编码仅需 $m \times \log_2 k = 64$ 比特，却能有效保留原始向量的几何结构。

PQ 算法的关键创新在于其**非对称距离计算**（Asymmetric Distance Computation, ADC）机制：查询向量 x 保

持原始形式，而数据库向量 y 以其量化重建值 $q(y)$ 表示。两者之间的欧氏距离被近似为：

$$\tilde{d}(x, y) = \|x - q(y)\| = \sqrt{\sum_{j=1}^m \|u_j(x) - c_{j,i_j}\|^2}, \quad (12.1)$$

其中 c_{j,i_j} 是 y 在第 j 个子码本中的重建中心。该计算可通过预计算的查找表高效实现——对每个子空间，预先计算 $u_j(x)$ 到所有 k 个中心的距离，搜索时仅需 m 次查表与累加，避免浮点运算，极大提升速度。

为进一步应对十亿级规模的数据，PQ 常与倒排文件（Inverted File）结合，形成 **IVFADC** 结构：先通过粗粒度 k -means 将空间划分为若干簇，每个向量以其相对于簇中心的**残差向量**进行 PQ 编码；查询时仅访问最近的 w 个簇对应的倒排列表。这一策略不仅将搜索范围大幅缩小，还因残差能量更低而提升了量化精度。

值得注意的是，PQ 所体现的“分而治之 + 向量量化”思想，已超越 ANN 领域，广泛影响了后续多个方向。例如，图像生成模型 VQ-VAE [34] 与 RQ-VAE [21] 均基于向量量化构建离散隐空间；而在推荐大模型与生成式推荐系统中，这类离散表征方法也正发挥着日益重要的作用。因此，理解 PQ 不仅有助于掌握 ANN 的底层机制，更能为跨领域的算法设计提供思想启发。

12.1.2.2 HNSW 算法

如果说 PQ 是从**压缩视角**解决高维检索问题，那么 **HNSW**（Hierarchical Navigable Small World）则是从**图结构视角**提供了另一种强大范式。该方法由 Malkov 与 Yashunin 在 2018 年发表于 *IEEE TPAMI* 的论文《Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs》[28] 中提出，旨在克服传统近邻图方法在低维或高度聚类数据上性能急剧下降的瓶颈。

HNSW 的设计灵感融合了两个经典结构：**可导航小世界网络（NSW）**与**跳表（Skip List）**。其核心在于构建一个多层图结构，每一层对应不同尺度的空间划分。每个数据点被随机分配一个最大层数 l ，服从指数衰减分布（ $P(l) \propto e^{-l/m_L}$ ），使得大多数点仅存在于底层，而少数“枢纽”点贯穿高层。在每一层，点仅与局部近邻建立连接：高层图稀疏、连接跨度大，用于快速“粗定位”；底层图稠密、连接精细，用于最终“精调”。

HNSW 算法的搜索过程是自顶向下进行的。首先从全局入口点出发，在顶层贪心遍历以快速逼近目标区域；一旦达到局部极小值，便“下潜”至下一层，以此类推，直至底层返回结果。具体搜索过程如下图 12.4 所示。这种“由粗到精”的策略有效避免了传统近邻图中因局部最优陷阱导致的失败，尤其在高度聚类数据上表现卓越。

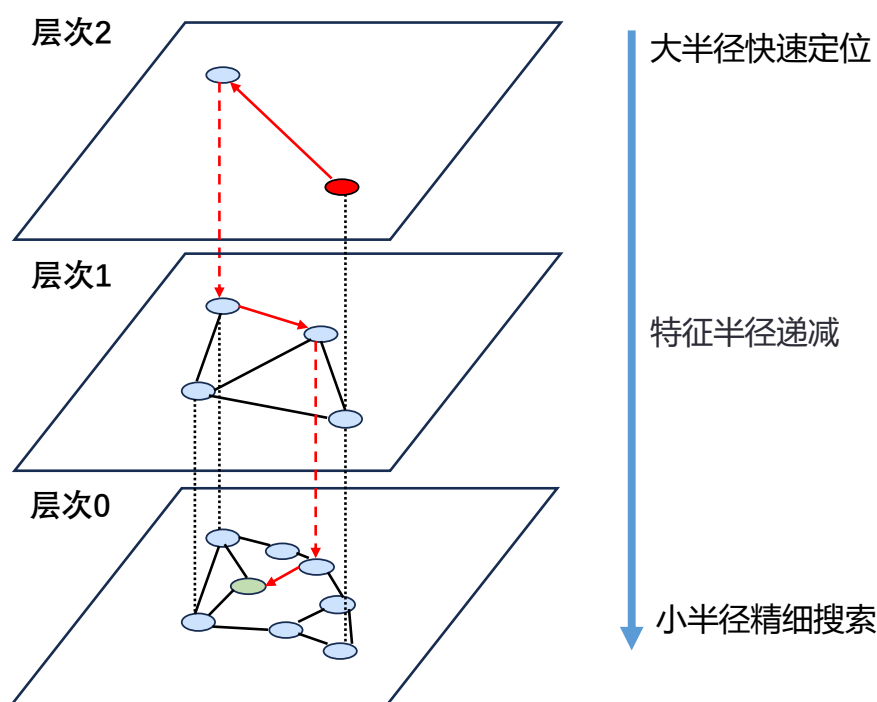


图 12.4: HNSW 近似最近邻搜索算法示意图。

更进一步，HNSW 在构建邻接关系时引入了**多样性启发式**（diversity heuristic）：并非简单连接最近的 M 个点，而是优先保留彼此距离较远的邻居，从而显著增强图的**全局连通性**，防止不同簇之间因缺乏跨簇边而割裂。值得一提的是，HNSW 是**完全基于图结构、无需辅助索引、适用于任意度量空间**的方法；无论是欧氏距离、余弦相似度，还是编辑距离、JS 散度等非欧度量，均可直接支持。其实验表现亦极为出色：在多个公开数据集上显著优于 FLANN、Annoy、LSH 及原始 NSW；在十亿级 SIFT 向量的检索中，甚至以更快的构建速度和更高精度超越了基于 PQ 的 Faiss。从结构上看，HNSW 可视为**跳表在高维无序空间中的自然推广**：跳表通过多层指针实现一维有序序列的快速查找，而 HNSW 则通过多层近邻图实现高维空间的高效导航。这一类比不仅揭示了其对数级搜索复杂度的根源，也为未来分布式实现提供了理论基础。

从 Annoy 的轻量树结构，到 Faiss 的多功能索引体系，再到 ScaNN 的极致 CPU 优化，ANN 技术的演进清晰地反映了推荐系统对**实时性、规模性与成本效益**的持续追求。而 PQ 与 HNSW 作为其中两大支柱，分别代表了**量化压缩与图导航**两种互补的技术路线：前者擅长内存受限、需极致压缩的场景，后者则在高召回、强鲁棒性需求下更具优势。

在实际工程中，二者也常被结合使用（如 Faiss 中的 IVF+HNSW 或 PQ+HNSW 混合索引），以兼顾速度、精度与资源消耗。更重要的是，理解这些底层算法的思想，有助于我们在面对新问题时进行**跨领域迁移与创新**。无论是设计新型召回模型，还是构建生成式推荐系统，向量表示与高效检索始终是核心基础。在后续章节中，我们将结合具体召回算法模型，进一步探讨如何基于这些 ANN 工具构建**高效、稳定、可扩展的向量召回服务**，并深入分析其在真实工业场景中的部署策略与优化技巧。

12.2 过滤限流模块

在现代推荐系统中，过滤限流模块扮演着“守门人”的关键角色。其核心使命是在内容进入召回、排序乃至最终曝光链路之前，主动识别并拦截**不合法、低质量或高风险**的内容，从而确保推荐结果既符合法律法规与平台社区准则，又能维护良好的用户体验与健康的内容生态。该模块以实际业务需求为导向，承担着对推荐内容生态进行主动调控的重要职责，并对内容的安全性、合规性实施严格把关。其作用贯穿内容生命周期的多个关键环节，尤其在**实时推荐流与批量内容入库**两大路径中尤为关键。为应对复杂多变的风险场景，过滤模块并非依赖单一手段，而是有机融合了**静态规则过滤、模型驱动判罚与人工审核兜底**三层机制，构建起一套多层次、高鲁棒性的内容风控体系。

过滤的依据来自对内容多维度信号的综合评估，典型维度与策略如表12.2所示。在这一框架中，**负向判罚子模块**构成了连接风控系统与推荐引擎的核心枢纽。它统一收口来自内容安全、社区治理、反作弊等多个子系统的负向信号——包括机器审核标签（如“涉政-高危”、“AI 生成-疑似”）、人工复审结论（如“确认违规-限流 7 天”）、用户举报聚合结果（如“24 小时内举报超 50 次且通过率 >80%”），以及行为异常检测（如同一 IP 批量发布相似视频）。基于信号的风险等级与时效性要求，系统执行差异化的处置策略：对高危内容实施**硬过滤**，直接阻断其进入任何推荐链路；对中低风险内容则采用**软限流**，如降低曝光权重、随机按比例过滤、限制分发范围（仅对粉丝可见或仅创作者自己可见、在大流量页面完全不分发等）、或延迟进入推荐池以待审核侧二次复核；对于特定群体敏感的内容，可进行**定向屏蔽**（如对未成年人隐藏成人内容等）；而对于边界案例，则通过**打标透传**，将风险标签传递至排序模型，由后续阶段综合决策。所有判罚策略均支持动态配置与实时生效，并通过统一的策略管理平台实现版本控制、灰度发布与效果回溯，确保策略迭代的敏捷性与可审计性。

值得注意的是，过滤限流模块并非孤立运行，而是深度嵌入推荐系统的整体架构之中。在**召回阶段之前**，它对候选内容池进行预过滤，有效减少无效计算；在**粗排与精排阶段**，过滤限流标签作为关键特征输入排序模型，辅助其学习平台的合规偏好；在**重排与兜底阶段**，系统还会结合实时风控信号进行最终校验，防止漏网之鱼的内容被分发出去。此外，该模块与日志监控、AB 实验平台及离线评估系统紧密联动，持续追踪过滤限流策略对核心业务指标（如点击率、用户留存、举报率）的影响，从而在“安全”与“体验”之间实现动态平衡。

需要特别说明的是，在实际工程实践中，过滤限流模块所依赖的绝大多数违规标签（如“封面涉黄”、“标题含违禁词”、“内容涉政”等）均由**风控团队**通过规则引擎、专用模型或 AI 机审系统产出。推荐系统通常只需接入这些标签，并依据预设策略执行相应操作，而无需关心标签背后的生成逻辑——这些逻辑属于**风控算法**的范畴。

表 12.2: 过滤限流策略示例

过滤限流维度	典型规则示例	过滤限流策略
封面/画面合规	封面含色情裸露、低俗暗示、血腥暴力、恐怖元素、敏感标识（如国旗误用）等	随机过滤 80% 的请求
标题/文本违规	包含违禁词、夸大误导、政治敏感词、虚假宣传、诱导点击等	随机过滤 80% 的请求
内容主题风险	涉及政治事件、宗教极端、民族歧视、谣言传播、非法集资、敏感社会事件等	100% 完全过滤，内容仅创作者可见
内容质量低下	画面模糊、音画不同步、纯黑屏/白屏、无实质信息、大量水印或广告插入等	随机过滤 75% 的请求，总曝光不能超过 X 次
原创性存疑	内容疑似 AI 批量生成、无真人出境、脚本高度模板化、存在高概率抄袭或洗稿行为、内容属于二次搬运内容、存在版权违规风险	全域随机过滤 60% 的请求
用户行为反馈	被大量用户举报、低完播率 + 高跳出率组合、负向互动（如踩、拉黑）集中爆发等	不同活跃度人群不同的随机过滤比例
发布者信誉	账号处于黑名单（比如黑产账号）、历史违规频发、新号无认证、批量发布相似内容等	大流量公域页面 100% 完全过滤，小流量页面随机 50% 的请求

下文将对风控算法的演进脉络作简要介绍。

以短视频场景中的**封面/画面违规识别**为例，风控算法的发展经历了从“任务专用小模型”到“统一多模态大模型”的显著演进。在大语言模型（Large Language Model, LLM）与多模态语言模型（Visual Language Model, VLM）普及之前，违规识别高度依赖**传统计算机视觉方法**。针对每一类风险（如色情、暴力、血腥、敏感标识等），算法团队需分别收集标注数据、设计特征、训练独立的分类模型。典型做法包括：使用 **MobileNet**[14] 或 **EfficientNet**[32] 进行低延迟初筛；用 **ResNet**[13] 或 **ViT**[9] 对高风险样本做精细判别；结合 **OpenCV** 人体检测与 **姿态估计模型** 判断是否过度裸露；并通过 **肤色区域占比**、**人脸年龄估计**（识别未成年人出境风险）、**视频帧一致性分析**（辅助 **deepfake** 辨别）等信号增强判断鲁棒性。这类方法虽在特定任务上可达到较高精度，但存在明显局限：**模型碎片化严重、迭代成本高、跨任务泛化能力弱**。每当新增一类违规类型（如“AI 换脸造假”），往往需从头构建数据 pipeline 与训练流程。

随着多模态大模型（如 **CLIP** [30]、**BLIP** [22]、**Qwen-VL** [1] 等）的兴起，这一局面被彻底改变。多模态语言模型能够将图像、文本、元数据等异构信号统一编码至共享语义空间，从而在一个统一框架下处理**多种违规任务**。例如，利用 **CLIP** 计算封面图与“色情”“暴力”等关键词的图文相似度；通过 **VLM** 联合理解视频画面、标题、评论的上下文语义，识别隐晦违规（如“擦边球”内容）；在图像与文本信号冲突时（如画面合规但标题含性暗示），由 **LLM** 进行语义仲裁并生成可解释的拒审理由。这种范式不仅显著提升了识别精度与泛化能力，还大幅降低了模型维护复杂度，**无需为每个子任务单独训练模型**，只需通过 **prompt** 工程或少量微调即可适配新场景。上述技术演进不仅体现了算法能力的跃迁，更反映了风控理念从“规则驱动、任务割裂”向“语义理解、统一建模”的深刻转变。

尽管当前过滤限流机制已相对成熟，但仍面临诸多挑战。**AI 生成内容**的快速演化使得传统检测模型日益滞后；跨模态违规（如画面合规但语音含敏感信息）对多模态理解能力提出更高要求；而误伤优质长尾内容的风险，则迫使系统在精度与召回之间寻求更精细的权衡。面向未来，过滤模块正朝着**智能化**（引入大语言模型进行语义级违规理解）、**个性化**（基于用户容忍度动态调整过滤强度）与**全球化**（支持多语言、多地区合规标准）方向持续演进。它不仅是技术防线，更是平台价值观的体现，正日益成为推荐系统不可或缺的“安全基石”。

值得一提的是，本节所提及的 **ViT**、**CLIP**、**BLIP**、**Qwen-VL** 等多模态模型，不仅在风控领域发挥关键作用，也正成为推荐算法的新兴前沿技术。我们将在第15章中对这些技术在推荐系统中的应用进行详细探讨。

12.3 粗排模块

粗排层负责对召回结果进行初步打分与排序，快速过滤掉明显不相关的物品。由于需要处理海量候选集，粗排模型通常采用计算开销较低的轻量化架构，以保证系统的高吞吐和低延迟。为了满足业务的多样化目标，粗排模型往往需要同时预测用户在不同维度上的行为倾向，例如点击、观看时长、互动、转化等指标，统称为 **pxtr** (**Predictive eXpected Through Rate**)。不同 **pxtr** 所对应的排序结果往往存在差异，而最终呈现给用户的只能是一条排序序列，因此需要通过多目标融合机制将多个目标统一到同一排序框架中。

与此同时，推荐系统还需要适应业务目标的动态变化。例如，在产品增长阶段，平台可能更加关注用户时长与活跃度；而在产品成熟阶段，则可能更强调商业化收益或长期价值。因此，无论是在粗排还是后续的精排阶段，系统都需要具备灵活的流量调控能力，通过动态调整目标权重和流量分配策略，实现业务目标的实时切换与优化。

除了目标融合与流量调控之外，推荐链路还需要满足平台生态层面的约束。例如，为保障不同业务、垂类或内容供给方的曝光机会，系统通常会引入分业务 **Quota** 控制、保量策略以及截断机制，从而在整体效率与生态公平性之间取得平衡。

总体而言，粗排阶段的核心目标并非精确刻画用户对单个物品的兴趣程度，而是从大规模候选集中进行高效的粗粒度筛选，从候选集合优化 (**Candidate Set Optimization**) 的角度出发，为后续精排阶段提供一个兼具相关性、多样性和探索性的优质候选集。具体来说，粗排不仅需要覆盖用户潜在感兴趣的内容，还需要兼顾不同垂类内容的分布、多样化供给、候选集聚集度控制以及探索性样本的引入，从而避免候选集过度集中于单一类型内容，为后续排序阶段保留更大的优化空间。

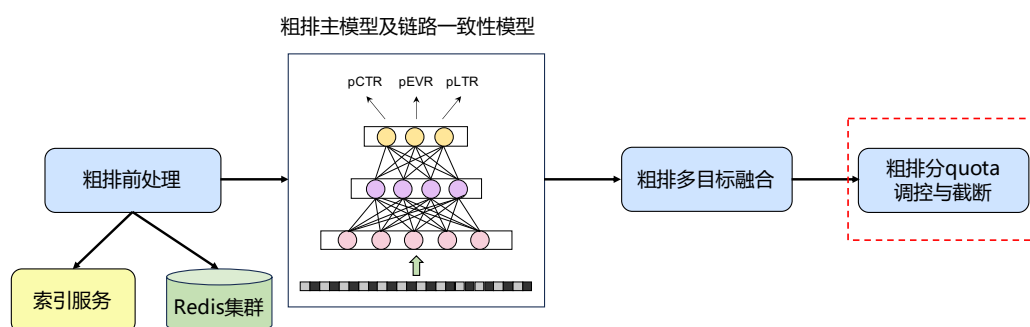


图 12.5: 粗排模块内部处理流程。

12.3.1 常用的模型预估值

这里首先对推荐系统中的一些常见术语进行简要介绍，尤其是在级联式 (**Cascade**) 推荐架构下，粗排、精排乃至重排阶段模型所输出的各种预估值（业内通常称为“**pxtr**”）及其对应的业务含义和计算方式。

在工业级推荐系统中，推荐模型本质上是为用户目标服务的。然而，实际业务场景中的优化目标往往并非单一指标，而是多个目标之间的综合权衡与协同优化。例如，平台既希望提升用户消费时长，也希望提高互动率、转化率以及商业化收益等指标。与此同时，用户在 APP 中会产生多种不同类型的行为反馈，如曝光、点击、观看、点赞、评论、分享、关注、购买等，而这些行为通常反映了用户不同层次的兴趣偏好和产品心智。

业务侧的多目标需求与用户侧的多行为反馈共同决定了现代推荐模型中通常会采用 **多任务学习 (Multi-Task Learning, MTL)** 的建模范式，即通过一个统一模型同时预测多个用户行为目标。因此，在推荐系统的各个排序阶段，模型往往会输出大量不同类型的预估值，用于刻画用户发生某种行为的概率或期望收益。

业界通常将这些预估值统称为 **pxtr (Predictive X Through Rate)**，其中 (X) 代表具体的用户行为类型。例如，点击对应 **CTR (Click-Through Rate)**、点赞对应 **LTR (Like-Through Rate)**、评论对应 **CTR (Comment-Through Rate)** 等。表 12.3 汇总了推荐系统中常见的 **pxtr** 指标、其业务含义以及对应的计算公式。

表 12.3: 各类用户行为预估 (pxtr) 的定义及计算公式

缩写	定义及计算公式
CTR	点击率 (Click-Through Rate): 用户点击行为占曝光次数的比例。 计算公式: $CTR = \frac{\text{点击次数 (Clicks)}}{\text{曝光次数 (Impressions)}} \times 100\%$
LTR	评论率 (Like Rate): 用户点赞行为占曝光次数的比例。 计算公式: $LTR = \frac{\text{点赞次数 (Likes)}}{\text{曝光次数 (Impressions)}} \times 100\%$
CMR	评论率 (Comment Rate): 用户评论行为占曝光次数的比例。 计算公式: $CMR = \frac{\text{评论次数 (Comments)}}{\text{曝光次数 (Impressions)}} \times 100\%$
WTR	关注率 (Follow Rate): 用户关注行为占曝光次数的比例。 计算公式: $WTR = \frac{\text{关注次数 (Follows)}}{\text{曝光次数 (Impressions)}} \times 100\%$
EVR	有效播放率 (Effective View Rate): 有效播放次数占总播放次数的比例 (通常以播放时长达到阈值为有效标准)。 计算公式: $EVR = \frac{\text{有效播放次数 (Effective Plays)}}{\text{总播放次数 (Total Plays)}} \times 100\%$
LVR	长播率 (Long View Rate): 长时播放次数占总播放次数的比例 (通常以播放完成为长播标准)。 计算公式: $LVR = \frac{\text{长时播放次数 (Long Plays)}}{\text{总播放次数 (Total Plays)}} \times 100\%$
VTR	播放时长率 (Video Time Rate): 平均播放时长占内容总时长的比例。 计算公式: $VTR = \frac{\text{总播放时长 (Total Watch Time)}}{\text{内容总时长 (Content Duration)} \times \text{播放次数 (Plays)}} \times 100\%$
CVR	转化率 (Conversion Rate): 用户完成目标转化行为 (如购买、下载) 占点击次数的比例。 计算公式: $CVR = \frac{\text{转化次数 (Conversions)}}{\text{点击次数 (Clicks)}} \times 100\%$
GTR	打赏率 (Gratuity Rate): 用户打赏行为占曝光次数或播放次数的比例。 计算公式: $GTR = \frac{\text{打赏次数 (Gratuities)}}{\text{曝光次数 (Impressions) 或播放次数 (Plays)}} \times 100\%$

从表 12.3 可以看出, 不同的 pxtr 指标往往对应不同的业务场景和用户行为。例如, CTR (Click-Through Rate) 通常应用于存在显式点击行为的推荐场景, 如资讯流、图文社区以及内容详情页推荐等。在这类场景中, 点击行为能够直接反映用户对推荐内容的兴趣程度。而在短视频上下滑场景中, 用户通常无需通过点击进入内容, 因此点击行为本身并不存在或意义较弱。此时, CTR 所承担的兴趣刻画作用往往会被有效观看率 (EVR, Effective View Rate)、观看时长 (Watch Time) 等指标所替代, 用于衡量用户对视频内容的实际消费意愿和兴趣程度。类似地, GTR (Gift Through Rate) 更多出现在直播推荐场景中, 用于预估用户观看某场直播后产生打赏行为的概率, 从而刻画用户对于主播及直播内容的付费意愿和商业价值。

总体来看, 这些 xtr 指标本质上都是通过用户的显式反馈 (如点击、点赞、评论、分享、关注、购买、打赏等) 以及隐式反馈 (如观看时长、停留时长、滑动行为等) 来衡量用户对推荐内容的兴趣程度, 并进一步指导推荐系统的排序决策。然而, 从建模角度来看, 不同目标之间的数据分布存在显著差异。相比于曝光和观看等高频行为, 点击 (CTR)、转化 (CVR)、打赏 (GTR) 等行为往往具有极强的稀疏性, 其正样本占比通常远低于前者。这种数据稀疏问题不仅增加了模型训练的难度, 也会导致预估结果存在较大的方差和偏差, 使得模型容易出现概率高估 (Over-estimation) 或低估 (Under-estimation) 的现象。

因此, 在工业级推荐系统中, 除了提升模型结构和特征表达能力之外, 如何解决样本稀疏、概率校准 (Calibration)、数据偏置以及多目标协同学习等问题, 也成为提升 pxtr 预估精度的重要研究方向。

12.3.2 粗排主模型与链路一致性模型

在级联式推荐系统发展的早期阶段, 粗排模块通常仅包含一个粗排主模型, 用于对召回阶段返回的大规模候选集进行初步筛选。由于粗排阶段面对的候选集规模通常在数千级别 (约 3000~5000 个候选物品), 而工业级推荐系统对于整体请求延迟往往有严格要求 (通常单次请求需控制在 500ms 以内), 因此分配给粗排模型的推理时间通常只有几十毫秒。在这样的时延约束下, 粗排模型的网络结构必须兼顾效果与计算效率, 难以采用过

于复杂的建模方式。

工业界早期广泛采用的粗排模型是**双塔（Two-Tower）架构**。该架构由**用户塔（User Tower）**和**物品塔（Item Tower）**组成，分别接收用户侧特征和物品侧特征作为输入，并独立学习用户和物品的表征向量。最终通过用户向量与物品向量之间的点积计算得到对应的预估分数。当模型需要同时预测多个用户行为目标时，通常会为不同任务构建独立的任务塔（Task Tower），分别输出对应行为的用户表征和物品表征，并计算得到多个不同的 **pxtr** 预估值。

然而，随着推荐系统的快速发展，工业界逐渐意识到双塔结构虽然具备较高的计算效率，但其表达能力存在明显局限。一方面，用户侧特征与物品侧特征在编码阶段彼此独立，缺乏充分的特征交叉与交互建模能力，导致模型难以捕获复杂的用户兴趣模式；另一方面，由于粗排阶段对时延要求极高，其模型复杂度远低于后续精排模型，因此粗排输出的 **pxtr** 往往难以达到精排阶段的预测精度。

除此之外，级联式推荐架构还面临另一个重要问题：不同排序阶段通常由不同团队独立负责优化，各阶段模型的训练目标、样本构建方式以及优化方向往往并不完全一致。因此，粗排阶段筛选出的候选集与精排阶段真正偏好的物品之间可能存在较大的分布偏差。这种偏差会导致大量精排偏好的优质物品在粗排阶段被提前过滤，从而限制整个推荐链路的最终效果。

为了解决上述问题，工业界开始关注推荐链路中的一致性（Consistency）优化，并逐渐形成了**粗排链路一致性模型（Consistency Model）**或**粗排级联学习模型（Cascade Learning Model）**的技术方案。其核心思想是利用下游精排模型的排序结果作为监督信号，引导粗排模型学习精排阶段的排序偏好。例如，可以将精排 **Top-K** 物品构造为正样本，将精排候选集中未进入 **Top-K** 的物品构造为负样本，通过二分类学习的方式训练粗排模型。这样一来，粗排模型无需引入额外复杂的网络结构，便能够在有限算力预算下学习到部分精排模型所蕴含的高阶排序能力，从而提升上下游排序结果的一致性。

从工程实践角度来看，粗排级联学习模型往往能够在首次上线时取得较为显著的收益。一方面，其提升了粗排与精排之间的链路一致性，减少了优质候选物品被提前过滤的概率；另一方面，它能够在不增加在线推理成本的情况下，将精排模型的部分知识蒸馏到粗排阶段，因此具有较高的投入产出比。

然而，级联学习模型的优化空间通常也存在较为明显的天花板。其根本原因在于，链路一致性本身只是系统层面的中间目标，而非最终业务目标。提高粗排与精排之间的一致性，并不必然意味着用户满意度、观看时长、互动率或商业化收益等核心业务指标会同步提升。换句话说，链路一致性的提升与业务收益的增长之间并不存在严格的因果关系。因此，当粗排模型已经能够较好地复现精排偏好后，继续提升一致性指标所带来的边际收益往往会逐渐减弱，甚至可能导致系统陷入“过度模仿精排”的局部最优状态，从而限制后续效果的进一步突破。

12.3.3 常用的模型评估指标

对于大多数 **pxtr** 任务（如 **CTR**、**CVR**、**LTR**、**FTR** 等），本质上都可以建模为二分类问题，即预测用户是否会发生某种特定行为。因此，传统机器学习中的分类评估指标同样适用于推荐模型的效果评估。然而，推荐系统最终服务于排序场景，仅仅关注分类正确率是不够的，还需要进一步衡量模型的排序能力以及概率预估能力。因此，工业界通常从分类指标、排序指标和概率校准指标三个维度对模型进行评估。

12.3.3.1 分类指标

分类指标主要用于衡量模型对于正负样本的区分能力。

- **Accuracy（准确率）**

Accuracy 表示模型所有预测结果中预测正确的比例，其计算公式为：

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

其中，**TP**、**TN**、**FP** 和 **FN** 分别表示真正例、真负例、假正例和假负例。

在推荐系统中, 由于正负样本通常极度不平衡, 例如点击率往往仅为百分之几甚至千分之几, 因此 Accuracy 通常不能真实反映模型性能, 在实际业务中较少作为核心评估指标。

- **Precision (精确率)**

Precision 表示模型预测为正样本的结果中, 真实为正样本的比例:

$$Precision = \frac{TP}{TP + FP}$$

Precision 越高, 说明模型误报的正样本越少。

- **Recall (召回率)**

Recall 表示所有真实正样本中被模型成功识别出来的比例:

$$Recall = \frac{TP}{TP + FN}$$

Recall 越高, 说明模型遗漏正样本的情况越少。

- **F1 Score**

F1 Score 是 Precision 和 Recall 的调和平均值, 其计算公式为:

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

F1 Score 能够综合衡量模型的查准率和查全率, 在需要同时兼顾误判和漏判成本的场景中具有较好的参考价值。

12.3.3.2 排序指标

相比于分类指标, 排序指标更加符合推荐系统的实际优化目标, 因此也是工业界最常使用的离线评估指标。

- **AUC (Area Under ROC Curve)**

AUC 用于衡量模型将正样本排序在负样本之前的能力。从概率角度来看, 其可以表示为:

$$AUC = P(s_{pos} > s_{neg})$$

其中, s_{pos} 和 s_{neg} 分别表示正样本和负样本的预测分数。

AUC 的取值范围为 $[0, 1]$ 。当 AUC 等于 0.5 时, 模型与随机排序等价; 当 AUC 越接近 1 时, 说明模型对正负样本的区分能力越强。

- **UAUC (User-wise AUC)**

UAUC 首先对每个用户单独计算 AUC, 然后对所有用户求平均:

$$UAUC = \frac{1}{N} \sum_{u=1}^N AUC_u$$

UAUC 能够避免少数活跃用户的大量样本对整体评估结果产生过强影响, 因此更加关注用户维度的排序效果。

- **GAUC (Group-wise AUC)**

GAUC 在计算用户级 AUC 之后, 根据用户样本量进行加权平均:

$$GAUC = \frac{\sum_{u=1}^N n_u AUC_u}{\sum_{u=1}^N n_u}$$

其中, n_u 表示用户 u 对应的样本数。

相较于 UAUC, GAUC 更加符合实际流量分布, 因此是工业界应用最广泛的排序评估指标之一。

- **WUAUC (Weighted UAUC)**

WUAUC 是在 UAUC 基础上引入业务权重后的扩展指标，其定义为：

$$WUAUC = \frac{\sum_{u=1}^N w_u AUC_u}{\sum_{u=1}^N w_u}$$

其中， w_u 可以表示曝光量、观看时长、GMV 贡献等业务价值权重。

- **NDCG (Normalized Discounted Cumulative Gain)**

NDCG 主要用于评估 Top-K 结果的排序质量，并考虑了排序位置的重要性。

首先定义 DCG：

$$DCG@K = \sum_{i=1}^K \frac{rel_i}{\log_2(i+1)}$$

其中， rel_i 表示位置 i 处物品的相关性得分。

然后利用理想排序下的 DCG（记为 IDCG）进行归一化：

$$NDCG@K = \frac{DCG@K}{IDCG@K}$$

NDCG 的取值范围为 $[0, 1]$ ，数值越大表示 Top-K 推荐结果越接近理想排序。该指标广泛应用于首页推荐、搜索排序等关注头部位置效果的场景。

12.3.3.3 概率校准指标

仅有较高的排序能力并不意味着模型能够输出准确的概率预估值。在推荐系统中，模型预测结果往往会参与后续的多目标融合、流量分配以及竞价决策，因此概率校准能力同样十分重要。

- **PCOC (Predicted Click over Observed Click)**

PCOC 用于衡量模型预测概率与真实概率之间的一致性，其计算公式为：

$$PCOC = \frac{\sum_i \hat{y}_i}{\sum_i y_i}$$

其中， $\hat{y}_i$ 表示模型预测概率， y_i 表示真实标签。

理想情况下：

$$PCOC = 1$$

当 $PCOC > 1$ 时，说明模型整体存在高估现象（Over-estimation）；当 $PCOC < 1$ 时，说明模型整体存在低估现象（Under-estimation）。

与 AUC 关注排序能力不同，PCOC 关注模型输出概率与真实概率之间的匹配程度，因此常被用于评估模型的校准效果。

需要注意的是，AUC、GAUC、NDCG 等指标主要衡量模型的相对排序能力，而 PCOC 衡量的是模型的绝对概率预估能力。一个优秀的推荐模型不仅需要具备较强的排序能力，还应当保证预测概率与真实分布尽可能一致。在工业界实践中，AUC（或 GAUC）与 PCOC 通常会被联合监控，共同评估模型质量。此外，部分公司和论文也将该指标称为 COPC（Calibration Of Predicted CTR），二者本质上都用于评估模型的概率校准能力，但具体公式定义可能存在差异，因此在实际使用时应以具体业务定义为准。

12.3.4 粗排多目标融合

在粗排主模型、链路一致性模型以及其他辅助模型完成推理之后，系统通常会得到多个不同目标对应的 $\mathbf{pxtr}$ 预估值，例如 p_{ctr} 、 p_{cvr} 、 p_{ltr} 、 p_{evtr} 以及链路一致性模型输出的排序分数等。由于最终系统只能输出一个统一的排序结果，因此需要通过多目标融合（Multi-Objective Fusion）将多个目标的预估结果融合为一个最终排序分数，并据此完成候选集排序。

12.3.4.1 线性融合

工业界最早也是最常见的融合方式是线性融合，其形式如下：

$$s_{agg} = w_1 p_{ctr} + w_2 p_{cvr} + \dots + w_n p_{cascade} \quad (12.2)$$

其中， w_i 表示对应目标的权重，反映该目标对于最终排序结果的重要程度。

线性融合具有实现简单、易于解释和便于人工调参等优点，因此长期以来一直是推荐系统中的主流融合方案。然而，其缺点也十分明显。首先，线性模型只能刻画不同目标之间的加性关系，难以表达复杂的非线性交互。例如用户的点击行为和点赞行为往往具有协同关系，仅仅对两者进行线性加权可能无法充分利用这种关联信息。因此工业界通常会进一步构造高阶交叉特征，例如： $p_{ctr} \times p_{ltr}$ ， $p_{ctr} \times p_{cvr}$ 并将这些交叉项作为新的排序因子加入融合公式之中。

在实际工程中，这种不断向融合公式中增加新排序因子的过程被策略算法工程师称为“加队列”。其本质是在已有排序体系中引入新的信息来源，希望通过额外因子捕获增量收益，从而推动业务指标提升。

另一方面，线性融合对 $\mathbf{pxtr}$ 的绝对数值高度敏感。一方面，不同模型版本输出的 $\mathbf{pxtr}$ 分布往往并不一致。即使模型排序能力接近，由于概率校准程度不同，其均值和方差也可能存在较大差异，从而导致原有融合权重失效。另一方面，不同目标本身的数值范围差异也较大。例如： $p_{evtr} \gg p_{cmtr}$ 。有效播放率通常远高于评论率，因此稀疏目标对应的权重往往更加敏感，调参难度显著增加。

此外，随着融合因子数量不断增加，线性公式会逐渐变得复杂，大量权重之间存在耦合关系，使得维护成本和调参成本不断上升。

12.3.4.2 乘法融合

为了缓解线性融合容易被单个因子主导的问题，工业界逐渐引入了乘法形式的融合策略。其典型形式如下：

$$s_{agg} = \prod_{i=1}^n (\beta_i + \alpha_i p_i)^{\gamma_i} \quad (12.3)$$

展开后可以写成：

$$s_{agg} = (\beta_1 + \alpha_1 p_{ctr})^{\gamma_1} (\beta_2 + \alpha_2 p_{cvr})^{\gamma_2} \dots (\beta_n + \alpha_n p_{cascade})^{\gamma_n} \quad (12.4)$$

与线性融合相比，乘法融合更加关注多个目标的协同表现。在线性融合下，一个物品只要在某个目标上表现极好，即有可能获得较高的最终排序分数。而在乘法融合下，如果某个关键目标表现较差，则整体分数会受到明显抑制。因此乘法融合更倾向于选择在多个目标上均表现良好的物品，从而获得更加均衡的推荐结果。

12.3.4.3 基于序关系的融合

除了利用 $\mathbf{pxtr}$ 的绝对数值进行融合之外，工业界还提出了一类基于排序序关系（Rank-based Fusion）的融合方法。该方法首先对每个 $\mathbf{pxtr}$ 因子独立排序，得到对应的排序位置：

$$Rank_{ctr}, \quad Rank_{cvr}, \quad \dots, \quad Rank_{cascade} \quad (12.5)$$

随后根据排序位置构造融合分数：

$$s_{agg} = w_1 \frac{1}{Rank_{ctr}} + w_2 \frac{1}{Rank_{cur}} + \cdots + w_n \frac{1}{Rank_{cascade}} \quad (12.6)$$

这种方法最大的优势在于其对于 **pxtr** 的具体取值范围、分布形态以及校准程度并不敏感，仅依赖于排序顺序本身，因此具有较好的鲁棒性。在实际工业系统中， $\frac{1}{Rank}$ 只是最基础的一种形式，还衍生出了大量变种。

例如幂函数形式：

$$S_i = \frac{w_i}{Rank_i^\lambda + b} \quad (12.7)$$

，归一化幂函数形式：

$$S_i = w_i \left(1 - \left(\frac{Rank_i}{N} \right)^\lambda \right) \quad (12.8)$$

，指数形式：

$$S_i = w_i \left(e^{k(2\frac{Rank_i}{N}-1)} - e^{-k(2\frac{Rank_i}{N}-1)} \right) \quad (12.9)$$

，以及带平滑项的排序衰减形式：

$$S_i = w_i \left(1 - \left(\frac{Rank_i}{N+b} \right)^\lambda \right) \quad (12.10)$$

其中， N 表示候选集大小， λ 、 k 和 b 为可调超参数。

不过，纯 **Rank** 融合也存在明显缺陷。由于只保留了排序位置，而丢失了原始 **pxtr** 的数值信息，因此无法感知不同物品之间真实的分数差异。例如排名第一和第二的物品可能分数极其接近，也可能存在数量级差异，但在 **Rank** 融合中都会被视为相邻位置。因此近年来工业界又逐渐发展出了结合 **Rank** 信息与 **Value** 信息的混合融合方法，在保持鲁棒性的同时兼顾数值信息的表达能力。

12.3.4.4 自动化融合

上述融合方式本质上都属于人工设计排序公式的方法，其效果高度依赖于算法工程师的经验以及对业务的理解。

近年来，随着 **AutoML** 和强化学习的发展，工业界开始尝试自动搜索融合参数。例如利用 **Cross-Entropy Method (CEM)**、贝叶斯优化 (**Bayesian Optimization**)、强化学习 (**Reinforcement Learning**) 等方法自动寻找最优融合参数，从而减少人工调参成本。

进一步地，研究人员开始思考一个更加根本的问题：既然最终目标是获得最优排序结果，那么是否可以不再人为设计固定的融合公式，而是直接让模型学习融合函数本身？换言之，能否将多个 **pxtr** 因子作为输入，通过一个端到端模型直接输出最终排序分数？

围绕这一问题，工业界逐渐发展出了学习式融合 (**Learning-to-Fuse**)、端到端排序 (**End-to-End Ranking**) 以及强化学习排序等一系列高级技术方案。相关内容将在第14章“推荐系统经典问题”中进一步展开讨论。

12.3.5 粗排分 quota 调控与截断

除了多目标融合与排序之外，粗排阶段通常还需要承担候选集调控 (**Candidate Set Control**) 的职责。其中最重要的一项工作便是 **Quota** 调控，即对不同类型内容、不同业务目标以及不同流量池的候选物品数量进行控制和分配。

在实际业务中，系统的优化目标往往并非单一维度。例如在直播推荐场景中，平台既希望提升用户观看时长和互动率，又需要兼顾商业化收益、内容生态健康以及主播成长等多方面目标。因此，粗排阶段通常会针对不同类型的主播构建独立的候选池，例如颜值主播池、电商带货主播池、游戏主播池等；同时还会构建用户高点击率主播池、用户关注主播池、高价值主播池、新主播池等不同流量池。随后根据用户画像、实时兴趣以及平台运营目标，对不同候选池设置对应的 **Quota** 配额，从而控制最终候选集中各类主播的占比。

类似地，在短视频推荐场景中，**Quota** 调控同样广泛存在。例如系统可能同时维护时长向视频池、互动向视频池、新视频池、创作者扶持池、商业化内容池等多个候选池。粗排阶段会根据当前业务目标和用户偏好，动态调整不同候选池的流量分配比例，从而实现用户体验、内容生态和平台收益之间的平衡。

从更本质的角度来看，**Quota** 调控实际上是在排序目标之外引入额外的业务约束。单纯依赖排序模型进行筛选，往往会导致候选集逐渐集中于某些高分内容，而造成内容同质化和生态失衡。而通过合理的配额控制，可以确保不同内容类型、不同创作者群体以及不同业务方向都能够获得一定的曝光机会。

Quota 的分配方式既可以采用固定规则策略实现，例如为不同候选池预先设置固定比例；也可以采用更加智能的动态分配机制。例如根据用户实时行为反馈、流量供给情况以及业务指标变化，动态调整各个候选池的配额占比。近年来，一些工业界系统还会引入强化学习、在线优化或预测模型来学习最优的 **Quota** 分配策略，从而实现更加精细化的流量调控。

在完成多目标融合排序和 **Quota** 调控之后，粗排阶段通常还会进一步进行保量、截断以及多样性约束等处理。最终将候选集规模从召回阶段的数千个物品压缩至数百个物品（通常为 100~500 个左右），并将这些候选物品送入精排阶段进行更加精细化的排序与筛选。

总体而言，粗排阶段的核心职责并非精确预测用户对于单个物品的兴趣程度，而是从召回阶段返回的海量候选集中构建一个高质量的候选集合（**High Quality Candidate Set**）。这个候选集合不仅需要包含用户潜在感兴趣的内容，还需要同时满足业务约束、多样性约束以及探索性需求。

因此，从系统视角来看，粗排阶段优化的对象实际上是整个候选集，而不仅仅是单个物品。一个优秀的粗排系统应当在相关性（**Relevance**）、多样性（**Diversity**）、探索性（**Exploration**）以及业务约束（**Business Constraints**）之间取得平衡，为后续精排阶段提供一个优质且具有充分优化空间的候选集合。例如，在候选集中需要兼顾不同内容垂类的覆盖度、内容日龄分布、新老内容比例、视频时长分布以及创作者生态等因素，从而避免候选集过度集中于某一类内容，提升推荐系统的长期效果和生态健康度。

12.4 精排模块

在推荐系统的级联式架构中，精排（**Ranking**）模块负责对粗排阶段输出的候选物品集合进行更加精细化的排序与筛选，并最终选出 **Top-K** 个物品送入后续的重排模块。相比于粗排阶段通常采用的轻量化双塔模型，精排模型往往拥有更复杂的网络结构、更丰富的特征交互能力以及更强的表达能力，因此能够对用户兴趣进行更加准确的建模，从而获得更高的排序精度。

从功能定位上来看，粗排阶段的主要目标是在大规模候选集中快速筛选出用户可能感兴趣的候选集合，而精排阶段则是在这一候选集合内部进一步挖掘用户偏好，对每个候选物品进行更加精准的价值评估。换句话说，粗排更关注候选集质量（**Candidate Set Quality**），而精排则更关注最终排序质量（**Final Ranking Quality**）。因此，精排模型通常会围绕用户真实满意度以及业务目标进行建模，例如点击、观看时长、互动行为、转化行为以及商业价值等，通过更加精确的预估结果完成最终排序。

如图 12.1 所示，精排模块通常接收粗排阶段筛选出的数百个候选物品（例如 100~500 个），并从中进一步选出 **Top 50~100** 个候选物品送入后续重排阶段。在实际工业场景中，精排阶段处理的候选集规模并非固定不变，而是会根据系统延迟预算、在线算力资源以及业务需求进行动态调整。例如在高并发场景下，为了保证系统整体吞吐量，精排阶段可能仅处理 100 个左右的候选物品；而在算力资源相对充足的场景下，则可能适当扩大精排候选集规模，以进一步提升最终排序效果。

尽管精排阶段处理的候选数量远少于粗排阶段，但由于其直接决定了最终推荐结果的主体质量，因此往往承担着推荐系统中最核心的排序优化任务。当前工业界大量先进的推荐模型、多任务学习框架以及用户兴趣建

模方法，也主要集中在精排阶段进行落地与迭代。

精排模块的整体处理流程如图 12.6 所示，其核心可以划分为三个阶段：精排主模型与 Learning to Rank (LTR) 模型、多目标融合以及流量调控。其中，精排主模型与 LTR 模型主要负责对用户与候选物品之间的匹配程度进行建模，并输出各种用户行为反馈以及业务目标对应的预估值，例如点击率 (CTR)、观看时长 (Watch Time)、点赞率 (LTR)、评论率 (CMTR)、转化率 (CVR) 以及其他业务 Reward 等。这些预估值从不同维度刻画了用户对于候选物品的潜在兴趣和价值，是后续排序决策的重要基础。在获得各类预估值之后，系统进入多目标融合阶段。由于推荐系统往往同时面临用户体验、内容生态、商业化收益等多个优化目标，因此需要通过多目标融合策略或融合模型，将多个 pctr 以及业务目标进行统一建模，最终得到一个综合排序分数 (Ranking Score)。随后系统依据该融合分数对候选物品进行排序，从而形成精排阶段的初步排序结果。

然而，仅仅依靠模型分数进行排序通常难以满足复杂的业务需求。与粗排阶段类似，精排阶段同样需要在排序结果中引入各种业务约束和生态约束，因此会进一步进行流量调控 (Traffic Control)。例如，在内容推荐场景中，需要为新作品提供冷启动流量扶持，为低粉丝创作者提供一定的曝光保护；在直播场景中，可能需要保证新主播或特定类型主播获得基础流量；在电商场景中，则可能需要针对大促活动、营销会场或重点商品进行流量倾斜。不同业务场景下的流量调控策略各不相同，但其核心目标都是在模型排序结果与业务目标之间取得平衡。

除此之外，精排阶段通常还会引入保量 (Guarantee) 机制，对特定内容池、创作者群体或业务方向设置最低曝光约束，以避免模型排序导致流量过度集中于少数头部内容。经过流量调控和保量策略处理后，系统最终从候选集中选出约 Top 50~100 个物品，并将其送入后续的重排模块进行进一步优化。

总体而言，精排阶段的核心职责是在粗排提供的高质量候选集基础上，尽可能准确地评估用户对每个候选物品的真实兴趣和潜在价值，并在兼顾业务约束与生态约束的前提下，输出一个兼具效果与业务合理性的排序结果。

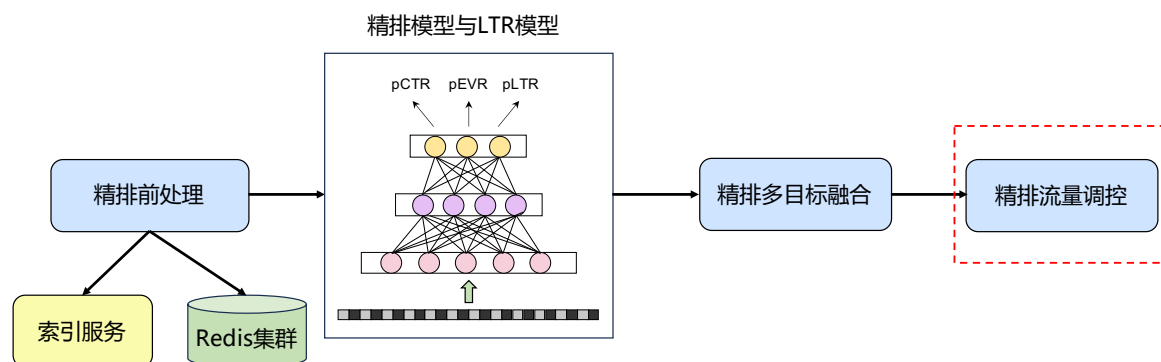


图 12.6: 精排模块内部处理流程。

12.4.1 精排主模型及 LTR 模型

由于精排阶段处理的候选物品数量远少于粗排阶段，因此其对于单个候选物品的计算资源预算更加充足。基于这一特点，工业界通常采用表达能力更强的深度神经网络 (Deep Neural Network, DNN) 作为精排主模型的基础架构。与粗排阶段广泛使用的双塔模型不同，精排模型通常会将用户侧特征、物品侧特征以及上下文特征进行统一编码，并拼接 (Concatenate) 后输入到 DNN 网络中进行建模。由于模型能够同时观察用户与物品两侧的信息，因此可以在网络内部自动学习更加复杂的特征交叉关系和非线性关联。例如，用户年龄与内容类别、用户历史兴趣与当前内容属性之间的高阶交互关系，都可以通过 DNN 结构进行有效建模。

相比于双塔结构，DNN 模型不仅具备更强的特征交互能力，也拥有更高的模型表达能力，因此在排序精度上通常能够取得更好的效果。正因如此，DNN 已逐渐成为工业界精排模型的标准架构之一。需要指出的是，粗排采用双塔架构而精排采用 DNN 架构，并非因为两者在建模能力上存在本质差异，而是推荐系统在效果、算力和时延约束之间进行工程权衡的结果。粗排阶段需要在数千个候选物品上进行快速推理，因此更加关注计算效率；而精排阶段候选集规模已经大幅缩减，可以接受更加复杂的模型结构，从而获得更高的排序精度。从理论

上来说，如果系统拥有充足的算力资源并能够满足在线时延要求，那么精排阶段常用的 DNN 架构同样可以迁移到粗排阶段使用。因此，理解粗排与精排模型架构差异背后的工程约束，比单纯记忆模型形式更加重要。

在模型功能层面，精排阶段通常同时包含精排主模型（Main Ranking Model）和 Learning to Rank（LTR）模型两类模型。其中，精排主模型主要负责对用户各种行为反馈进行直接建模。由于推荐系统往往需要同时优化多个目标，因此精排主模型通常采用多任务学习（Multi-Task Learning, MTL）框架，对点击、有效播放、点赞、评论、收藏、关注、转发等多种用户行为进行联合预估，并输出对应的 $pxtr$ 结果。

而 LTR 模型则更多地从排序目标的角度进行建模。与直接预测用户行为标签不同，LTR 模型通常会基于业务经验构造排序 Reward 或排序监督信号，例如用户综合满意度、长周期价值、高价值用户行为组合等，通过排序学习的方式直接优化最终排序效果。从某种意义上来说，LTR 模型可以看作是对精排主模型预估目标的重要补充，它能够学习一些难以通过单一行为标签刻画的排序偏好。

因此，在实际工业系统中，精排主模型与 LTR 模型往往并非相互替代的关系，而是相互协作、相互补充的关系。前者负责刻画用户具体行为发生的概率，后者负责学习更加贴近最终业务目标的排序偏好，两者共同构成精排阶段最核心的预测与排序能力。

12.4.2 精排模型常见的预估任务

在介绍精排模型结构之前，我们首先需要了解精排主模型以及 LTR 模型究竟在预估什么。事实上，无论模型结构如何演进，其核心任务始终是对用户未来可能产生的行为以及业务价值进行预测。因此，精排阶段的模型通常包含大量不同的预估任务（Prediction Task），这些任务共同构成了后续排序决策和多目标融合的基础。

从机器学习建模的角度来看，精排阶段常见的预估任务可以大致划分为两类：分类任务（Classification Task）和回归任务（Regression Task）。分类任务主要用于预测某种用户行为是否会发生，因此通常采用离散的 0-1 标签进行建模。例如点击（Click）、点赞（Like）、评论（Comment）、收藏（Favorite）、关注（Follow）、转发（Share）、购买（Purchase）等用户反馈行为，本质上都可以被视为二分类问题。对于某一次曝光而言，如果用户发生了对应行为，则标签记为 1；否则记为 0。模型经过训练后即可输出对应行为发生的概率，即工业界常见的各种 $pxtr$ 指标，例如：

$$p_{ctr}, p_{ltr}, p_{cmtr}, p_{ftr}, p_{cvr} \quad (12.11)$$

这些预估指标分别对应点击率、点赞率、评论率、收藏率以及转化率等不同目标。

除了用户的显式互动行为之外，一些隐式反馈也经常采用分类方式进行建模。例如短视频场景中的有效播放率（Effective View Rate）、完整播放率（Finish Rate）、长时观看率（Long View Rate）等指标，本质上都是将连续观看行为转换为某种阈值事件进行预测。例如：

$$label = \begin{cases} 1, & watch_time > 5s \\ 0, & otherwise \end{cases} \quad (12.12)$$

通过这种方式，模型能够学习用户是否会达到某个预先设定的满意度标准。

另一类常见任务是回归任务。与分类任务不同，回归任务直接预测连续数值，因此更加适用于观看时长、停留时长、消费金额、GMV 等连续型业务指标。例如在短视频推荐场景中，用户观看时长（Watch Time）通常是一个连续变量，因此可以直接采用回归模型进行建模：

$$\hat{y} = f(x) \quad (12.13)$$

其中 $\hat{y}$ 表示模型预测的观看时长。类似地，在电商推荐场景中，也可以直接预测用户未来可能产生的消费金额、订单价值或者生命周期价值（Lifetime Value, LTV）等连续型指标。

除了上述单一行为目标之外，随着推荐系统的发展，越来越多的精排模型开始直接预测业务 Reward。例如用户满意度（Satisfaction）、用户长期价值（Long-Term Value）、商业价值（Revenue）、GMV、广告收益等高层目

标。此时模型的预测对象已经不再局限于单个用户行为，而是对多个行为进行加权组合后形成的综合收益指标。

因此，从工业界的角度来看，精排模型和 LTR 模型的预估目标已经逐渐从早期单一的 CTR 预测，演进为覆盖点击、互动、观看、转化、消费以及长期价值等多个维度的多任务学习体系。后续的多目标融合阶段则会进一步将这些不同维度的预估结果统一整合，从而得到最终用于排序的综合分数。

12.4.2.1 时长预估

时长预估 (Watch Time Prediction) 是精排模型中最重要的预估任务之一。在短视频、直播以及长视频推荐场景中，用户观看时长往往比点击行为更加能够反映用户的真实满意度，因此时长类指标已经逐渐成为现代推荐系统最核心的优化目标之一。

从机器学习的角度来看，用户播放时长本质上是一个连续值，因此天然属于回归问题。然而在工业实践中，直接对播放时长进行回归建模通常会面临以下几个问题：

- 时长分布呈现严重长尾特征，大量样本集中于较小的时长区间；
- 用户行为噪声较大，相同内容在不同曝光场景下可能产生较大波动；
- MSE 等回归损失容易受到异常样本影响，导致训练不稳定；
- 回归任务通常比分类任务更难收敛。

因此，工业界通常不会直接对原始时长进行回归，而是采用一些特殊的建模技巧，将连续时长目标转换为分类问题或者半分类问题进行学习。

12.4.2.2 有效观看建模

最简单的方法是阈值离散化 (Threshold Binarization)。例如定义用户观看时长超过阈值 T 秒即视为有效观看：

$$label = \mathbb{I}(watch_time > T) \quad (12.14)$$

，其中 $\mathbb{I}(\cdot)$ 为指示函数。

此时模型学习的目标变为：

$$p_{ctr} = P(watch_time > T) \quad (12.15)$$

这种方法实现简单且训练稳定，因此被广泛应用于短视频推荐系统中。

12.4.2.3 加权二分类时长建模

另一种更加经典的时长建模方法来自 YouTube 推荐系统 [6]，通常被称为 Weighted Log Loss (WLL) 或 Weighted Logistic Regression。该方法并不直接对播放时长进行回归，而是将时长预估问题转化为一个特殊的二分类学习问题。

与普通 CTR 预估不同，Weighted Log Loss 的核心思想是：对于一次曝光样本，其既贡献正样本损失，也贡献负样本损失，只不过两部分损失具有不同的权重。通过这种方式，可以利用二分类任务训练稳定、易于优化的特点，间接学习连续的观看时长信息。

假设用户对于某个视频（或直播间）的真实观看时长为：

$$t \geq 0 \quad (12.16)$$

对于任意一次曝光样本，都贡献一个负样本部分：

$$y = 0 \quad (12.17)$$

对应权重为：

$$w_{neg} = 1 \quad (12.18)$$

如果用户产生了观看行为（即 $t > 0$ ），则额外贡献一个正样本部分：

$$y = 1 \quad (12.19)$$

对应权重定义为：

$$w_{pos} = t \quad (12.20)$$

从工程实现的角度来看，一个具有观看时长的曝光样本实际上会被展开成两条训练样本：

表 12.4: Weighted Log Loss 时长预估标签及权重定义

Label	Weight	含义
0	1	负样本部分
1	t	正样本部分

因此，对于具有观看行为的样本而言，其同时参与正样本损失和负样本损失的计算。换句话说，同一个样本既贡献了：

$$-\log(1 - p) \quad (12.21)$$

对应的负样本损失，又贡献了：

$$-t \log p \quad (12.22)$$

对应的正样本损失。

将两部分损失合并之后，可以得到单个曝光样本的损失函数：

$$L_i = -t_i \log p_i - \log(1 - p_i) \quad (12.23)$$

其中：

$$p_i = \sigma(z_i) \quad (12.24)$$

表示模型输出的 Sigmoid 概率。

从整个训练集的角度来看，其优化目标可以写为：

$$L = - \sum_i (t_i \log p_i + \log(1 - p_i)) \quad (12.25)$$

这里需要特别注意，模型输出的 p_i 已经不再表示传统意义上的点击率或者观看概率，而是隐式编码了用户观看时长的信息。这也就是工业界推荐系统中时长预估任务下所说的 **pvr** 的指标。

下面简要分析一下该损失函数的最优解。对于固定特征条件 x ，设该条件下的真实条件期望观看时长为：

$$\mu(x) = E[t|x] \quad (12.26)$$

则对应的条件风险可以写为：

$$R(p|x) = \mu(x) \log p + \log(1 - p) \quad (12.27)$$

对 p 求导：

$$\frac{\partial R}{\partial p} = \frac{\mu(x)}{p} + \frac{1}{1-p} \quad (12.28)$$

令导数为零：

$$\frac{\mu(x)}{p} + \frac{1}{1-p} = 0 \quad (12.29)$$

整理可得：

$$p = \frac{\mu(x)}{1 + \mu(x)} \quad (12.30)$$

因此：

$$\frac{p}{1-p} = \mu(x) = E[t|x] \quad (12.31)$$

即模型输出概率的 **Odds** 恰好对应于用户条件期望观看时长。因此上线推理时，可以利用 **Odds** 反变换恢复观看时长预估值：

$$\widehat{watch_time} = \frac{p}{1-p} \quad (12.32)$$

在工业界实践中，我们通常也把这种反变换的过程称之为“**时长反解**”。

若模型直接输出 **Logit**：

$$z = \log \frac{p}{1-p} \quad (12.33)$$

则进一步有：

$$\widehat{watch_time} = e^z \quad (12.34)$$

需要注意的是，工业界实际应用时，由于观看时长通常呈现明显的长尾分布，因此往往不会直接使用原始时长作为权重，而是采用截断或者压缩后的时长进行训练。例如：

$$w = t' = \min(t, T_{max}) \quad (12.35)$$

或者：

$$w = t' = \log(1 + t) \quad (12.36)$$

，从而降低极端长时长样本对于模型训练的影响，提高训练稳定性。

总体而言，**Weighted Log Loss** 本质上是在分类任务框架下学习条件期望观看时长。它同时兼具分类模型易于训练和连续时长建模能力强两方面优势，因此至今仍然是短视频、直播以及长视频推荐系统中最经典、最常见的时长建模方案之一。

12.4.2.4 离散化分桶建模

除了上述方法之外，近年来工业界也广泛采用连续时长离散化（Discretization）的方法进行建模。假设用户观看时长分布范围为：

$$t \in [0, T_{max}] \quad (12.37)$$

首先将时长划分为 N 个桶：

$$B_1, B_2, \dots, B_N \quad (12.38)$$

常见分桶方式包括：

- 等距分桶（Equal Width Binning）
- 等频分桶（Equal Frequency Binning）
- 分位数分桶（Quantile Binning）
- 对数分桶（Logarithmic Binning）

其中推荐系统实践中最常见的是分位数分桶，因为能够缓解长尾分布问题。若采用多分类建模，则标签定义为：

$$y = k \quad \text{if} \quad t \in B_k \quad (12.39)$$

模型输出：

$$P(B_k|x) \quad (12.40)$$

最终恢复时长：

$$\hat{t} = \sum_{k=1}^N P(B_k|x) \cdot c_k \quad (12.41)$$

其中 c_k 为第 k 个桶对应的代表时长。

近年来，快手科技提出的 D2Q（Duration-Deconfounded Quantile-based Watch-time Prediction Framework）方法 [39]，进一步将分位数学习（Quantile Learning）引入时长预估任务，从而避免模型直接拟合长尾且高噪声的原始观看时长分布。

传统的时长回归模型通常直接学习：

$$E[t|x] \quad (12.42)$$

即用户观看时长的条件期望。然而在实际推荐场景中，观看时长往往呈现明显的长尾分布特征，大量样本集中在较短时长区间，而少量样本则具有极长的观看时长。此时直接学习均值容易受到异常值影响，导致模型训练不稳定。

D2Q 的核心思想是将原始时长空间映射到分位数空间（Quantile Space）进行建模。具体而言，首先基于全量训练数据统计观看时长分布，并计算其累积分布函数（CDF）：

$$F(t) = P(T \leq t) \quad (12.43)$$

，随后将原始观看时长 t 映射为对应的分位数标签：

$$q = F(t) \quad (12.44)$$

，其中：

$$q \in [0, 1] \quad (12.45)$$

表示该样本在整体观看时长分布中的相对位置。例如，若某个样本对应的观看时长处于全体样本的 90% 分位点，则有：

$$q = 0.9 \quad (12.46)$$

。此时模型学习的目标不再是原始观看时长，而是分位数空间中的目标：

$$\hat{q} = f(x) \quad (12.47)$$

。预测阶段再通过分位数函数（Quantile Function）完成反变换：

$$\hat{t} = F^{-1}(\hat{q}) \quad (12.48)$$

，从而恢复对应的观看时长估计值。

从另一种角度理解，D2Q 也可以被看作是在学习多个分位点：

$$Q(\tau_1), Q(\tau_2), \dots, Q(\tau_m) \quad (12.49)$$

，其中：

$$\tau_i \in (0, 1) \quad (12.50)$$

，表示不同的分位数水平。论文中通常采用 Quantile Loss 进行优化：

$$L_\tau = \max(\tau(y - \hat{y}), (\tau - 1)(y - \hat{y})) \quad (12.51)$$

该损失函数能够有效学习不同分位点对应的条件分布信息，从而更加完整地刻画用户观看时长的整体分布，而不仅仅是均值信息。相比传统均值回归方法，D2Q 能够更好地处理长尾时长分布、高噪声样本以及异方差问题，因此近年来被广泛应用于短视频、直播以及长视频推荐系统中。

当然，在实际工业界落地过程中，并不一定严格采用论文中的 Quantile Loss 进行训练。许多工程实践会首先利用历史数据统计时长分位数，将原始观看时长映射为归一化后的分位数标签：

$$q \in [0, 1] \quad (12.52)$$

。随后直接采用均方误差损失（Mean Squared Error, MSE）进行回归学习：

$$L_{MSE} = \frac{1}{N} \sum_{i=1}^N (q_i - \hat{q}_i)^2 \quad (12.53)$$

这种方法实现简单、训练稳定，并且同样能够获得分位数空间建模带来的长尾分布鲁棒性。因此在工业界的时长预估任务中，基于分位数标签的 MSE 回归也是一种十分常见的实现方案。

12.4.2.5 稀疏互动信号建模

除了上一章节介绍的时长预估任务及其分类、回归建模方法之外，精排模型还需要对大量互动类行为信号进行建模，例如点击、点赞、评论、收藏、关注、转发、分享、打赏以及购买等行为。这些互动信号通常对应用户更强的兴趣表达，因此在推荐系统的排序决策中往往具有较高的价值。

然而，与时长信号相比，互动类信号具有一个显著特点，即其数据分布通常非常稀疏。对于时长类目标而言，只要用户对某个曝光物品产生观看行为，系统就能够获得对应的观看时长反馈。因此从数据分布的角度来看，时长信号通常属于相对稠密（Dense）的监督信号。例如在短视频、直播等场景中，大部分曝光都会产生一定的播放时长，因此时长标签的覆盖率通常较高。

而互动类目标则完全不同。以推荐系统中常见的行为指标为例：

- CTR (Click Through Rate)：通常在 3% ~ 10% 左右；
- LTR (Like Through Rate)：通常低于 1%；
- CMTR (Comment Through Rate)：往往低于千分级；
- FTR (Follow Through Rate)：通常低于万分级；
- CVR (Conversion Rate)：在电商与广告场景中通常更加稀疏。

随着用户行为价值的不断提升，其对应标签往往也会越来越稀疏。这意味着在训练过程中，绝大部分样本都是负样本，而真正有价值的正样本数量极其有限。因此，在精排模型中，一个非常重要的研究方向便是如何有效学习这些稀疏互动目标。

12.4.2.6 样本加权 (Sample Weighting)

针对稀疏目标，最简单也是最常见的方法是样本加权（Sample Weighting）。其核心思想是提高高价值正样本在训练过程中的梯度贡献，从而缓解正负样本极度不平衡的问题。例如在直播推荐场景中，如果需要建模用户打赏概率（GTR, Gift Through Rate），那么每一次打赏行为往往伴随着具体的打赏金额（G-Value, Gift Value）。此时可以将打赏金额作为正样本权重：

$$w_i = gift_amount_i \quad (12.54)$$

，对应的加权交叉熵损失为：

$$L = - \sum_i w_i (y_i \log p_i + (1 - y_i) \log(1 - p_i)) \quad (12.55)$$

这样一来，高金额打赏行为将产生更大的梯度贡献，使模型更加关注高价值用户行为。从业务角度来看，这种方式不仅能够缓解样本稀疏问题，同时也能够引导模型学习不同正样本之间的价值差异。例如打赏 100 元与打赏 1 元虽然都属于正样本，但显然对应着完全不同的用户价值和兴趣强度。

12.4.2.7 稠密目标辅助稀疏目标学习

除了样本加权之外，另一类非常重要的方法是利用稠密目标辅助稀疏目标学习（Dense-to-Sparse Learning）。其基本思想来源于用户行为之间天然存在的漏斗关系（User Behavior Funnel）。例如在电商推荐场景中：

$$Exposure \rightarrow Click \rightarrow Purchase \quad (12.56)$$

用户必须先点击商品，才有可能产生购买行为。因此 CTR 相比于 CVR 更加稠密，而 CVR 又依赖于 CTR 的发生。基于这种行为依赖关系，研究人员提出了大量经典模型来利用稠密目标辅助稀疏目标学习，其中最具代表性的工作包括：

- CTCVR (Click-Through and Conversion Rate)
- ESMM (Entire Space Multi-Task Model) [26]
- ESCM² [35]

这类方法通常通过共享参数、条件概率分解或者联合建模等方式，让稠密目标为稀疏目标提供额外监督信号，从而显著提升稀疏目标的学习效果。

12.4.2.8 行为关联建模

除了漏斗关系之外，不同互动行为之间本身也往往存在较强的相关性。例如在短视频和直播场景中，当用户对某位创作者产生较高兴趣时，点赞、评论、收藏以及关注等行为往往会共同出现。从概率角度来看，可以认为这些行为之间并非相互独立，而是存在潜在的因果关系或者兴趣传递关系：

$$Like \rightarrow Comment \rightarrow Follow \quad (12.57)$$

，或者：

$$Like \rightarrow Favorite \rightarrow Follow \quad (12.58)$$

如果能够显式建模这些行为之间的依赖关系，往往能够提升多个目标的整体预测效果。基于这一思想，工业界提出了大量行为关系建模方法。例如美团提出的 AITM (Adaptive Information Transfer Multi-task framework) [37] 模型，便通过信息传递机制显式建模不同任务之间的依赖关系，从而解决传统多任务学习中任务之间相互干扰的问题。这一类挖掘多个目标之间相关性以及隐式关联的方法，在工业界也被称为“多目标联合建模”或者“多目标感知建模”，即让多个稀疏目标在建模的时候能够互相感知到其他的目标信号，不仅在信号层面形成补充，也能够让模型挖掘出不同信号之间的隐式联系。

除此之外，近几年来出现的 PLE (Progressive Layered Extraction) [33]、MMoE (Multi-gate Mixture-of-Experts) [24]、SNR (Sub-Network Routing) [25] 等多任务学习框架，也都在一定程度上尝试挖掘不同目标之间的共享信息和差异化信息，从而提升稀疏目标的学习效果。

总体而言，稀疏互动目标建模是现代推荐系统精排模型中的核心研究方向之一。围绕样本不平衡、行为漏斗建模、目标关联建模以及多任务学习等问题，学术界和工业界已经提出了大量经典方法。这里只对相关思路进行了简要介绍，后续章节将进一步深入分析 ESMM、AITM、MMoE、PLE 等代表性模型的设计原理及其在工业界中的应用实践。

12.4.3 精排多目标融合

精排多目标融合阶段与粗排阶段的目标基本一致，其核心任务都是将多个模型输出的不同目标预估值 (pxtr) 进行统一融合，并依据融合分完成最终排序。在粗排阶段，由于模型结构相对简单，参与融合的因子通常以粗排主模型输出的若干个 pxtr 为主；而在精排阶段，由于候选集规模进一步缩小，系统能够容纳更多复杂模型参与打分，因此融合阶段需要处理的排序因子数量也会显著增加。

除了精排主模型输出的 CTR、EVTR、LTR、CVR 等基础预估值之外，工业界通常还会引入大量专门针对特定业务目标构建的 LTR 模型，例如：

- D2Q 时长预估模型；
- 用户长期价值 (LTV) 模型；
- 用户兴趣挖掘模型；
- 基于人工经验 Reward 设计的 LTR 模型；
- Session 级强化学习模型；
- 创作者价值模型；
- 内容质量评估模型；
- 新内容冷启动模型等

在大型工业界推荐系统中，参与融合的排序因子数量往往达到数十个甚至上百个。因此工业界通常会采用统一的多目标融合框架对这些因子进行管理组合，例如加法公式、乘法公式以及基于序 (Rank) 的融合方式等。这一类融合与排序过程通常统称为 **Ensemble Sort (ES)**，即利用多个模型的排序能力进行集成学习，从而获得优于单模型排序的整体效果。

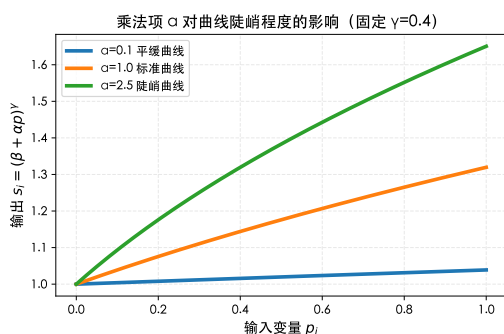
12.4.3.1 乘法公式及形状分析

在工业界众多融合方法中，乘法公式（Multiplicative Formula）是一种应用非常广泛的融合框架。其基本形式如下：

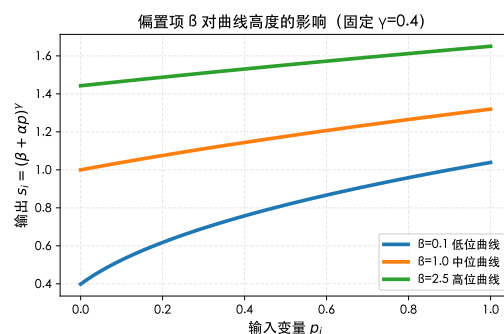
$$s_i = (\beta_i + \alpha_i p_i)^{\gamma_i} \quad (12.59)$$

。其中：

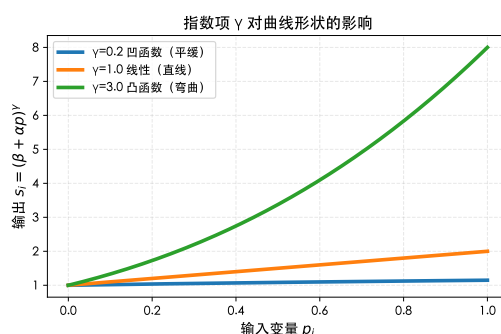
- p_i 表示第 i 个排序因子的原始预估值；
- α_i 为乘法系数（Scaling Factor）；
- β_i 为偏置项（Bias Term）；
- γ_i 为指数项（Exponent Term）。



(a) 乘法项 α 的影响



(b) 偏置项 β 的影响



(c) 指数项 γ 的影响

图 12.7: 乘法公式 $s_i = (\beta_i + \alpha_i p_i)^{\gamma_i}$ 中三个超参数的影响对比

图12.7展示了三个超参数对于最终得分曲线的影响。从工程实践角度来看，这三个参数实际上分别对应着排序系统中的三个重要控制维度：

- 区分度控制（ α_i ）；
- 基准权重控制（ β_i ）；
- 流量分配形状控制（ γ_i ）。

其中，乘法系数 α_i 主要负责调节原始 p_{xtr} 的放大倍数。当 α_i 较大时，不同物品之间原本微小的 p_{xtr} 差异会被进一步放大，从而提高排序结果的区分能力。此时系统更容易将流量集中分配给头部物品。反之，当 α_i 较小时，不同物品之间的得分差异被压缩，排序结果趋于平滑，系统整体探索性会有所增强。偏置项 β_i 则主要用于控制该因子的基础贡献水平。由于许多 p_{xtr} 本身取值较小，例如点赞率、评论率、分享率等稀疏目标，如果直接参与乘法融合，很容易导致整体得分迅速衰减。因此工业界通常会引入偏置项来稳定数值分布，同时调节不同目标在融合过程中的基础影响力。

相比之下，指数项 γ_i 往往是整个乘法公式中最重要的超参数。其本质决定了排序系统对于头部物品和尾部

物品的偏好程度。

当

$$\gamma_i > 1 \quad (12.60)$$

时，变换函数呈现凸函数（Convex Function）形态。此时高分物品会被进一步放大，而低分物品增长缓慢。因此系统会更倾向于选择在该目标上表现特别优秀的物品。从流量分发角度来看，这种配置会进一步强化头部效应（Matthew Effect）。当

$$0 < \gamma_i < 1 \quad (12.61)$$

时，函数呈现凹函数（Concave Function）形态。此时高分与低分物品之间的差距被压缩，系统会给予更多中腰部物品曝光机会，从而提升推荐结果的多样性和探索性。

特殊地，当

$$\gamma_i = 1 \quad (12.62)$$

时，

$$(\beta_i + \alpha_i p_i)^{\gamma_i} = \beta_i + \alpha_i p_i \quad (12.63)$$

公式退化为线性变换，不再具有额外的非线性放大能力。

总体而言：

- α_i 决定区分度；
- β_i 决定基础贡献；
- γ_i 决定流量分配形状。

因此，在工业界实践中，指数项 γ_i 通常也是最需要重点调优的参数。

12.4.3.2 序变换及形状分析

除了前述基于分数值（Value）的乘法融合公式之外，工业界还广泛使用基于排序位置（Rank）的融合方法。其核心思想是不直接利用 $\mathbf{pxtr}$ 的具体数值，而是仅利用不同候选物品在各个目标上的排序位置进行融合。

这种方法的出发点非常简单：不同模型输出的 $\mathbf{pxtr}$ 往往存在数值尺度不一致、分布漂移以及校准误差等问题。例如 CTR 的取值可能集中在 $[0.01, 0.2]$ 区间，而 CVR 的取值可能集中在 $[10^{-4}, 10^{-2}]$ 区间。如果直接利用数值进行融合，则需要频繁调整权重以适应不同因子的分布变化。而 Rank 本身只保留相对顺序信息，因此天然具有更强的鲁棒性。

为了对不同排序位置赋予不同的重要程度，工业界通常会设计各种非线性序变换函数。本章节选取以下五种具有代表性的变换方式进行分析：

$$(1) \text{ 倒数变换: } S_i = \frac{1}{\text{Rank}_i} \quad (12.64)$$

$$(2) \text{ 平滑倒数变换: } S_i = \frac{w_i}{\text{Rank}_i^\lambda + b} \quad (12.65)$$

$$(3) \text{ 归一化幂次变换: } S_i = w_i \left(1 - \left(\frac{\text{Rank}_i}{N} \right)^\lambda \right) \quad (12.66)$$

$$(4) \text{ 指数双曲变换: } S_i = w_i \left(e^{k(2\frac{\text{Rank}_i}{N}-1)} - e^{-k(2\frac{\text{Rank}_i}{N}-1)} \right) \quad (12.67)$$

$$(5) \text{ 带偏置归一化变换: } S_i = w_i \left(1 - \left(\frac{\text{Rank}_i}{N+b} \right)^\lambda \right) \quad (12.68)$$

其中， Rank_i 表示候选物品在某个排序因子中的位置， N 表示候选集长度， w_i 表示对应因子的权重， λ 、 b 和 k 为控制曲线形状的超参数。

上述五种典型序变换函数的曲线形状如图 12.8 所示。

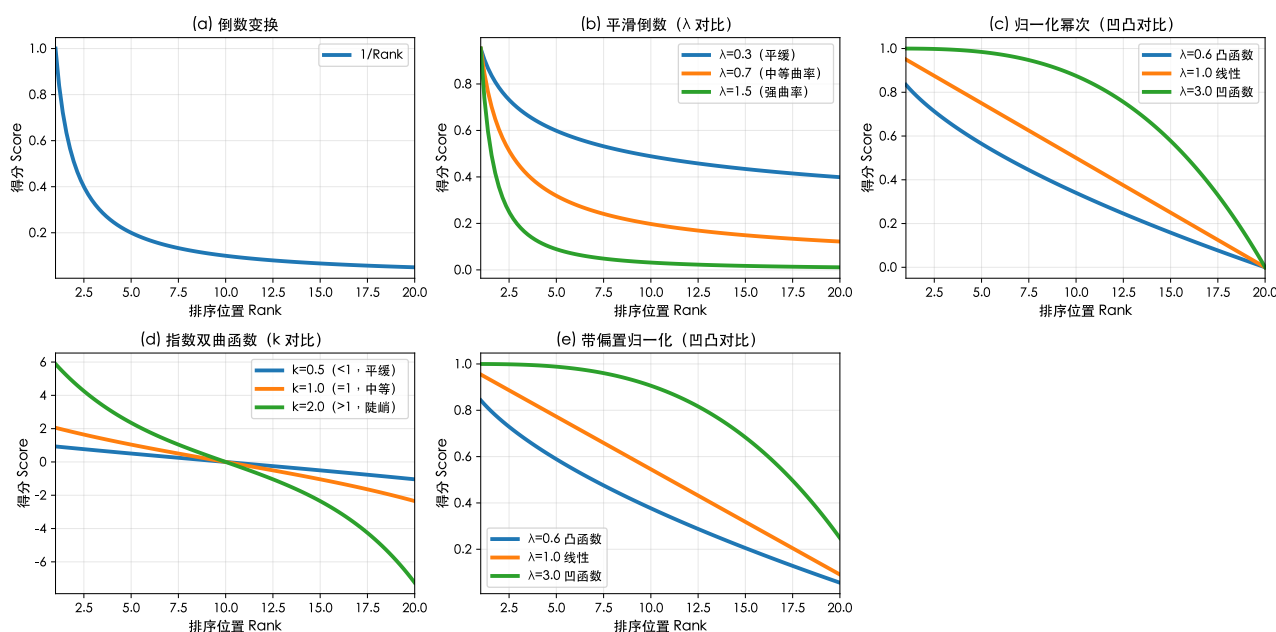


图 12.8: 不同排序位置变换函数的得分曲线对比。(a) 倒数变换；(b) 平滑倒数变换；(c) 归一化幂次变换；(d) 指数双曲变换；(e) 带偏置归一化变换。

从图 12.8(a) 可以看出，传统倒数变换是一种非常激进的头部分化策略。由于

$$\frac{1}{1} = 1, \quad \frac{1}{2} = 0.5, \quad \frac{1}{10} = 0.1 \quad (12.69)$$

因此得分会随着 Rank 的增加快速衰减。这意味着排序靠前的少量物品能够获得远高于其他物品的融合得分。在实际业务中，这种变换往往会强烈偏向于某个目标上的 Top-K 候选物品。例如 CTR 排名第一和第二的物品之间差异巨大，而第 50 名和第 60 名之间几乎没有区别。因此倒数变换本质上是一种典型的 Head-Oriented 排序策略。

图 12.8(b) 展示了平滑倒数变换的效果。相比传统倒数变换，其引入了超参数 λ 和偏置项 b 用于调节曲线形状。当

$$\lambda = 0.3 \quad (12.70)$$

时，曲线下降速度较慢，不同 Rank 之间的得分差异相对平缓；当

$$\lambda = 0.7 \quad (12.71)$$

时，曲线开始体现明显的头部强化效果；而当

$$\lambda = 1.5 \quad (12.72)$$

时，曲线在前几个位置急剧下降，Rank 大于 5 之后得分迅速趋近于零。这意味着较大的 λ 会显著提升 Top-K 物品的优势，使得某个排序因子的头部候选更容易在最终 ES 融合中胜出。因此，从工程经验来看， λ 本质上控制的是排序系统对于头部物品的偏好程度。

图 12.8(c) 和图 12.8(e) 分别展示了归一化幂次变换以及带偏置归一化变换。与倒数类函数不同，这两类函数能够更加灵活地控制整个排序区间内的得分分布。当

$$\lambda < 1 \quad (12.73)$$

时，曲线整体呈现凸函数形态，此时头部区域下降速度较慢，而尾部区域下降速度较快。排序系统会更加关注

Top-K 候选物品之间的差异。当

$$\lambda = 1 \quad (12.74)$$

时，曲线退化为线性形式。而当

$$\lambda > 1 \quad (12.75)$$

时，曲线呈现凹函数特征。此时头部物品之间的差异被压缩，而中腰部物品之间的区分度得到增强。

因此这类函数最大的优势在于能够灵活控制排序资源究竟应该集中在头部区域还是分散到中腰部区域。例如在一些探索性推荐场景中，过度强化头部可能会导致内容同质化，此时往往会采用较大的 λ 来压平头部差异，从而增加中腰部内容的曝光机会。

图 12.8(d) 中的指数双曲变换则体现了另一种思路。其本质上对应双曲正弦函数：

$$S_i = 2w_i \sinh \left(k \left(2 \frac{\text{Rank}_i}{N} - 1 \right) \right) \quad (12.76)$$

与前面几种单调曲率函数不同，双曲函数在不同区间会表现出不同的敏感性。以

$$k = 2.0 \quad (12.77)$$

为例，可以观察到：

- Rank 位于 Top-5 时，曲线下降非常迅速，强化头部候选之间的区分度；
- Rank 位于中腰部区间时，曲线变化趋于平缓，降低中部候选之间的差异；
- Rank 位于尾部区域时，曲线再次变陡，从而重新拉开尾部候选之间的距离。

这种特性使得双曲函数能够实现局部区域的差异化建模。例如有些业务希望重点区分 Top-K 和尾部内容，而对中间区域不那么敏感，此时双曲函数往往能够取得更好的效果。

总体来看，不同序变换函数本质上对应着不同的流量分配哲学。倒数变换强调头部优先；平滑倒数变换强调可控的头部强化；幂次变换强调头中尾资源的重新分配；双曲变换则能够实现不同排序区域的差异化调节。因此在工业界实践中，并不存在绝对最优的序变换函数。最终采用何种曲线形状，通常需要结合具体业务目标，通过大量线上 A/B 实验不断验证和迭代。

最后，在讲完了乘法公式变换和序变换以及它们对应的排序公式形状分析之后，我们可以发现，不同的 Rank 变换函数本质上都在解决同一个问题：

 **笔记** 如何分配不同排序位置之间的区分能力？

有些函数更关注头部物品之间的排序精度；有些函数更关注中腰部物品之间的竞争关系；还有一些函数会刻意强化尾部物品之间的区分能力，从而为探索流量提供更多空间。因此这些变换函数本质上并不存在绝对优劣，其效果最终取决于具体业务场景、候选集分布以及线上流量分发目标。在工业界实践中，策略算法工程师通常会结合 AB 实验，通过超参数搜索、自动化调参以及强化学习等方法寻找最优的 Rank 变换形状，而不是依赖固定的经验公式。

12.4.3.3 排序公式超参数寻优

在粗排多目标融合章节中，我们介绍了加法公式、乘法公式以及基于序变换的融合公式。这些公式虽然能够较好地融合多个 p_{xtr} 因子，但往往包含大量超参数，例如加法公式中的权重参数、乘法公式中的偏置项与指数项，以及序变换中的曲率参数等。传统工业界通常采用人工经验调参的方式确定这些超参数。然而随着排序因子数量不断增加，一个实际的排序公式往往包含几十甚至上百个待优化参数。此时依赖人工经验进行调参不仅效率较低，而且难以找到全局最优解。因此近年来越来越多的推荐系统开始引入自动化搜参 (Auto Tuning) 技术，其中最经典的方法之一便是 Cross-Entropy Method (CEM) 算法。

12.4.3.3.1 CEM 算法基本思想 假设排序公式包含 N 个待优化参数：

$$\theta = (\theta_1, \theta_2, \dots, \theta_N) \quad (12.78)$$

我们的目标是寻找最优参数：

$$\theta^* = \arg \max_{\theta} R(\theta) \quad (12.79)$$

其中 $R(\theta)$ 表示线上 AB 实验所反馈的业务收益指标，例如：

- 人均观看时长；
- 用户留存率；
- GMV；
- 广告收益；
- 长期价值 (LTV) 等。

由于线上指标通常无法直接求导：

$$\frac{\partial R}{\partial \theta_i} \quad (12.80)$$

因此传统梯度下降方法无法直接应用。

CEM 的核心思想是：**不直接搜索最优参数，而是搜索最优参数所在的概率分布**。假设参数服从一个多维高斯分布：

$$\theta \sim \mathcal{N}(\mu, \Sigma) \quad (12.81)$$

，其中：

$$\mu = (\mu_1, \mu_2, \dots, \mu_N) \quad (12.82)$$

表示参数均值， Σ 表示协方差矩阵。CEM 算法并不直接优化参数本身，而是不断更新：

$$(\mu, \Sigma) \quad (12.83)$$

使采样分布逐渐向高收益区域收缩。

12.4.3.3.2 采样阶段 在第 t 轮迭代时，从当前分布中采样：

$$\theta_1, \theta_2, \dots, \theta_K \quad (12.84)$$

满足：

$$\theta_i \sim \mathcal{N}(\mu^{(t)}, \Sigma^{(t)}) \quad (12.85)$$

每一个采样点都对对应一套排序公式参数。例如：

$$\theta_1 = (w_1, w_2, \gamma_1, \gamma_2, \dots) \quad (12.86)$$

代表一个具体的多目标融合配置。

12.4.3.3.3 线上评估阶段 对于每组参数，都分配一部分线上流量进行实验。经过若干小时后获得对应收益：

$$R_1, R_2, \dots, R_K \quad (12.87)$$

，然后按照收益排序：

$$R_{(1)} \geq R_{(2)} \geq \dots \geq R_{(K)} \quad (12.88)$$

并保留前 ρ 比例的优秀样本：

$$\mathcal{E} = \{\theta^*(1), \dots, \theta^*(M)\} \quad (12.89)$$

，其中：

$$M = \rho K \quad (12.90)$$

。通常来说， ρ 参数满足： $\rho \in [0.05, 0.2]$ 。这一部分样本被称为 **Elite Samples**（精英样本）。

12.4.3.3.4 分布更新阶段 在分布更新阶段，我们利用精英样本重新估计高斯分布参数。需要对均值和协方差分别进行更新。其中，均值更新如下：

$$\boldsymbol{\mu}^{(t+1)} = \frac{1}{M} \sum_{i=1}^M \boldsymbol{\theta}_{(i)} \quad (12.91)$$

，协方差更新如下：

$$\Sigma^{(t+1)} = \frac{1}{M} \sum_{i=1}^M (\boldsymbol{\theta}_{(i)} - \boldsymbol{\mu}^{(t+1)})(\boldsymbol{\theta}_{(i)} - \boldsymbol{\mu}^{(t+1)})^T \quad (12.92)$$

这样新的采样分布会逐渐向高收益区域移动。随着迭代不断进行，会有：

$$\Sigma^{(t)} \rightarrow 0 \quad (12.93)$$

，即采样空间不断收缩，最终收敛到一个局部最优解附近。

需要注意的是，CEM 并不是简单的随机搜索。其理论基础来源于最小化两个概率分布之间的 **KL Divergence**：

$$D_{KL}(g^* \| f_{\theta}) \quad (12.94)$$

，其中： g^* 表示理想的最优解分布， f_{θ} 表示当前采样分布。最小化 **KL Divergence** 等价于最小化交叉熵（**Cross Entropy**），因此该方法被称为 **Cross-Entropy Method**。

12.4.3.3.5 工业界在线搜参流程 在推荐系统实际落地过程中，CEM 通常用于排序公式自动调优。例如，对于一个乘法融合公式：

$$Score = \prod_{i=1}^N (\beta_i + \alpha_i p_i)^{\gamma_i} \quad (12.95)$$

，其中：

$$\boldsymbol{\theta} = (\alpha_1, \dots, \alpha_N, \beta_1, \dots, \beta_N, \gamma_1, \dots, \gamma_N) \quad (12.96)$$

全部作为待搜索参数。

系统首先随机生成数百组参数配置，然后将线上流量切分给不同实验桶。经过数小时甚至数天的数据积累后，根据观看时长、留存率、GMV 等指标选出收益最好的若干组参数，再利用这些参数重新拟合采样分布并继续搜索。经过多轮迭代之后，参数空间会逐渐向收益更高的区域收缩，从而自动获得优于人工调参的排序公式。

从本质上看，CEM 并不是在优化排序公式本身，而是在不断学习：

$$P(\boldsymbol{\theta} \mid R \text{ 较高}) \quad (12.97)$$

即“高收益排序公式最有可能出现在哪些参数区域”因此，CEM 可以被看作一种基于概率分布迭代更新的黑盒优化方法，在推荐系统、多目标融合、强化学习策略搜索以及自动机器学习（**AutoML**）等领域均有广泛应用。

12.4.3.3.6 工业界 CEM 应用时存在的问题 虽然从理论上来看，CEM 能够通过不断迭代更新采样分布逐步逼近最优参数区域，但在工业界推荐系统的实际应用过程中，我们发现其效果往往没有理论分析中那么理想。究其原因，问题并不完全出在优化算法本身，而更多来自推荐系统环境本身所具有的复杂性与不确定性。

首先，推荐系统的在线反馈数据天然具有较大的噪声（Noise）。在理想情况下，我们希望每组超参数对应一个准确且稳定的收益指标： $R(\theta)$ 。然而在线系统中的收益往往受到大量随机因素影响。例如用户的实时兴趣变化、节假日效应、热点事件、内容供给变化、流量波动甚至埋点异常等因素都会影响最终观测到的指标。因此实际观测到的收益通常更接近于：

$$\hat{R}(\theta) = R(\theta) + \epsilon \quad (12.98)$$

，其中： ϵ 表示各种来源的随机噪声。这意味着某一组参数在线上表现更好，并不一定是因为参数本身更优，也有可能仅仅是采样到了更有利的流量或者更活跃的用户群体。因此 CEM 每一轮筛选出的 Elite Samples 并不一定是真正意义上的最优参数区域，也有可能只是在局部最优做随机扰动。

其次，推荐系统中的用户反馈往往存在明显的**延迟反馈（Delay Feedback）**问题。例如观看时长、点赞、评论等行为通常能够在分钟级甚至秒级获得反馈，但很多更重要的业务指标却需要较长时间才能观测到。例如：

- 次日留存（Day-1 Retention）；
- 七日留存（Day-7 Retention）；
- 用户生命周期价值（LTV）；
- 长期 GMV；
- 用户长期满意度。

这些指标往往需要数天甚至数周的数据积累才能较为准确地评估。

因此在实际应用中会出现一个典型矛盾：

- 如果使用小时级反馈指标进行寻优，则反馈速度快，迭代效率高；
- 如果使用天级甚至周级反馈指标进行寻优，则目标更准确，但优化周期极长。

例如对于一轮 CEM 搜索，如果每次评估需要等待一天：

$$T_{eval} = 1 \text{ Day} \quad (12.99)$$

假设需要执行 20 轮迭代：

$$T_{total} = \#20 \times T_{eval} = 20 \text{ Days} \quad (12.100)$$

那么整个搜索过程可能需要数周时间才能完成。而在这期间，用户兴趣分布、内容生态以及流量结构可能已经发生明显变化，此时得到的最优参数甚至可能已经不再适用于当前系统。

进一步地，随着推荐系统复杂度不断提升，多目标融合公式中的参数规模也在迅速增长。假设一个乘法融合公式包含 $N = 50$ 个排序因子。对于每个因子分别需要优化：

$$\alpha_i, \beta_i, \gamma_i \quad (12.101)$$

这三个参数。则整体参数空间规模达到： $3N = 150$ 维。当参数维度不断增加时，搜索空间会呈指数级膨胀：

$$\mathcal{O}(c^N) \quad (12.102)$$

即所谓的**维度灾难（Curse of Dimensionality）**。此时即使采用 CEM，也往往需要大量采样才能覆盖有效搜索区域，因此搜索效率会迅速下降。很多情况下，算法甚至还没有收敛，业务环境已经发生变化。

正因为如此，近年来工业界开始探索更加自动化和智能化的排序公式优化方案。例如强化学习（Reinforcement Learning）方法尝试直接学习排序策略，而非搜索固定参数；Bayesian Optimization 则利用代理模型预测高收益区域；而随着大语言模型（LLM）的兴起，一些公司也开始尝试利用 LLM Agent 自动分析实验结果、生成新的排序策略，并进行超参数调整和实验规划。从形式上看，LLM Agent 的优化流程与传统 CEM 存在明显区别：

- CEM 基于概率分布进行随机搜索；
- Bayesian Optimization 基于代理模型进行搜索；

- 强化学习通过策略梯度学习最优策略；
- LLM Agent 则试图利用推理能力自动设计下一轮实验。

这些方法看起来越来越智能，也越来越自动化。但从工业界实践经验来看，它们最终仍然需要面对同样的问题：

- 数据噪声是否足够小；
- 反馈信号是否足够及时；
- 优化目标是否真正反映业务价值；
- 在线实验是否具有统计显著性；
- 用户长期价值是否被正确衡量。

事实上，在推荐系统自动寻优过程中，一个经常被忽视但又最关键的问题是：

 **笔记** 究竟什么指标才应该作为自动寻优的目标函数？

例如短视频推荐场景中：

- 小时级观看时长增长是否代表长期收益增长？
- 次日留存增长是否一定代表长期满意度提升？
- 互动率提升是否会损害用户体验？
- GMV 提升是否会降低内容生态健康度？

这些问题往往没有标准答案。

因此无论是 CEM、强化学习还是 LLM Agent，本质上都只是优化器（Optimizer）。真正决定系统最终效果的，仍然是目标函数设计、数据质量以及业务理解能力。很多时候，如何定义一个合理的优化目标，远比选择哪一种优化算法更加重要。这也是工业界推荐系统自动寻优领域至今仍然持续研究和探索的重要方向。

12.4.4 精排流量调控

精排流量调控（Traffic Control）是精排模块中的最后一个核心环节。在完成精排主模型、LTR 模型以及多目标融合之后，系统已经能够得到每个候选物品对应的融合排序分数。然而，在真实的工业场景中，仅仅按照融合分从高到低直接选取 Top-K 物品往往并不能满足业务需求。因此，在最终结果输出之前，通常还需要经过一层流量调控模块，对候选集合进行进一步约束与优化。

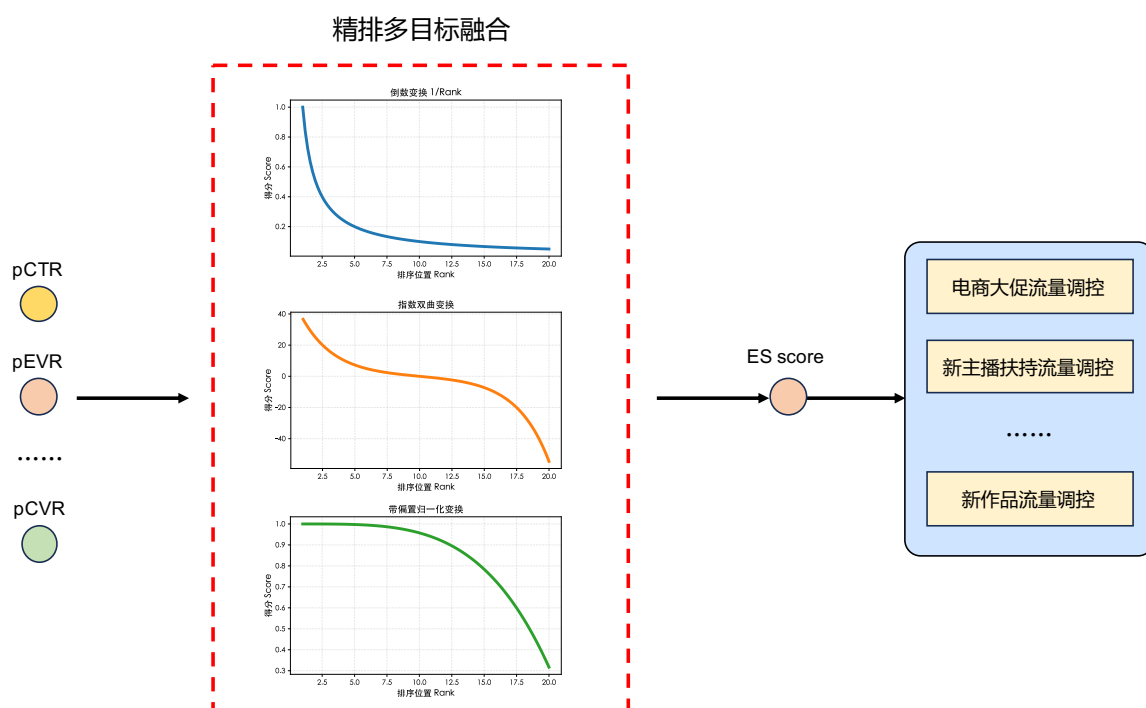


图 12.9: 精排级联式多目标融合与流量调控模块框架。

从系统架构视角来看，工业界精排链路的流量调控暂无统一落地规范，业内主流实现思路是将**多目标融合**与**流量调控**解耦为两个串行独立阶段，整体架构如图 12.9 所示。系统先经由多目标融合模块输出统一排序得分：在融合计算前，会对精排、LTR 模型输出的各类 PXTR 指标做自定义非线性打分变换，再依托加权加和或乘积融合公式完成多目标分数合并；得到基准排序列表后，进入后置流量调控环节，依托业务规则、配额约束与流量分配策略对已有排序做局部微调，不改动原始排序的整体相对次序。在流量分配实现上，既可以复用粗排多目标融合中介绍的分配额管控机制，也可针对重点扶持业务，通过动态 Boost 调整融合得分、限定区间浮动 Rank 等精细化方式实现定向流量倾斜。

另一种常见方案则是借鉴粗排阶段的多通道（Multi-Channel）或多 Quota 架构，如下图 12.10 所示。系统会根据不同业务目标来构建多个业务目标分别对应的候选池，然后针对每个候选池，内部独立完成多目标融合与排序，得到每个通道下各自的候选物品集合。然后再按照预先设定的 Quota 进行结果合并，最终多个通道共同组成最终的 Top-K 输出结果。在这种架构下，多目标融合实际上已经被嵌入到了各个流量通道内部，因此流量调控与排序融合往往是统一设计、统一优化的。

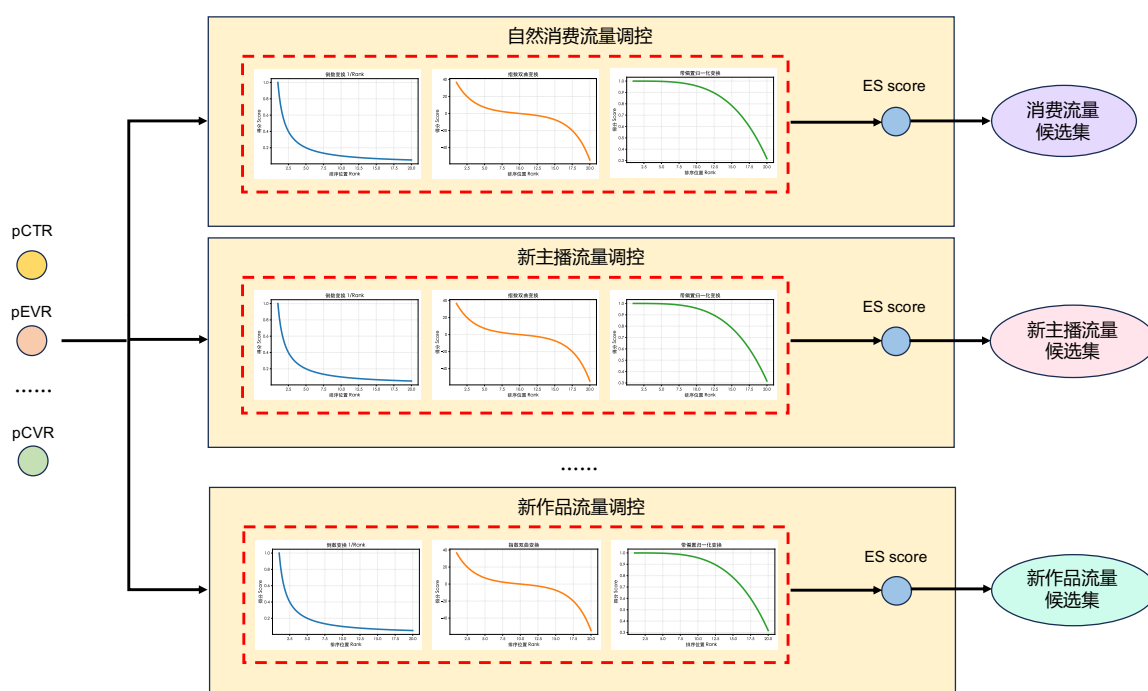


图 12.10: 精排多通道式流量调控模块框架。

从业务目标视角而言，精排阶段的流量调控核心价值在于调和**模型最优**与**业务最优**之间的天然矛盾。模型输出的排序分值仅拟合历史数据的统计分布规律，大量落地层面的业务约束难以被模型直接建模；若仅依靠 PXTR 指标经非线性融合后的分值筛选 Top-K 候选，该分数仅刻画用户消费行为偏好，无法承载繁杂的业务管控逻辑。工业推荐系统常见的业务约束可归纳为：

- 新内容冷启动扶持；
- 低粉丝创作者流量保护；
- 新晋主播成长资源倾斜；
- 电商大促货品保量投放；
- 商业化广告曝光保底；
- 优质稀缺内容探索导流；
- 长尾内容基础曝光兜底；
- 各垂类内容分发均衡管控；
- 用户推荐结果多样性约束。

上述生态与运营类目标难以嵌入模型损失函数做联合优化，因此需要在精排打分完成后，依托流量调控模块做

显式后置约束。以短视频推荐为例，纯模型驱动的排序容易使搞笑、娱乐等高点击内容垄断曝光，知识、生活、体育等垂类内容分发受限，虽短期核心指标利好，但会持续破坏平台内容生态；直播场景同理，完全依照预估收益排序会不断虹吸流量向头部主播集中，新晋与中腰部主播缺少成长空间，最终造成平台生态僵化。

由此可见，流量调控承担着平台生态治理（Ecosystem Governance）的关键职能。这也体现了工业推荐系统的复杂性：用户即时转化、点击率预估这类用户行为规律可交由模型拟合优化，而刚性业务规则、运营配额约束与平台生态平衡等诉求，则需要通过策略调控的方式落地实现。

另一方面，从优化问题的角度来看，精排流量调控实际上可以被看作一个带约束条件的排序优化问题：

$$\max \sum_{i=1}^N Score_i \quad (12.103)$$

同时满足：

$$\sum_i x_i^{new} \geq Q_{new} \quad (12.104)$$

$$\sum_i x_i^{creator} \geq Q_{creator} \quad (12.105)$$

$$\sum_i x_i^{commerce} \geq Q_{commerce} \quad (12.106)$$

，其中 $x_i \in \{0, 1\}$ 表示候选物品是否被最终选中， Q 表示对应业务目标要求的最小流量配额。因此，精排流量调控实际上是在最大化排序收益与满足业务约束之间寻找平衡点。

近年来随着强化学习（Reinforcement Learning）、组合优化（Combinatorial Optimization）以及大模型 Agent 技术的发展，一些先进推荐系统已经开始尝试将流量调控问题建模为动态决策问题。系统不再依赖人工配置固定 Quota，而是根据实时流量状态、内容供给情况以及用户反馈动态调整流量分配策略。例如：

$$Q_t = f(S_t^{user}, S_t^{Content}, S_t^{Platform}) \quad (12.107)$$

其中，Quota Q_t 会随着实时环境变化而动态更新。不过从目前工业界实践来看，大多数线上系统仍然以规则策略、Quota 控制以及轻量级优化框架为主，而强化学习和大模型 Agent 类方案更多应用于部分复杂业务场景或者离线辅助决策系统之中。

除了前面介绍的基于 Quota 的流量调控机制之外，工业界推荐系统中还广泛存在另一类更加直接的流量控制方式，即保量（Guarantee Delivery）机制。与 Quota 更多关注不同内容池之间的流量分配比例不同，保量机制关注的是某一类特定候选集合是否能够获得最低限度的曝光机会，从而满足平台的业务目标和运营需求。

在实际业务中，推荐系统并不总是单纯追求短期点击率、观看时长等指标的最大化。对于一些处于萌芽期的新业务、新内容形态或者平台重点扶持的方向，如果完全依赖模型排序，其流量往往会被成熟业务和头部内容挤压，难以获得成长空间。因此，平台通常会为这些业务设置专门的保量策略，确保其能够获得稳定的曝光机会。例如新上线的内容频道、新的视频互动功能、新的电商业务形态等，都可能在早期通过保量机制获得冷启动流量支持。

此外，保量机制也广泛应用于创作者生态建设中。例如平台运营团队筛选出的优质创作者、优质主播、重点签约作者或者官方活动参与创作者等，往往会进入特定的白名单池。对于这些候选集合，系统会根据业务目标设置最低曝光量、最低曝光人数或者最低流量占比等约束条件，从而保证优质内容能够被用户发现，而不会因为短期数据不足而被排序模型完全压制。

从实现方式来看，保量机制通常可以表示为一种带约束的排序优化问题。在最终结果生成过程中，除了优化整体排序收益之外，还需要满足：

$$Exposure_{target} \geq Q_{target} \quad (12.108)$$

其中， $Exposure_{target}$ 表示目标候选集合实际获得的曝光量， Q_{target} 表示业务要求的最低保量目标。

在工程实践中，保量可以通过多种方式实现。例如在排序完成后进行强插（Force Insert），直接将符合条件的候选物品插入最终结果；也可以在流量调控阶段预留固定坑位；或者将保量约束直接融入多目标融合与流量分配框架之中，通过动态调整排序分数实现更加平滑的流量控制。保量机制本质上是在模型最优与业务最优之间寻找平衡，一方面推荐系统需要充分利用模型能力提升用户体验和核心指标；另一方面也需要兼顾业务发展、内容生态建设以及平台长期价值。因此，保量机制往往是精排流量调控模块中不可或缺的重要组成部分，也是策略算法与产品运营深度结合的典型体现。

总体而言，精排流量调控并不仅仅是简单的保量或者 Quota 控制模块，而是连接算法目标与平台业务目标的重要桥梁。一方面需要尽可能保留模型排序带来的收益，另一方面又需要兼顾内容生态、创作者成长、商业化目标以及长期用户价值等复杂约束。正因如此，流量调控往往也是推荐系统中策略算法工程师参与最深、业务理解要求最高的模块之一。

12.5 重排模块

在工业级推荐系统中，重排（re-ranking）模块通常位于精排（full-ranking）模块之后，其核心作用是在精排输出的候选列表基础上，引入业务规则与策略目标进行二次调整。典型的应用包括提升结果多样性、控制广告曝光优先级、调节流量分发等，以确保最终推荐结果更契合平台的整体运营目标。然而，读者可能会产生一个根本性的疑问：**为何需要专门设置重排模块？**要回答这一问题，关键在于厘清精排与重排在目标与方法上的本质差异。

传统精排模块通常采用复杂的深度神经网络（Deep Neural Network, DNN）对每个候选物品进行单点式（point-wise）打分，并依据得分高低直接排序。这种策略本质上是一种**贪心选择**——它仅追求每个位置上的局部最优，而忽视了整个推荐序列的全局效果。以短视频推荐场景为例（如图 12.11 所示），用户在应用中通过上下滑动浏览内容，系统依次展示短视频。出于排版示意，图中将视频横向排列，但实际交互是纵向连续的。在此产品形态下，用户通常无法预知下一个视频的内容，其在单次会话（session）中消费的视频数量，与视频序列的整体排列顺序密切相关。若排序不佳，用户可能在浏览几条后便退出，导致人均消费深度下降，损害用户体验。



图 12.11: 短视频序列示意图。

此外，观察数据还揭示了一个显著现象：随着用户在会话中浏览位置的后移，其单条视频的观看时长往往呈递减趋势。这反映出强烈的**位置偏置**（position bias）：即用户对靠前位置的内容天然赋予更高注意力，而对后续内容的耐心逐渐衰减。这种由产品交互形态引发的用户行为偏差，是推荐系统在分发策略中必须重点考量的因素。正是由于上下滑动的产品设计及其带来的位置偏置特性，工业界实践中发现：仅依赖精排阶段的单点打分排序，难以实现全局最优的推荐效果，极易陷入局部最优陷阱。因此，重排模块应运而生，其核心目标是从**整个推荐序列的视角**出发，进行更全局、更协同的排序优化。

值得进一步追问的是：**为何不在精排阶段就直接进行序列级的全局优化**？答案在于计算复杂度与工程可行性的权衡。假设精排阶段的候选集规模为 500，而最终需展示的视频数量为 8，则可能的排列组合总数高达 500^8 ，搜索空间极其庞大，难以在毫秒级响应要求下完成优化。相比之下，在精排已筛选出 top-100 的高质量候选后，仅从对这 100 个物品的顺序进行重排，其优化空间显著缩小 (100^8)。为进一步压缩搜索空间，也可以将候选集截断至最终的 top K ($K < 10$) 之后再重新排序，此时最多仅有 $K!$ 种排列。这一策略既大幅降低了计算开销，又能在有限延迟内实现更优的序列级决策。因此，重排模块的引入，是在保证系统效率的前提下，对推荐结果进行精细化、策略化与全局化调优的关键一环。

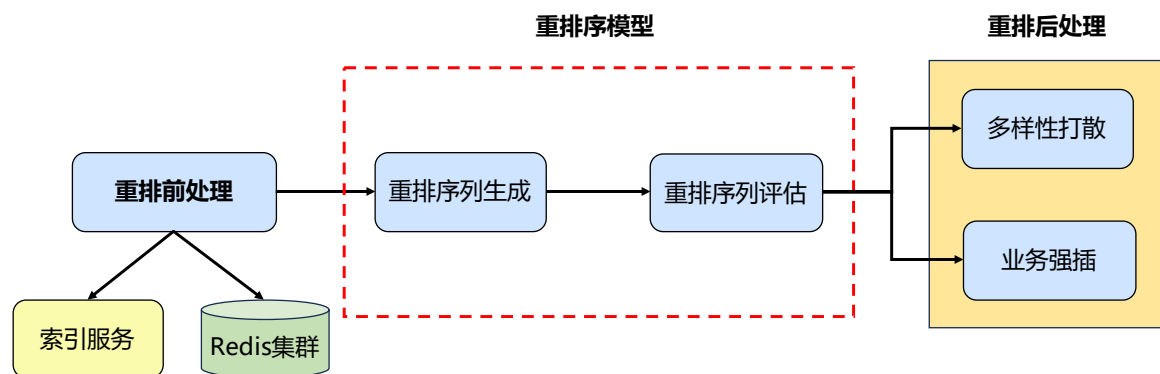


图 12.12: 重排模块内部处理流程。

重排模块 (Reranking) 的内部处理流程如图 12.12 所示，主要包括**重排前处理 (Pre-processing)**、**序列生成 (List Generation)**、**序列评估 (List Evaluation)** 以及**多样性打散 (Diversification)** 等多个环节。其中，重排前处理阶段主要负责从外部正排索引服务、Redis 集群以及其他在线存储系统中获取重排所需的特征信息，并完成候选物品属性初始化、特征补充以及必要的预处理工作，为后续重排计算做好准备。在整个重排模块中，序列生成与序列评估构成了最核心的技术框架。目前工业界主流推荐系统普遍采用经典的**两阶段重排架构 (Two-Stage Reranking Framework)**，即“**序列生成 (List Generator) + 序列评估 (List Evaluator)**”的范式。其中，序列生成阶段负责构造若干候选排序序列，而序列评估阶段则负责从多个候选序列中选择最优结果，从而实现用户体验、业务目标以及内容生态等多方面因素的综合优化。相比于召回、粗排和精排阶段主要关注单个候选物品的价值评估，重排阶段关注的对象已经从单个 Item 上升到整个推荐序列 (List) 层面。因此，重排模块不仅需要考虑单个物品本身的质量，还需要关注候选物品之间的相互关系。例如相邻内容是否过于相似、同一创作者内容是否出现过于集中、不同垂类内容是否具有合理分布，以及广告、直播、电商等特殊内容在整个推荐序列中的插入位置是否合适等问题。

近年来，随着生成式模型和大语言模型的发展，也出现了一些新的重排范式。例如部分研究尝试采用**一阶段式重排架构 (One-Stage Reranking)** 直接对最终序列进行优化，或者使用**生成式重排网络 (Generative Reranking Network, GRN)** 通过生成模型直接产生推荐序列。这些方法能够在更大程度上建模序列内部复杂的依赖关系，但由于这些新颖的方法并未完全普及，目前大规模工业系统中仍然以两阶段重排架构为主。关于这些前沿方向，我们将在后续第 13 章和第 15 章中进行详细介绍。

此外，读者还需要注意的是，重排模块并不是所有推荐系统都必须具备的标准模块，其是否存在与具体业务形态密切相关。如果推荐内容本身构成一个连续消费的信息流，例如短视频推荐、图文推荐或者信息流推荐场景，那么用户会连续浏览多个内容，系统需要从整体序列层面考虑内容多样性、创作者分布、商业化内容插入以及用户长期体验等问题，此时重排模块通常是不可或缺的。然而，对于一些非主体内容推荐系统，例如直播推荐系统、广告推荐系统或者电商活动推荐系统，其推荐结果往往只是作为主信息流中的少量插入内容存在。例如在短视频信息流中，广告位和直播位通常只占据整体流量中的很小比例。对于这类场景而言，推荐系统更加关注单个候选内容的匹配效果，而较少关注整个序列层面的优化问题，因此很多时候并不会单独设计复杂的重排模块，而是直接依赖精排结果完成最终推荐。

因此，从本质上来看，重排模块是否需要存在，并不取决于推荐系统架构本身，而取决于业务场景是否需

要从“单物品最优”进一步优化到“整体序列最优”。当推荐目标从单个内容质量提升扩展到用户整体浏览体验优化时，重排模块的重要性便会逐渐凸显出来。

12.5.1 序列生成

在序列生成过程中，本质上是在序列的高维空间中搜索潜在的最优序列集合。由于搜索空间依然较大，通常难以进行暴力枚举——线上系统的算力与延迟约束无法承受。因此，序列生成常被视为**序列维度的召回阶段**。目前工业界常用的序列生成方法众多，其中两类经典策略被广泛采用。一类是**基于多目标融合排序的贪心序列生成方法**，另一类是**基于束搜索（beam search）的序列生成方法**。基于多目标融合排序的贪心序列生成方法通过对每个候选物品在多个业务目标（如有效播放率 $pevr$ 、长播率 $plvr$ 、点赞率 $pltr$ 、评论率 $pcmr$ 、转发率 $pftr$ 等）上的预估值进行加权融合，得到一个综合得分，并依此直接排序生成推荐序列。以短视频场景为例，假设候选集包含 5 个物品，其各项指标的预估值如表 12.5 所示。

表 12.5: 重排序多目标融合序列生成阶段物品示例 $pxtr$ 预估值。

物品 ID	预估有效播放率 $pevr$	预估长播率 $plvr$	预估点赞率 $pltr$	预估评论率 $pcmr$	预估转发率 $pftr$
item_0	0.374540	0.020584	0.611853	0.607545	0.122038
item_1	0.950714	0.969910	0.139494	0.170524	0.495177
item_2	0.731994	0.832443	0.292145	0.065052	0.034389
item_3	0.598658	0.212339	0.366362	0.948886	0.909320
item_4	0.156019	0.181825	0.456070	0.965632	0.258780

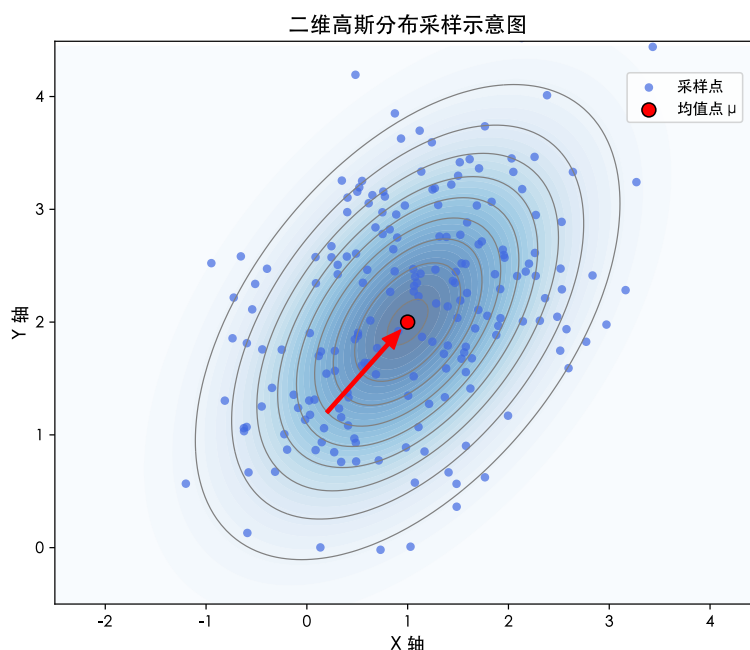


图 12.13: 多维高斯分布采样示意图。

对于上述的候选物品，可以设置一个排序的融合公式，比如线性融合的排序公式：

$$\text{final_score} = w_1 \times p_evr + w_2 \times p_lvr + w_3 \times p_ltr + w_4 \times p_cmr + w_5 \times p_ftr \quad (12.109)$$

然后根据 final_score 即可进行排序得到一个贪心策略生成的序列。由于在上述排序公式中有 w_1 、 w_2 、 w_3 、 w_4 、 w_5 这 5 个参数，因此，我们可以将其整合成一个 5 维的参数向量，即

$$\mathbf{w} = [w_1, w_2, w_3, w_4, w_5] \quad (12.110)$$

在实际应用中，可首先根据人工经验设定权重向量 $\mathbf{w}$ 的基准值，并将其作为均值向量，通过多元高斯分布进行采样，从而生成多组不同的权重组合，以获得多样化的排序结果。为便于读者直观理解高斯采样的含义，图 12.13 以二维高斯分布为例，展示了在参数向量附近进行扰动采样的过程。图中红色点表示初始权重向量的位置，即高斯分布的均值向量；周围的蓝色点则为从该多元高斯分布中采样得到的样本点。可以观察到，距离红色点越远的采样点数量越少，体现了高斯分布在均值附近具有更高概率密度的特性。

详细的算法代码实现如下 12.1 和 12.2 所示。在这里首先我们可以随机初始化物品的 `pxtr` 特征，注意在实际应用中需要替换成真实的 `pxtr`。

代码 12.1: 基于多目标融合排序的贪心序列生成算法代码（初始化）

```
import numpy as np
import pandas as pd

# ===== 模拟物品特征 =====
num_items = 10
np.random.seed(42)

# 每个物品的5个特征: p_evr, p_lvr, p_ltr, p_cmr, p_ftr
items = pd.DataFrame({
    'item_id': [f'item_{i}' for i in range(num_items)],
    'p_evr': np.random.rand(num_items),
    'p_lvr': np.random.rand(num_items),
    'p_ltr': np.random.rand(num_items),
    'p_cmr': np.random.rand(num_items),
    'p_ftr': np.random.rand(num_items)
})
print("示例物品特征: ")
print(items.head())
```

接下来我们可以定义高斯采样的数据分布并基于高斯采样之后的权重来生成重排序序列。其中，协方差矩阵作为超参数控制探索范围：协方差越小，采样越集中；反之则探索更广。

代码 12.2: 基于多目标融合排序的贪心序列生成算法代码（采样与序列生成）

```
# ===== 定义高斯采样 =====
mean = np.array([0.2, 0.3, 0.2, 0.2, 0.1]) # 权重均值
cov = np.diag([0.01]*5) # 协方差矩阵（独立采样）
num_samples = 5 # 生成5组不同的权重组合
weights_samples = np.random.multivariate_normal(mean, cov, num_samples)
print("\n采样得到的权重向量: ")
print(weights_samples)

# ===== 基于权重生成重排序序列 =====
rerank_sequences = []
for i, w in enumerate(weights_samples):
    # 计算每个物品的 final_score
    items['final_score'] = (
```

```

w[0]*items['p_evr'] + w[1]*items['p_lvr'] + w[2]*items['p_ltr'] + w[3]*items['p_cmr'] + w[4]*items['p_ftr']
)
# 根据 score 排序
ranked = items.sort_values(by='final_score', ascending=False)['item_id'].tolist()
rerank_sequences.append({
    'sample_id': i, 'weights': w, 'ranking': ranked
})

# ===== 输出示例 =====
for seq in rerank_sequences:
    print(f"\n采样{seq['sample_id']}权重:{np.round(seq['weights'],3)}")
    print(f"排序结果:{seq['ranking']}")

```

尽管该方法利用了多目标信息，但其本质仍是贪心策略，每一步仅选取当前得分最高的物品。虽然计算高效，却易陷入局部最优。因此，需依赖大量采样以扩大覆盖范围。为在贪心搜索与穷举搜索之间取得平衡，**束搜索**（beam search）成为一种更具探索能力的替代方案。该算法最早由 Lowerre 在 1976 年的语音识别系统 Harpy 中提出 [23]，随后 Rubin 与 Reddy 在 1977 年 IJCAI 会议论文中将其形式化为一种非回溯的确定性启发式搜索方法 [31]。此后，Beam Search 被广泛应用于机器翻译 [11, 36]、文本生成 [29]、强化学习序列建模 [16] 等多个领域。

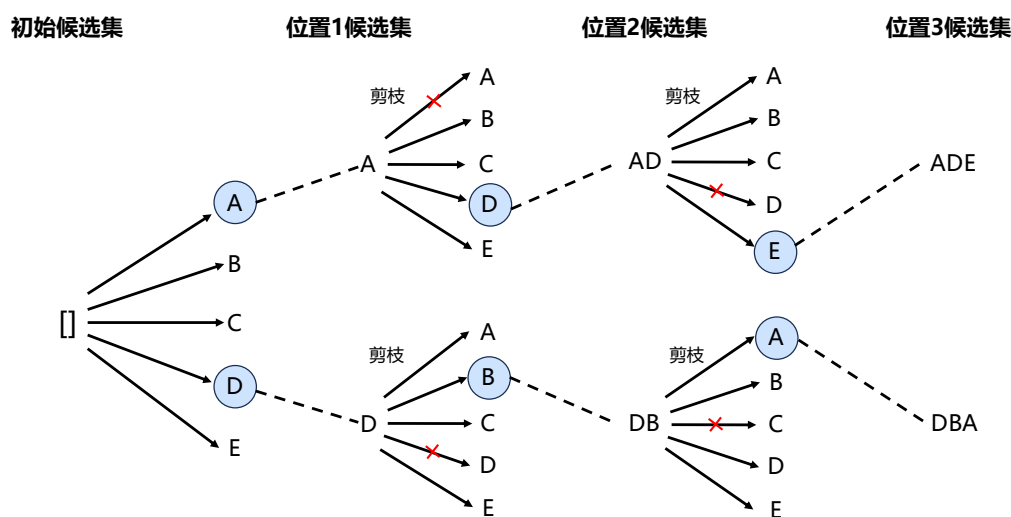


图 12.14: Beam Search 算法基本原理示意图。

如图 12.14 所示，Beam Search 在每一步扩展时保留 top-K 条候选路径（即“束宽”），并在搜索过程中对不合法序列进行剪枝。例如，可嵌入业务规则：相同垂类内容不得连续出现。此类约束可在中间步骤提前剔除无效路径，提升搜索效率。下面的代码展示了一种融合高斯采样与 Beam Search 的序列生成实现：在每一步不仅扩展候选序列，还动态采样新的权重向量，以增强探索能力。当然，也可在搜索全程固定权重，但会牺牲部分多样性。

首先仍然是每个物品 p_xtr 的初始化和相关 Python 库函数的导入，如 12.3 所示。

代码 12.3: 基于 Beam Search 的序列生成算法代码（初始化）

```

import numpy as np
import pandas as pd

# ===== 模拟物品特征 =====
num_items = 10

```

```

np.random.seed(42)

# 每个物品的5个特征: p_evr, p_lvr, p_ltr, p_cmr, p_ftr
items = pd.DataFrame({
    'item_id': [f'item_{i}' for i in range(num_items)],
    'p_evr': np.random.rand(num_items),
    'p_lvr': np.random.rand(num_items),
    'p_ltr': np.random.rand(num_items),
    'p_cmr': np.random.rand(num_items),
    'p_ftr': np.random.rand(num_items)
})

print("示例物品特征: ")
print(items.head())

```

接下来，我们定义权重向量，并且在 beam search 的过程中不断更新权重向量，同时完成序列的生成。以下12.4是 beam search 和序列生成部分的核心代码。

代码 12.4: 基于 Beam Search 的序列生成算法代码 (beam search)

```

# ===== 定义权重参数 =====
mean = np.array([0.2, 0.3, 0.2, 0.2, 0.1]) # 权重均值
cov = np.diag([0.01]*5)                  # 协方差矩阵 (独立采样)

# ===== 基于 Beam Search 生成重排序序列 =====
def beam_search_rerank(items, mean, cov, beam_size=3, num_steps=5):
    # 初始化 beam: 每个元素为 (当前序列, 当前分数, 权重)
    initial_weights = np.random.multivariate_normal(mean, cov, beam_size)
    beam = []

    for w in initial_weights:
        # 计算初始分数
        items['final_score'] = (
            w[0]*items['p_evr'] + w[1]*items['p_lvr'] + w[2]*items['p_ltr'] + w[3]*items['p_cmr'] + ↵
            ↵ w[4]*items['p_ftr']
        )
        # 初始排序
        ranked = items.sort_values(by='final_score', ascending=False)['item_id'].tolist()
        # 计算序列总分数 (使用 topN 分数和)
        total_score = items['final_score'].nlargest(len(ranked)).sum()
        beam.append((ranked, total_score, w))

    # beam search 迭代优化
    for step in range(num_steps):
        candidates = []
        # 扩展当前 beam 中的每个候选

        for seq, score, w in beam:
            # 生成新的权重候选

```

```

new_weights = np.random.multivariate_normal(w, cov, beam_size)
# 基于新的权重进行序列生成
for new_w in new_weights:
    # 基于新权重计算分数
    items['final_score'] = (
        new_w[0]*items['p_evr'] + new_w[1]*items['p_lvr'] + new_w[2]*items['p_ltr'] + ↵
        ↵ new_w[3]*items['p_cmr'] + new_w[4]*items['p_ftr']
    )
    # 生成新排序
    new_ranked = items.sort_values(by='final_score', ascending=False)['item_id'].tolist(↵
        ↵ ())
    new_score = items['final_score'].nlargest(len(new_ranked)).sum()
    candidates.append((new_ranked, new_score, new_w))

# 筛选出最优的 beam_size 个候选
candidates.sort(key=lambda x: x[1], reverse=True)
beam = candidates[:beam_size]
return beam

```

最后是整个基于 beam search 的序列生成算法代码的程序入口，包括了对于整个 beam search 函数的调用以及输出最终生成的序列结果，如下12.5所示。

代码 12.5: 基于 Beam Search 的序列生成算法代码（程序入口）

```

# 运行 beam search
beam_size = 3
num_steps = 5
best_sequences = beam_search_rerank(items, mean, cov, beam_size, num_steps)
# ===== 输出结果 =====
print(f"\nBeamSearch结果(beam_size={beam_size},steps={num_steps})")
for i, (ranking, score, weights) in enumerate(best_sequences):
    print(f"\n候选{i+1}权重:{np.round(weights,3)}")
    print(f"序列分数:{np.round(score,3)}")
    print(f"排序结果:{ranking}")

```

12.5.2 序列评估

重排序的序列生成过程通常由多种方法并行提供候选序列，而序列评估则相对固定，普遍采用监督学习范式对整个序列的价值进行建模。以短视频上下滑场景为例，最简化的评估方式是将序列中每个物品的单点分数 $f(x_i)$ 按位置衰减加权求和。这里的分数 $f(x_i)$ 可以是单一的标签，比如观看时长，也可以是多个目标经过线性或非线性融合之后的融合分数。考虑到位置偏置，权重可设为 $w_i = \frac{1}{i}$ 或指数衰减形式 $w_i = e^{-(i-1)}$ ，则序列价值定义为：

$$\text{value} = \sum_{i=1}^N w_i \times f(x_i) \quad (12.111)$$

基于此，可采用回归损失进行训练：

$$\mathcal{L} = \frac{1}{B} \sum_{i=1}^B \|\hat{y}_i - \text{value}_i\|^2 \quad (12.112)$$

其中 B 是整个序列数据的批次大小。

更简化的方式是直接以序列整体观看时长 y 为标签，并通过阈值转化为二分类任务：

$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^B y_i \log \hat{y}_i + (1 - y_i) \log (1 - \hat{y}_i) \quad (12.113)$$

第三种更贴合产品逻辑的评估方式，基于用户“是否继续下滑”的行为建模曝光概率。设用户打开 App 的初始概率为 p_0 ，观看第 j 个视频后继续下滑的概率为 $p_{\text{slide}}(x_j)$ ，则第 i 个物品的曝光概率为：

$$p(\text{item}_i) = \begin{cases} p_0, & i = 1 \\ p_0 \prod_{j=1}^{i-1} p_{\text{slide}}(x_j), & i > 1 \end{cases} \quad (12.114)$$

相应地，第 i 个物品的价值为：

$$\text{value}_i = p(\text{item}_i) \times f(x_i) \quad (12.115)$$

而整个序列 L 的整体价值定义为：

$$\text{value} = \sum_{i=1}^N \text{value}_i \quad (12.116)$$

在实际训练中，若序列价值为各物品价值之和，则模型重点在于准确预估每个位置的物品价值。此时可对每个物品分别施加损失：

$$\begin{aligned} \mathcal{L} &= \frac{1}{B} \sum_{i=1}^B \sum_{j=1}^N \mathcal{L}_{ij} \\ \mathcal{L}_{ij} &= \|\hat{y}_{ij} - \text{value}_{ij}\|^2 \end{aligned} \quad (12.117)$$

从强化学习的视角来看，重排模块中的序列生成（List Generator）可以类比为 **Actor**，负责在巨大的序列空间中生成候选排序方案；而序列评估（List Evaluator）则对应 **Critic**，用于评估不同排序序列的质量，并为生成策略提供优化信号。近年来，强化学习在序列生成阶段得到了广泛应用，例如利用策略梯度（Policy Gradient）、蒙特卡洛树搜索（Monte Carlo Tree Search, MCTS）等方法提升序列搜索效率和生成质量。然而，完全使用强化学习替代传统重排流程在工业界仍面临诸多挑战，例如 Q 值估计不稳定、样本效率较低、训练过程难以收敛，以及线上探索带来的业务风险等问题。因此，目前大多数工业界推荐系统仍然采用“序列生成 + 序列评估”的两阶段重排架构，而强化学习更多作为其中某些关键环节的优化手段进行局部应用。关于强化学习在推荐系统中的具体实践，包括强化学习重排、流量调控、排序公式超参数寻优以及广告竞价等典型应用，我们将在第 17 章中进行详细介绍。

12.5.3 多样性打散与强插

重排模块的最后一个阶段是**重排后处理**（post-processing），其核心任务是在序列评估选出的最优候选序列基础上，进一步嵌入强约束性业务逻辑，以确保最终输出的推荐结果既符合算法优化目标，又满足产品运营与用户体验的多维要求。其中，最为关键的两个环节是**多样性打散**（diversity diversification）与**业务强插**（business-level forced insertion）。

多样性打散旨在缓解推荐结果中内容同质化的问题，避免系统在短时间内向用户密集推送同类内容，从而引发“信息茧房”效应或审美疲劳。大量实践表明，若相同类别（如影视、美食、萌宠等）的内容在推荐列表中连

续或高频出现，用户极易产生倦怠感，进而降低滑动意愿、缩短单次使用时长，甚至影响长期留存。因此，引入多样性机制不仅是提升用户体验的重要手段，更是保障平台核心指标（如人均观看时长、次日留存率等）的关键策略。

目前，工业界主流的多样性打散方法可分为两大类：**基于规则的多样性打散**与**基于模型的多样性打散**。**基于规则的多样性打散**逻辑清晰、实现简单、线上可控性强，是大多数推荐系统初期或对稳定性要求较高的场景中的首选方案。其典型做法是对特定内容类别在推荐列表中的出现频次或位置间隔施加硬性约束。例如，在单次推荐列表长度为 10 的短视频场景中，可设定“10 出 1”规则：即最多允许 1 个短剧类长视频出现在该列表中。更进一步，还可引入**跨刷次的滑动窗口约束**，例如在用户最近滑动的 20 个视频构成的窗口内，影视类内容不得超过 3 个。此类规则虽显“粗粒度”，但因其可解释性强、调试便捷，常被用于兜底保障，尤其适用于广告、热点、合规等强业务诉求场景。

相比之下，**基于模型的多样性打散**则试图在保持相关性的同时，以更精细、更自适应的方式建模内容间的差异性。这类方法通常将多样性视为一个可优化的目标函数，与相关性进行联合权衡。其中，三种经典算法被广泛研究与应用：

- **MMR** (Maximal Marginal Relevance, 最大边缘相关) [3]: 通过在每一步选择与用户兴趣最相关、同时与已选内容最不相似的物品，实现相关性与多样性的贪心平衡。其核心思想是最大化边际收益，形式简洁，易于嵌入现有排序流程。
- **DPP** (Determinantal Point Process, 行列式点过程) [19]: 一种基于概率点过程的子集选择模型，能够自然地刻画物品间的“排斥性”(repulsion)，从而在全局范围内生成高相关性且高多样性的推荐子集。尽管 DPP 理论优美，但其计算复杂度较高，通常需结合采样或近似算法以适配线上延迟要求。
- **SSD** (Sliding Spectrum Decomposition, 滑动频谱分解) [15]: 一种面向序列推荐场景的高效多样性算法，通过在滑动窗口内对内容特征进行谱分解，动态识别并抑制冗余模式。SSD 特别适合处理长序列、高吞吐的推荐场景，在保证多样性的同时对系统性能影响较小。

在实际系统中，上述方法往往并非孤立使用，而是根据业务阶段、资源预算与效果目标进行组合。例如，可先通过规则打散进行粗筛，再以 MMR 或 SSD 进行细粒度优化；或在 DPP 生成候选子集后，叠加业务强插逻辑（如插入运营活动视频、公益内容等），确保策略落地。这种“规则兜底 + 模型优化 + 业务干预”的多层次后处理架构，已成为现代工业级推荐系统重排阶段的标准范式。

下面我们重点介绍一下 MMR、DPP 和 SSD 这三种多样性打散算法以及它们之间的区别。

12.5.3.1 MMR 算法

MMR (Maximal Marginal Relevance, 最大边际相关性) 算法最早由 Carbonell 与 Goldstein 于 1998 年在论文《The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries》中提出 [3]。其核心思想是在文本检索重排序与摘要生成任务中，同时兼顾**查询相关性**与**信息新颖性**，从而在保留关键信息的同时有效减少结果冗余，尤其在多文档摘要任务中展现出显著优势。从向量空间的角度理解，所谓“信息新颖性”即体现为候选内容与已选内容之间的**低相似性**（或高不相似性）。

设 C 为物品的候选集合， Q 为查询，经初始信息检索后得到的结果集记为 $R = IR(Q, C, \theta)$ 。令 S 表示当前已从 R 中选出的子集，则 $R \setminus S$ 表示尚未被选中的剩余候选。在此设定下，MMR 的优化目标可形式化为：

$$MMR \stackrel{def}{=} \arg \max_{D_i \in R \setminus S} [\lambda (Sim_1(D_i, Q) - (1 - \lambda) \max_{D_j \in S} Sim_2(D_i, D_j))] \quad (12.118)$$

其中， $\lambda \in [0, 1]$ 是一个超参数，用于平衡**边际相关性** (Marginal Relevance) 与**相关性创新度** (Relevant Novelty)； Sim_1 衡量候选文档 D_i 与查询 Q 的相关性， Sim_2 则度量候选文档 D_i 与已选集合 S 中任一文档 D_j 的相似性。两者可采用相同的相似度度量（如余弦相似度），也可根据任务需求分别设计。

在信息检索的实际应用中，MMR 算法通常以如下方式执行：首先初始化一个空集合 S ，并从初始结果集 R 中选取与查询 Q 相关性最高的文档作为 S 的首个元素。随后，通过迭代过程，对 $R \setminus S$ 中的每个剩余候选文档，依据公式 12.118 计算其 MMR 分数——该分数综合了其与查询的相关性以及与其与已选文档的最大相似度（即冗余

程度)。每次迭代选择 MMR 分数最高的文档加入 S ，直至 S 达到预设的规模。最终得到的结果集在保持高相关性的同时，显著降低了内容冗余。

基于上述流程，信息检索场景下的 MMR 算法核心实现如代码清单 12.6 所示：

代码 12.6: 基于 MMR 算法的信息检索代码

```
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.feature_extraction.text import TfidfVectorizer

def mmr_selection(query: str, documents: list, n_selected: int = 5, lambda_param: float = 0.5, ←
    ↪ vectorizer=None) -> list:
    # 初始化文本向量化工具
    if vectorizer is None:
        vectorizer = TfidfVectorizer(stop_words='english') # 英文停用词处理
    # 合并查询和文档，统一向量化（确保向量空间一致）
    all_texts = [query] + documents
    tfidf_matrix = vectorizer.fit_transform(all_texts)
    # 提取查询向量和文档向量
    query_vec = tfidf_matrix[0:1] # 第0个是查询向量
    doc_vecs = tfidf_matrix[1:] # 后续是文档向量（与 documents 顺序对应）
    # 计算所有文档与查询的相关性（余弦相似度）
    relevance = cosine_similarity(doc_vecs, query_vec).flatten() # 形状: (n_docs,)
    # 初始化已选集合 (S) 和待选集合（剩余文档索引）
    selected_indices = []
    remaining_indices = list(range(len(documents)))
    # 第一次选择：仅按相关性（因为 S 为空，无法计算多样性）
    if remaining_indices:
        first_idx = np.argmax(relevance[remaining_indices])
        first_idx = remaining_indices[first_idx] # 转换为原始索引
        selected_indices.append(first_idx)
        remaining_indices.remove(first_idx)
    # 迭代选择：平衡相关性和多样性
    for _ in range(n_selected - 1):
        if not remaining_indices:
            break # 待选集合为空时停止
        mmr_scores = []
        for doc_idx in remaining_indices:
            # 1. 计算当前文档与查询的相关性（已归一化）
            rel_score = relevance[doc_idx]
            # 2. 计算当前文档与已选文档的最大相似度（多样性指标）
            # 已选文档向量
            selected_vecs = doc_vecs[selected_indices]
            # 当前文档与所有已选文档的相似度
            sim_to_selected = cosine_similarity(doc_vecs[doc_idx:doc_idx+1], selected_vecs).flatten() ←
            ↪ ()
            max_sim = np.max(sim_to_selected) # 取最大相似度（最冗余的已选文档）
            # 3. 计算MMR分数：*相关性 - (1-)*最大冗余度
            mmr_score = lambda_param * rel_score - (1 - lambda_param) * max_sim
```

```

        mmr_scores.append(mmr_score)
    # 选择MMR分数最高的文档
    best_idx = remaining_indices[np.argmax(mmr_scores)]
    selected_indices.append(best_idx)
    remaining_indices.remove(best_idx)
    # 返回筛选后的文档（按选择顺序）
    return [documents[i] for i in selected_indices]

```

为验证上述实现，可使用构造的样例数据进行测试。代码清单 12.7 提供了一个完整的调用示例，展示了如何利用 MMR 从一组与查询相关的文档中筛选出低冗余、高相关性的结果子集：

代码 12.7: 基于 MMR 算法的信息检索代码 (程序入口)

```

if __name__ == "__main__":
    # 示例数据：与 "人工智能在医疗中的应用" 相关的文档
    query = "人工智能 医疗 应用"
    documents = [
        "人工智能可用于医学影像识别，辅助医生检测肿瘤。",
        "医学影像识别是人工智能在医疗中的重要应用，尤其擅长肺癌筛查。", # 与第1篇高度相似
        "AI技术能分析电子病历，预测患者的疾病风险。",
        "电子病历分析可帮助医生快速判断患者病情，AI在其中起到关键作用。", # 与第3篇高度相似
        "人工智能驱动的药物研发能缩短新药上市时间。",
        "AI在手术机器人中的应用提高了微创手术的精度。",
        "手术机器人结合AI技术后，患者术后恢复时间显著缩短。" # 与第6篇高度相似
    ]

    # 用MMR筛选3篇文档（=0.6：稍侧重相关性）
    selected = mmr_selection(
        query=query,
        documents=documents,
        n_selected=3,
        lambda_param=0.6,
        # 中文需使用适合的向量器（这里用默认参数，实际可优化）
        vectorizer=TfidfVectorizer(analyzer='word', stop_words=['的', '是', '在'])
    )

    # 输出结果
    print("筛选后的文档：")
    for i, doc in enumerate(selected, 1):
        print(f"{i}. {doc}")

```

在推荐系统场景中，MMR 同样适用于重排序阶段的多样性控制。此时，系统通常已通过精排或重排模型获得了 $\text{top-}K$ 个候选物品及其对应的打分。这些打分可直接作为公式 12.118 中的第一项 $\text{Sim}_1(D_i, Q)$ ，即物品与用户意图（或上下文）的相关性度量。而第二项物品间的相似度 $\text{Sim}_2(D_i, D_j)$ ，则可借助推荐模型中物品 ID 对应的 embedding 向量，通过点积或余弦相似度进行计算。

值得注意的是，若将多样性打散机制前置至精排模块，也可将 MMR 视为一种贪心策略：从 $\text{top-}N$ 候选中逐步选择 M ($M < N$) 个物品，使得最终集合在相关性与多样性之间取得平衡。尽管如此，**重排序阶段仍是实现多样性控制的关键环节**，因其能基于更完整的上下文信息进行全局优化。当然，也可在前链路模块中进行初步尝试，形成多阶段协同的多样性保障机制。

12.5.3.2 DPP 算法

DPP (Determinantal Point Process, 行列式点过程) 的思想最早可追溯至 1975 年 Macchi 在论文《The coincidence approach to stochastic point processes》[27] 中的工作。该模型最初用于描述量子力学中费米子的统计行为。由于泡利不相容原理，费米子倾向于彼此“排斥”，从而在空间中呈现出天然的多样性分布。此后，DPP 在数学领域得到了深入发展，被广泛应用于随机矩阵特征值分布、随机生成树、不相交路径等问题的研究。

DPP 从数学与物理领域向机器学习领域的迁移始于 2011 年。Kulesza 与 Taskar 在 UAI 会议上发表了题为《Learning Determinantal Point Processes》[20] 的开创性论文，首次提出了可学习的、带参数化的 DPP 模型，使得 DPP 能够从数据中自动拟合多样性偏好。随后，他们在 2012 年于 *Foundations and Trends in Machine Learning* 上发表了综述性文章《Determinantal Point Processes for Machine Learning》[19]，系统阐述了 DPP 的直观含义、核心算法及其在机器学习中的典型应用场景。这些工作极大地推动了 DPP 在多样性推荐、文档摘要、图像检索等任务中的应用。

在机器学习任务中，DPP 常被用作一种建模元素间“排斥性” (repulsion) 的概率模型，特别适用于需要在保持相关性的同时增强结果多样性的场景。例如，在信息检索或推荐系统中，DPP 能够有效刻画被选物品之间的不相似性，从而避免冗余结果的出现。然而，从贝叶斯推断的角度来看，DPP 的最大后验概率 (MAP, Maximum A Posteriori) 推断是一个 NP-难问题。传统的精确求解方法计算复杂度通常高达 $O(N^4)$ ，严重制约了其在大规模推荐场景中的实际部署。虽然后续研究提出了复杂度为 $O(N^3)$ 的近似算法，但往往以牺牲解的质量为代价：即在降低计算开销的同时，难以保证多样性效果的稳定性与最优性。

针对这一挑战，Chen 等人在 NeurIPS 2018 发表的论文《Fast Greedy MAP Inference for Determinantal Point Process to Improve Recommendation Diversity》[4] 提出了一种时间复杂度为 $O(N^3)$ 高效且高精度的贪心 MAP 推断算法。该方法在显著降低计算复杂度的同时，有效保持了 DPP 所期望的多样性增益，从而为 DPP 在工业级推荐系统中的落地提供了切实可行的技术路径。

下面我们重点介绍 DPP 的数学原理和几何解释。DPP 是一种优雅的概率模型，能够表达负相关关系。形式上，给定离散集合 $Z = \{1, 2, \dots, N\}$ ，则 DPP $\mathcal{P}$ 是作用在 2^Z (即集合 Z 的所有子集) 上的概率测度。当 $\mathcal{P}$ 对空集赋予非零概率时，存在一个半正定矩阵 $(\mathbf{L} = \mathbf{B}^T \mathbf{B}) \in \mathbb{R}^{N \times N}$ ，使得对于每个子集 $Y \subseteq Z$ 有：

$$\mathcal{P}(Y) \propto \det(\mathbf{L}_Y) = \text{Vol}^2(\{\mathbf{B}_i\}_{i \in Y}) \quad (12.119)$$

注意这里对于矩阵 $\mathbf{L}$ 增加了半正定的限制，这种 DPP 定义实际上被称之为“L-ensemble”的 DPP [19]。

从公式 12.119 可以看出，一个集合 Y 的 DPP 概率与集合中元素组成的集合 $\{\mathbf{B}_i\}_{i \in Y}$ 中所有向量 $\mathbf{B}_i$ 张成的高维超平行体的体积平方成正比。因此，当集合 Y 对应的所有向量 $\mathbf{B}_i$ 张成的高维超平行体的体积越大，则在 DPP 中对应的概率也会越大。图 12.15 展示了二维空间和三维空间中的向量张成的几何结构，分别对应的是平行四边形和平行六面体。如果每个向量的长度不变，任意两个向量之间的夹角变小，都会导致这两个向量的相似性增大，从而导致超几何体的体积变小，DPP 的概率也会变小。

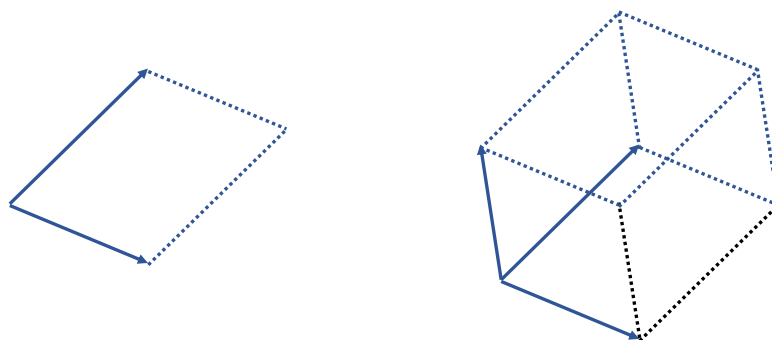


图 12.15: DPP 几何结构示意图。

公式 12.119 所示的 DPP 对应的 MAP 推断公式如下：

$$\mathcal{P}(Y)_{\text{MAP}} = \arg \max_{Y \subseteq Z} \det(\mathbf{L}_Y) \quad (12.120)$$

在推荐场景中，我们可以将矩阵 $\mathbf{B}$ 中每个物品对应的向量 $\mathbf{B}_i$ 定义为 $\mathbf{B}_i = r_i \mathbf{f}_i$ ，其中 r_i 是这个物品对应的 score，具体可以用推荐系统当前模块下物品的融合分。而 $\mathbf{f}_i$ 是物品 i 对应的一个向量，可以是重排模型或者精排模型产出的 item embedding。注意这里的向量 $\mathbf{f}_i$ 需要做归一化的处理，保证向量模长 $\|\mathbf{f}_i\|_2 = 1$ 。因此矩阵 $\mathbf{L}$ 中对应的元素可以写作：

$$\mathbf{L}_{ij} = \langle \mathbf{B}_i, \mathbf{B}_j \rangle = \langle r_i \mathbf{f}_i, r_j \mathbf{f}_j \rangle = r_i r_j \langle \mathbf{f}_i, \mathbf{f}_j \rangle \quad (12.121)$$

所以 kernel 矩阵 $\mathbf{L}$ 可以写成：

$$\mathbf{L} = \text{Diag}(\mathbf{r}) \cdot \mathbf{S} \cdot \text{Diag}(\mathbf{r}) \quad (12.122)$$

注意这里的 $\mathbf{S}$ 是一个相似度矩阵且 $\mathbf{S}_{ij} = \langle \mathbf{f}_i, \mathbf{f}_j \rangle$ ， $\text{Diag}(\mathbf{r})$ 是根据所有物品的相关性分数组成的相关性向量 $\mathbf{r}$ 所构成的对角矩阵。由于 $\langle \mathbf{f}_i, \mathbf{f}_j \rangle$ 实际上表示了 cosine 相似度，取值范围是 $[-1, 1]$ 。为了保证 $\mathbf{L}$ 是半正定矩阵，可以通过 $\mathbf{S}_{ij} = \frac{1 + \langle \mathbf{f}_i, \mathbf{f}_j \rangle}{2}$ 的处理方式将 $\mathbf{S}_{ij}$ 取值限制在 $[0, 1]$ 范围内。

根据行列式的如下两个性质：

1. $\det(AB) = \det(A) \det(B)$
2. 对角矩阵的行列式等于对角元素的乘积： $\det(\text{Diag}(\mathbf{r})) = \prod_{i=1}^N r_i$

我们可以得到矩阵 $\mathbf{L}$ 的对数行列式为：

$$\log \det \mathbf{L} = \sum_{i=1}^N \log(r_i^2) + \log \det(\mathbf{S}) \quad (12.123)$$

其中第一项 $\sum_{i=1}^N \log(r_i^2)$ 表示用户与子集 Y 中所有物品的相关性，而公式第二项 $\log \det(\mathbf{S})$ 表示整个子集 Y 中物品之间的多样性。

在推荐系统中为了更好地平衡上述相关性与多样性，可以在上式中增加 λ 超参数来进行权衡得到：

$$\log \det \mathbf{L}' = \lambda \sum_{i=1}^N \log(r_i^2) + (1 - \lambda) \log \det(\mathbf{S}) \quad (12.124)$$

其中矩阵 $\mathbf{L}' = \text{Diag}(\exp(\alpha \mathbf{r})) \cdot \mathbf{S} \cdot \text{Diag}(\exp(\alpha \mathbf{r}))$ ， $\alpha = \theta / (2(1 - \theta))$ 。

对于公式 12.123 和 12.124 的最大化求解，文献 [12] 从理论上证明了 DPP 的 MAP 推理是一种子模函数最大化 (submodular function) 问题，而子模函数最大化问题虽然是 NP-难问题，但通常有贪心的迭代算法进行近似的求解。假设集合函数 (set function) f 是定义在集合上 2^Z 上的实值函数。如果一个集合函数 f 的**边际增益 (marginal gains)** 是非递增的，即对于任意的 $i \in Z$ 和任意满足 $X \subseteq Y \subseteq Z \setminus \{i\}$ 的集合，都有如下公式成立：

$$f(X \cup \{i\}) - f(X) \geq f(Y \cup \{i\}) - f(Y) \quad (12.125)$$

而贪心求解 DPP 对应的子模函数最大化问题则是首先初始化目标集合 $Y = \emptyset$ ，然后在每次迭代中根据如下公式 12.126 更新，直到最终目标集合中物品数量达到预期个数终止。

$$j = \arg \max_{i \in Z \setminus Y} \log \det(\mathbf{L}_{Y \cup \{i\}}) - \log \det(\mathbf{L}_Y) \quad (12.126)$$

对于公式 12.126 的求解，该论文中引入了 Cholesky 分解的算法，每次迭代根据 $\mathbf{L}_Y$ 的 Cholesky 分解结果 $\mathbf{L}_Y = \mathbf{V}\mathbf{V}^T$ ，通过递推公式计算出 $\mathbf{L}_{Y \cup \{i\}}$ ，进而得到 $\log \det(\mathbf{L}_{Y \cup \{i\}}) - \log \det(\mathbf{L}_Y)$ 的结果，从而求出满足最大条件的物品编号 j 。对于任意的物品 $i \in Z \setminus Y$ ，矩阵 $\mathbf{L}_{Y \cup \{i\}}$ 的 Cholesky 分解可以通过如下递推公式进行更新：

$$\mathbf{L}_{Y \cup \{i\}} = \begin{bmatrix} \mathbf{L}_Y & \mathbf{L}_{Y,i} \\ \mathbf{L}_{i,Y} & \mathbf{L}_{ii} \end{bmatrix} = \begin{bmatrix} \mathbf{V} & \mathbf{0} \\ \mathbf{c}_i & d_i \end{bmatrix} \begin{bmatrix} \mathbf{V}^T & \mathbf{c}_i^T \\ \mathbf{0}^T & d_i \end{bmatrix} \quad (12.127)$$

其中 $\mathbf{L}_{Y,i}$ 表示的是取矩阵 $\mathbf{L}$ 中的 Y 中包含的编号对应的行以及第 i 列形成的子矩阵。 $\mathbf{L}_{i,i}$ 表示矩阵 $\mathbf{L}$ 中位置 (i, i) 的元素。

根据上面的 Cholesky 分解公式，通过矩阵乘法可得 $\mathbf{L}_{Y,i}$ 和 $\mathbf{L}_{ii}$ 与待求解的 $\mathbf{c}_i$ 和 d_i 之间的关系：

$$\begin{aligned}\mathbf{V}\mathbf{c}_i^\top &= \mathbf{L}_{Y,i} \\ d_i^2 &= \mathbf{L}_{ii} - \|\mathbf{c}_i\|_2^2\end{aligned}\quad (12.128)$$

由于公式 12.127 中 $\mathbf{L}_{Y \cup \{i\}}$ 也可以写成一个矩阵与其转置相乘的形式，因此 $\mathbf{L}_{Y \cup \{i\}}$ 的行列式等于右侧分解矩阵行列式的平方。由此可得：

$$\det(\mathbf{L}_{Y \cup \{i\}}) = \det(\mathbf{V}\mathbf{V}^\top) \cdot d_i^2 = \det(\mathbf{L}_Y) \cdot d_i^2 \quad (12.129)$$

这样贪心迭代的目标函数可以转化为：

$$\begin{aligned}j &= \arg \max_{i \in Z \setminus Y} \log \det(\mathbf{L}_{Y \cup \{i\}}) - \log \det(\mathbf{L}_Y) \\ &= \arg \max_{i \in Z \setminus Y} \log \det(\mathbf{L}_Y) + \log(d_i^2) - \log \det(\mathbf{L}_Y) \\ &= \arg \max_{i \in Z \setminus Y} \log(d_i^2)\end{aligned}\quad (12.130)$$

在一次迭代完成求出最优的 j 之后，就可以更新公式 12.127 中的矩阵 $\mathbf{L}_{Y \cup \{j\}}$ 为：

$$\mathbf{L}_{Y \cup \{j\}} = \begin{bmatrix} \mathbf{V} & \mathbf{0} \\ \mathbf{c}_j & d_j \end{bmatrix} \begin{bmatrix} \mathbf{V} & \mathbf{0} \\ \mathbf{c}_j & d_j \end{bmatrix}^\top \quad (12.131)$$

注意这里矩阵 $\mathbf{L}_{Y \cup \{j\}}$ 的迭代公式与集合 Y 密切相关，因此 $\mathbf{L}_Y$ 中每一行分别与加入集合 Y 的物品实际编号相对应，其中第一行表示的是相关性最大的那个物品对应的向量。

接着可以构造出新的 $\mathbf{c}'_i$ 满足：

$$\begin{bmatrix} \mathbf{V} & \mathbf{0} \\ \mathbf{c}_j & d_j \end{bmatrix} \mathbf{c}'_i{}^\top = \mathbf{L}_{Y \cup \{j\}, i} = \begin{bmatrix} \mathbf{L}_{Y,i} \\ \mathbf{L}_{ji} \end{bmatrix} \quad (12.132)$$

结合公式 12.128 中关于 $\mathbf{c}_i$ 的等式，即可得到向量 $\mathbf{c}$ 的迭代公式：

$$\mathbf{c}'_i = \begin{bmatrix} \mathbf{c}_i & \frac{\mathbf{L}_{ji} - \langle \mathbf{c}_j, \mathbf{c}_i \rangle}{d_j} \end{bmatrix} \doteq [\mathbf{c}_i \quad e_i] \quad (12.133)$$

此外，结合公式 12.128 可得数值 d_i 的迭代公式如下：

$$d_i'^2 = \mathbf{L}_{ii} - \|\mathbf{c}'_i\|_2^2 = \mathbf{L}_{ii} - \|\mathbf{c}_i\|_2^2 - e_i^2 = d_i^2 - e_i^2 \quad (12.134)$$

根据上述的公式推导，我们可以给出对应的 python 代码如下所示：

代码 12.8: 基于 DPP 贪心算法的多样性打散代码

```
import numpy as np
def fast_greedy_map_inference(item_embeddings, r_u, theta, K):
    """
    参数:
        item_embeddings: 物品嵌入矩阵（二维numpy数组，形状为[N, D]，N为物品数，D为嵌入维度）
        r_u: 物品与用户的相关性得分数组（一维numpy数组，长度为N）
        theta: 平衡参数（0 <= theta <= 1），控制相关性与多样性权重
        K: 需选中的物品数量（终止条件）
    返回:
        Y_g: 选中的物品索引列表（长度为K）
    """
    N = item_embeddings.shape[0] # 物品总数
    if K <= 0 or K > N:
        raise ValueError(f"K必须为正整数且不超过物品总数{N}")
```

```

# 1. 计算余弦相似度矩阵S并映射到[0,1]
# 对嵌入向量归一化 (余弦相似度 = 归一化后的点积)
norm_embeddings = item_embeddings / np.linalg.norm(item_embeddings, axis=1, keepdims=True)
# 计算余弦相似度矩阵 (点积)
cos_sim = norm_embeddings @ norm_embeddings.T # 形状 [M, M]
# 映射到 [0,1] 范围: (1 + cos_sim) / 2 (余弦相似度本身范围为 [-1,1])
S = (1 + cos_sim) / 2

# 2. 构建核矩阵L'
if theta == 1:
    alpha = 0 # 极端情况: 只考虑相关性, 不考虑多样性
else:
    alpha = theta / (2 * (1 - theta)) # 平衡参数
exp_alpha_r = np.exp(alpha * r_u) # 形状 [N]
# L' = Diag(exp(r_u)) · S · Diag(exp(r_u))
L = np.diag(exp_alpha_r) @ S @ np.diag(exp_alpha_r) # 形状 [N, N]

# 3. 核心迭代过程
# 初始化辅助参数
c = [[] for _ in range(N)] # c[i]为物品i的辅助向量 (列表存储, 动态增长)
d_squared = L.diagonal().copy() # d_i^2初始为L的对角线元素, 形状 [N]
Y_g = [] # 选中的物品集合

while len(Y_g) < K:
    # 选择当前最优物品 (未选中且log(d_i^2)最大)
    valid_mask = np.ones(N, dtype=bool)
    valid_mask[Y_g] = False # 排除已选中物品
    valid_indices = np.where(valid_mask)[0]
    if len(valid_indices) == 0:
        break # 理论上不会触发, 因K < N
    # 找到最大log(d_i^2)对应的物品
    j = valid_indices[np.argmax(np.log(d_squared[valid_indices]))]
    Y_g.append(j)

    # 若已选满K个, 提前终止
    if len(Y_g) == K:
        break

    # 增量更新所有未选中物品的c[i]和d_squared[i]
    j_idx = len(Y_g) - 1 # 当前j是第j_idx+1个选中的物品 (0-based)
    d_j = np.sqrt(d_squared[j]) # d_j = sqrt(d_j^2)

    for i in range(N):
        if i not in Y_g:
            # 计算内积<c_j, c_i> (c_j是已选物品j的辅助向量)
            dot_product = np.dot(c[j], c[i]) if (len(c[j]) > 0 and len(c[i]) > 0) else 0.0
            # 计算e_i = (L_ji - 内积) / d_j
            e_i = (L[j, i] - dot_product) / d_j
            # 更新c[i]: 追加e_i

```

```

c[i].append(e_i)
# 更新  $d_i^2$ :  $d_i^2 -= e_i^2$ 

d_squared[i] -= e_i ** 2

return Y_g

```

由于 DPP 贪心算法使用了向量和数值迭代的公式来更新 $\mathbf{c}_i$ 和 d_i ，在第 k 次迭代时，每个物品的向量 $\mathbf{c}_i$ 记录了该物品与前 k 个已选物品的相关性，因此 $\mathbf{c}_i$ 的长度为 k ，更新该向量的计算复杂度为 $O(k)$ 。由于未选择的物品有 $N - k$ 个，一次迭代的计算复杂度为 $O(k(N - k))$ 。假设算法总共迭代 K 次，则总的计算复杂度为 $\sum_{k=1}^K O(k(N - k)) = O(K^2 N)$ 。因此，如果没有任何约束而选择全部 N 个物品，则总计算复杂度为 $O(N^3)$ ；而如果仅选取 $K \ll N$ 个物品，则计算复杂度可显著降低至 $O(K^2 N)$ ，这使其在实际推荐系统中具有良好的可扩展性。

12.5.3.3 SSD 算法

SSD (Sliding Spectrum Decomposition, 滑动频谱分解) 算法源自小红书团队在 KDD 2021 会议上发表的论文《Sliding Spectrum Decomposition for Diversified Recommendation》。该算法的提出源于对信息流推荐场景中用户行为特性的深入观察：在用户持续下滑浏览内容的过程中，其内容消费深度往往较深，且对整个浏览序列的多样性具有**时序连贯性感知**，即当前内容的多样性体验不仅受当前物品影响，也受到历史已浏览内容的累积影响。传统基于滑动窗口的多样性方法（如窗口内 DPP）通常仅考虑局部窗口内的物品，而将窗口外的历史信息直接丢弃，这不仅容易陷入局部最优，也难以捕捉长序列上下文对整体多样性感知的贡献。

为克服这一局限，SSD 算法将用户完整的长度为 T 的浏览序列全部纳入建模范围。假设滑动窗口长度为 W ，则整个序列可划分为 $L = TW + 1$ 个连续滑动窗口。若每个物品的 Embedding 向量维度为 d ，则可将所有窗口的物品 Embedding 矩阵沿窗口维度堆叠，构成一个三维张量 $\mathcal{X} \in \mathbb{R}^{L \times W \times d}$ ，如下图 12.16 所示。

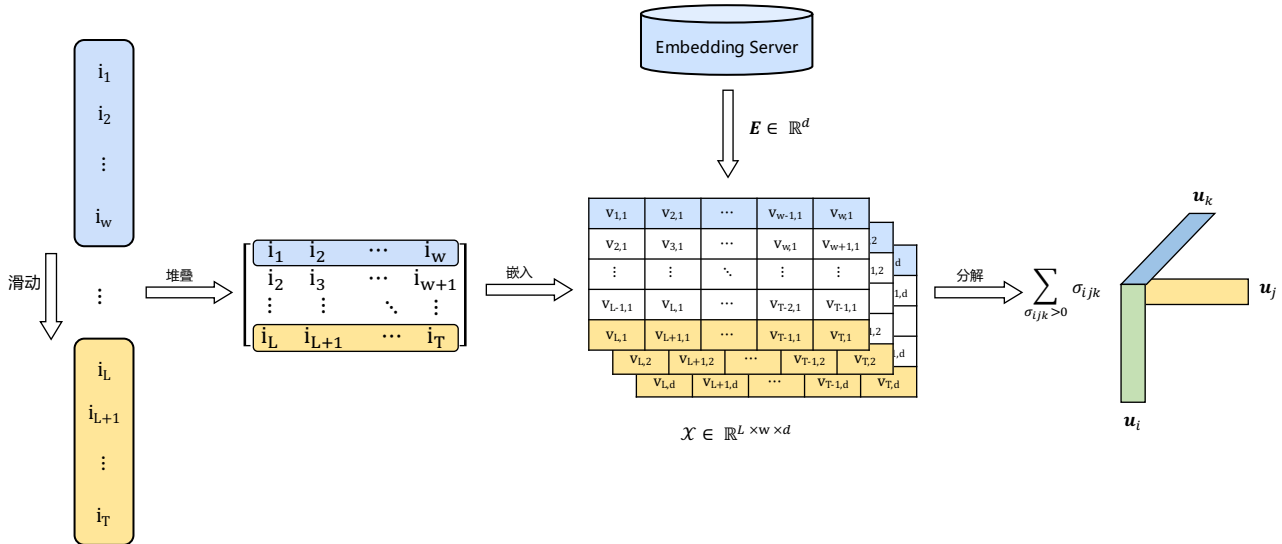


图 12.16: SSD 打散示意图。

受传统信号处理中奇异谱分析 (Singular Spectrum Analysis, SSA) 的启发，SSD 算法采用高阶奇异值分解 (Higher-Order Singular Value Decomposition, Higher-Order SVD) 对张量 $\mathcal{X}$ 进行分解，其形式可表示为：

$$\mathcal{X} = \sum_{\sigma_{ijk} > 0} \sigma_{ijk} \mathbf{u}_i^{(1)} \otimes \mathbf{u}_j^{(2)} \otimes \mathbf{u}_k^{(3)} \quad (12.135)$$

其中 σ_{ijk} 表示非零奇异值, $\mathbf{u}_i^{(1)} \in \mathbb{R}^L$ 、 $\mathbf{u}_j^{(2)} \in \mathbb{R}^W$ 、 $\mathbf{u}_k^{(3)} \in \mathbb{R}^d$ 分别为沿序列、窗口和特征三个维度的正交基向量, $\otimes$ 表示向量外积。

与 DPP 中“体积”反映多样性的思想类似, SSD 算法将张量 $\mathcal{X}$ 的“序列体积”定义为其所有非零奇异值的乘积(或加权和), 以此度量整个滑动窗口序列中物品间的全局多样性。在实际优化中, 同样需要在相关性与多样性之间进行权衡, 目标函数形式如下:

$$\max_{\{i_1, \dots, i_T\} \subset Z} \sum_{t=1}^T r_{i_t} + \gamma \sum_{\sigma_{ijk} > 0} \sigma_{ijk} \quad (12.136)$$

其中 r_{i_t} 为物品 i_t 的相关性得分, γ 为控制多样性强度的超参数。

在求解该优化问题时, SSD 依然采用贪心策略, 但其核心创新在于引入了一种计算高效且数值稳定的正交化机制: **一步改进型格拉姆-施密特正交化** (one-step Modified Gram-Schmidt, one-step MGS)。为清晰理解这一机制的动机与设计, 有必要首先回顾传统 Gram-Schmidt (GS) 正交化及其改进版本 MGS 的完整流程。

传统 Gram-Schmidt (GS) 正交化用于将一组线性无关的向量转换为正交向量组, 其核心步骤如下:

1. 给定待正交化的向量集合 $\{\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k\}$ 。
2. 初始化第一个正交向量 $\mathbf{u}_1 = \mathbf{v}_1$, 并归一化得到单位向量 $\mathbf{e}_1 = \mathbf{u}_1 / \|\mathbf{u}_1\|$ 。
3. 对第 m 个向量 $\mathbf{v}_m$ (其中 $m \geq 2$), 扣除其在所有已构造的正交单位向量 $\{\mathbf{e}_1, \dots, \mathbf{e}_{m-1}\}$ 上的投影:

$$\mathbf{u}_m = \mathbf{v}_m - \sum_{p=1}^{m-1} (\mathbf{v}_m^\top \mathbf{e}_p) \mathbf{e}_p \quad (12.137)$$

4. 将 $\mathbf{u}_m$ 归一化得到 $\mathbf{e}_m = \mathbf{u}_m / \|\mathbf{u}_m\|$, 重复步骤 3 和 4, 直至所有向量完成正交化。

尽管 GS 在理论上能生成正交基, 但在浮点运算中, 由于舍入误差的累积, 实际得到的向量往往仅近似正交, 尤其在向量数量较多或条件数较差时, 正交性会显著退化。

为提升数值稳定性, **改进型 Gram-Schmidt** (Modified Gram-Schmidt, MGS) 对算法流程进行了关键调整: 不再一次性扣除所有历史投影, 而是采用**逐步正交化**策略。其具体步骤如下:

1. 给定待正交化的向量集合 $Z = \{\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k\}$ 。
2. 初始化迭代步数 $k = 1$, 令第一个正交向量 $\mathbf{u}_1 = \mathbf{v}_1$, 归一化得 $\mathbf{e}_1 = \mathbf{u}_1 / \|\mathbf{u}_1\|$ 。
3. 对于第 $k \geq 2$ 次迭代, 对所有尚未正交化的向量 (即 $\mathbf{v}_k, \mathbf{v}_{k+1}, \dots, \mathbf{v}_n$), 依次扣除其在当前新生成的正交向量 $\mathbf{e}_{k-1}$ 上的投影:

$$\mathbf{v}_j \leftarrow \mathbf{v}_j - (\mathbf{v}_j^\top \mathbf{e}_{k-1}) \mathbf{e}_{k-1}, \quad \forall j \geq k \quad (12.138)$$

这一操作在每一步后立即更新剩余向量, 从而将误差限制在局部, 避免向后传播。

4. 将更新后的 $\mathbf{v}_k$ 作为 $\mathbf{u}_k$, 归一化得 $\mathbf{e}_k = \mathbf{u}_k / \|\mathbf{u}_k\|$, 重复步骤 3 和 4, 直至所有向量处理完毕。

MGS 的优势在于: 每次正交化操作仅依赖当前已构造的正交向量, 并立即作用于后续向量, 因此舍入误差不会在后续步骤中被放大, 数值稳定性显著优于传统 GS。此外, MGS 在处理线性相关向量时, 部分向量在正交化过程中模长会逐渐趋于零, 此时可通过设定阈值直接丢弃这些冗余向量, 避免无效计算。

SSD 算法在此基础上进一步简化, 提出“**一步改进型 MGS**” (one-step MGS) 策略, 以适配推荐系统中贪心选择的时序特性。具体而言, 在每次迭代选择新物品时, SSD 并不对候选向量执行完整的 MGS 正交化, 而是仅将其 Embedding 向量对**最新加入序列的物品向量**进行一次投影扣除。假设当前已选物品集合为 $\{i_1, \dots, i_{t-1}\}$, 则对任一未选候选物品 j , 其 Embedding 向量 $\mathbf{v}_j$ 被更新为:

$$\mathbf{v}_j \leftarrow \mathbf{v}_j - \frac{\langle \mathbf{v}_j, \mathbf{v}_{i_{t-1}} \rangle}{\|\mathbf{v}_{i_{t-1}}\|^2} \mathbf{v}_{i_{t-1}} \quad (12.139)$$

即仅扣除其在最新选中物品向量上的分量。这种“增量式”近似虽未实现全局正交, 但能有效削弱与最近内容的相似性, 契合用户对**局部时序多样性**的感知, 同时大幅降低计算开销。

基于上述思想, SSD 算法的 Python 实现如下所示:

代码 12.9: 基于 SSD 算法的多样性打散代码

```

import numpy as np

def ssd_without_sliding_window(r, embeddings, T, gamma):
    """
    参数:
        r: 物品的相关性得分列表, 长度为N
        embeddings: 物品的嵌入向量矩阵, 形状为[N, d] (N为物品数, d为嵌入维度)
        T: 推荐序列的长度
        gamma: 相关性与多样性的权衡系数

    返回:
        选中的物品序列索引列表, 长度为T
    """
    N, d = embeddings.shape
    selected = [] # 存储选中的物品索引
    # 初始化: 选择第一个相关性得分最高的物品
    i_t = np.argmax(r)
    selected.append(i_t)
    # 初始化体积V (Volume)
    V = np.linalg.norm(embeddings[i_t])

    for t in range(1, T): # 循环选择剩余T-1个物品
        # 对所有未选中的物品, 执行一步MGS正交化
        for j in range(N):
            if j not in selected:
                # 计算内积: v_j与已选物品i_t的嵌入向量的内积
                dot_product = np.dot(embeddings[j], embeddings[i_t])
                # 计算分母: 已选物品i_t的嵌入向量的自内积 (范数平方)
                denominator = np.dot(embeddings[i_t], embeddings[i_t])
                # 执行一步MGS: v_j = v_j - (v_j · v_{i_t} / v_{i_t} · v_{i_t}) * v_{i_t}
                embeddings[j] = embeddings[j] - (dot_product / denominator) * embeddings[i_t]

        # 选择下一个最优物品: 最大化 r_j + ||v_j|| * V
        candidates = [j for j in range(N) if j not in selected]
        scores = [r[j] + np.linalg.norm(embeddings[j]) * gamma * V for j in candidates]
        i_t = candidates[np.argmax(scores)]
        selected.append(i_t)
        # 更新体积V
        V = np.linalg.norm(embeddings[i_t]) * V
    return selected

```

值得注意的是, one-step MGS 的关键优势在于其增量计算特性: 无需显式维护完整的正交基, 也无需存储所有历史 Embedding, 仅需在每次迭代中对候选向量执行一次投影操作。该操作的时间复杂度为 $O(d)$ (d 为 Embedding 维度), 远低于传统 MGS 的 $O(td)$ (t 为已选物品数)。因此, SSD 贪心推理的整体时间复杂度为 $O(NTd)$, 其中 N 为候选物品总数, T 为最终序列长度。在典型推荐场景中, T 通常仅为 20–50, 而 N 可达数万量级, 此时 SSD 算法的复杂度显著优于传统 DPP 贪心算法的 $O(N^2d)$, 能够满足工业级实时推荐系统对低延迟与高吞吐的严苛要求, 同时在多样性效果上保持竞争力。

12.6 混排模块

混排（Blended Ranking）模块主要负责多个不同业务推荐结果的最终融合。例如在短视频类 App 中，用户通过上下滑不断浏览内容时，看到的往往并不只是短视频本身，还可能包括直播、广告、图文、商品卡片等多种内容形态。对于一次推荐请求而言，系统需要将这些来自不同业务线的推荐结果进行统一组织与排序，从而生成最终展示给用户的推荐列表。这类来自不同业务、不同内容形态的推荐结果，通常被称为异质化内容（Heterogeneous Content）。从推荐系统架构的角度来看，正如第 11 章图 11.3 所示，不同业务对应的推荐系统会分别完成召回、排序等流程，并输出各自的候选列表。推荐 Router 在接收到这些业务列表之后，会将其发送至混排服务，由混排模块统一完成跨业务内容的融合与排序，最终生成用户看到的结果序列。

读到这里，读者可能会产生一个问题：

 **笔记** 混排模块是否是所有推荐系统都必须具备的模块？

答案是否定的。如果一个推荐系统仅服务于单一内容形态，例如纯图文推荐系统、纯短视频推荐系统或者纯商品推荐系统，那么整个推荐链路只需要完成召回、排序和重排即可，并不一定需要单独设计混排模块。混排模块产生的根本原因在于业务形态的多样化。当平台逐渐从单一内容消费场景发展为综合内容生态，希望用户在一个浏览场景下既能够看到短视频，也能够看到直播、图文、广告以及电商内容时，便需要引入混排模块来解决不同业务之间的流量分配、内容融合以及排序优化问题。

因此，从本质上来看，混排模块并不是为了解决单个推荐系统内部的排序问题，而是用于解决多个推荐系统之间的协同推荐问题，其核心目标是在满足各业务流量诉求的同时，为用户提供更加丰富、多样且体验一致的内容消费序列。

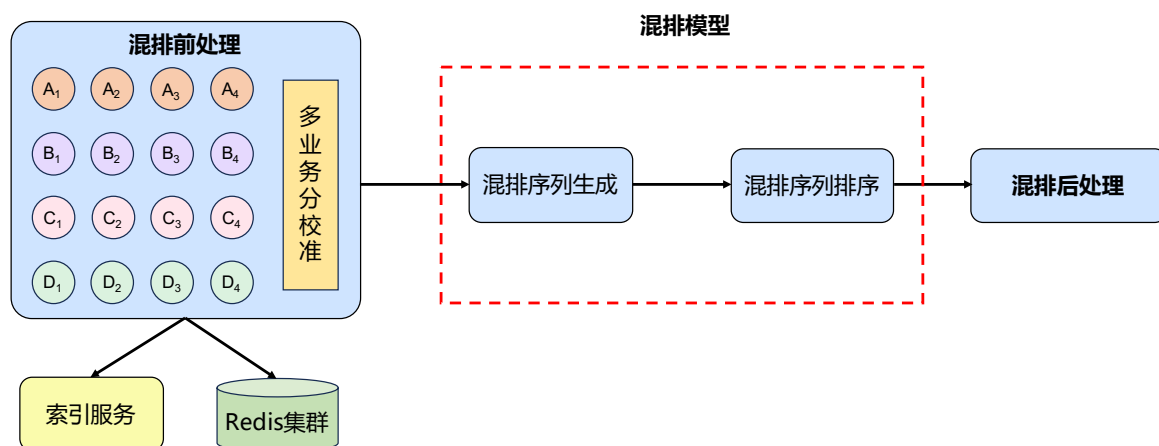


图 12.17: 混排模块内部处理流程。

混排模块的整体处理流程如图 12.17 所示，主要包括混排前处理、混排序列生成与剪枝、混排多业务排序以及混排后处理四个核心环节。其中，混排前处理环节主要负责接收推荐 Router 透传的多个业务推荐列表。这里以 $\{A_1, A_2, A_3, A_4\}, \dots, \{D_1, D_2, D_3, D_4\}$ 分别表示来自四个不同业务的排序结果。由于混排模块需要对多个推荐系统输出的结果进行统一排序，因此首先需要解决不同业务之间打分体系不一致的问题。例如短视频推荐系统输出的可能是用户时长收益分，直播推荐系统输出的可能是观看价值分，而广告系统输出的则可能是 CPM 或 ECPM 竞价分。这些分数往往来源不同、量纲不同、数值范围也存在较大差异，因此无法直接进行比较。

因此，在混排前处理阶段，通常需要对各业务的排序分进行校准（Calibration）与归一化（Normalization）处理，将不同业务的打分映射到统一的评分空间，例如归一化到 $[0, 1]$ 区间。只有完成分数对齐之后，不同业务的内容才能在同一个排序框架下进行公平竞争。否则，某些天然分值较高的业务（例如广告竞价系统）可能会在后续排序过程中获得过高权重，从而导致推荐结果过度偏向某一类内容，最终影响整体用户体验。

完成分数校准之后，系统进入混排序列生成与剪枝阶段。由于多个业务的候选内容可以组合形成大量不同的展示序列，因此通常需要设计高效的序列生成策略，在满足业务约束的前提下生成有限数量的候选序列，并通过启发式规则或剪枝策略减少搜索空间，从而降低后续混排序列排序阶段的计算开销。

在此基础上，混排模块会进入多业务排序阶段。该阶段与重排模块中的序列生成（List Generator）和序列评估（List Evaluator）框架较为类似，系统可以利用序列评估模型对整个推荐序列进行价值建模，从用户体验、业务收益以及内容生态等多个维度评估不同序列的优劣，并最终选择综合收益最优的混排结果。

最后，在完成多业务排序之后，还需要进入混排后处理阶段。该阶段主要负责执行各种业务约束和运营策略。例如运营活动的强插坑位、广告与直播内容的曝光比例控制、不同业务流量上下限约束，以及一些特殊内容的保量或降权策略等。通过后处理机制，可以保证最终生成的推荐列表不仅满足模型优化目标，同时也能够满足平台运营、商业化以及内容生态建设等多方面需求。

总体来看，混排模块的本质目标并不仅仅是将多个业务的内容简单拼接在一起，而是在统一的排序框架下，实现不同业务之间的分数对齐、流量分配和价值权衡，从而生成兼顾用户体验与业务收益的最终推荐序列。

12.6.1 混排前处理

在混排前处理环节，系统会接收来自多个不同业务的候选排序列表，例如广告序列、直播序列、短视频序列以及图文序列等。由于这些业务通常拥有各自独立的推荐系统，因此其排序依据往往也是各自业务内部定义的价值分。例如广告业务可能采用 CPM（Cost Per Mille）或 ECPM 作为排序依据，直播业务可能采用打赏价值、观看时长价值等指标，而短视频业务则可能采用用户满意度或时长收益相关指标进行排序。

然而，不同业务输出的价值分通常来源于各自推荐模型的 pxtr 预估结果，而模型预估值天然会存在一定程度的高估（Over-estimation）或低估（Under-estimation）问题。因此，在混排之前首先需要对各业务的价值分进行校准（Calibration）处理，使其尽可能接近真实价值。以广告业务为例，工业界通常会重点关注模型预估值与真实值之间的偏差情况，一个常见指标是 PCOC（Predicted Click Over Click）。理想情况下：

$$PCOC \approx 1 \quad (12.140)$$

这意味着模型预估结果与真实反馈基本一致，不存在明显的系统性高估或低估。如果某个业务的价值分长期存在偏差，那么其在后续混排过程中所获得的流量分配也会受到影响。

从某种角度来看，混排排序本质上可以理解多个业务在竞争有限流量坑位的过程。每个业务都会依据自身的价值分参与排序，而混排系统则负责决定最终哪些内容能够获得更靠前的曝光位置。如果不同业务的价值分未经校准直接参与竞争，那么整个排序过程就相当于在使用不同计量单位进行竞价，其结果往往是不公平且不稳定的。例如，当广告系统的价值分被系统性高估时，广告内容可能会在混排结果中获得远超预期的曝光量，从而挤占直播、短视频等其他业务的流量；反之，如果广告价值被低估，则可能导致广告曝光不足，直接影响平台商业化收入以及广告主投放效果。同样地，直播、图文等业务如果存在严重的分数偏差，也可能导致流量异常波动，影响整体生态平衡。

更严重的情况下，业务价值分未经有效校准还可能引发系统层面的流量分配故障。例如某个业务模型上线后出现预估值整体偏高的问题，混排系统可能会在短时间内向该业务倾斜大量流量，导致其他业务流量骤降；而当模型恢复正常后，流量又会快速回流，从而造成推荐生态的不稳定。因此，业务价值分校准不仅是混排排序效果优化的重要前提，也是保证多业务流量分配稳定性和系统安全性的关键环节。

总体而言，混排前处理阶段最核心的任务之一，就是将来自不同业务、不同模型、不同价值体系的排序分数映射到统一且可信的价值空间中，为后续的跨业务序列生成与排序奠定基础。

12.6.1.1 业务分校准方法

前面介绍了为什么混排前处理阶段必须进行多业务分校准，以及缺失校准环节可能给推荐系统带来的流量错配、业务失衡甚至系统风险等问题。接下来，我们重点介绍工业界最常见的几类分数校准方法。其中，应用最广泛、也是推荐系统领域最经典的校准方法之一便是保序回归（Isotonic Regression, IR）[10] 及其改进版本 SIR（Smoothed Isotonic Regression）[8]。

保序回归的核心思想非常直观，即：

定义 12.1 (保序回归核心思想)

如果模型预测分数越高，那么对应样本的真实后验概率 (Posterior Probability) 也应该越高。换句话说，校准函数应当满足单调递增约束：

$$f(p_i) \leq f(p_j), \quad \forall p_i \leq p_j \quad (12.141)$$

，其中 p_i 表示模型输出的原始 pxtr 分数， $f(\cdot)$ 表示校准函数。



在实际工程中，通常首先利用离线日志数据对模型输出的 pxtr 进行排序，然后按照分数区间进行分桶。例如划分为 N 个 Bucket：

$$B_1, B_2, \dots, B_N \quad (12.142)$$

。对于第 k 个桶，其真实后验概率可以表示为：

$$r_k = \frac{\sum_{i \in B_k} y_i}{|B_k|} \quad (12.143)$$

，其中 $y_i \in \{0, 1\}$ 表示真实标签。

理想情况下，不同桶的真实后验概率 r_i 应该满足：

$$r_1 \leq r_2 \leq \dots \leq r_N \quad (12.144)$$

，但由于数据噪声以及采样波动的存在，经常会出现 $r_k > r_{k+1}$ 这种违反单调性的情况。此时，保序回归会利用经典的 **PAVA (Pool Adjacent Violators Algorithm)** 算法 [7] 进行修正。当相邻两个桶违反单调性约束时，将两个桶进行合并：

$$B_k \leftarrow B_k \cup B_{k+1} \quad (12.145)$$

，新的后验概率为：

$$r_{new} = \frac{|B_k|r_k + |B_{k+1}|r_{k+1}}{|B_k| + |B_{k+1}|} \quad (12.146)$$

。随后继续检查新的桶序列是否满足单调约束，直到所有桶均满足：

$$r_1 \leq r_2 \leq \dots \leq r_M \quad (12.147)$$

，最终即可得到一个单调递增的校准映射表：

$$p \rightarrow f(p) \quad (12.148)$$

。在线推理时，根据模型输出的 pxtr 查询对应区间即可获得校准后的预估值。

虽然 IR 能够有效解决模型预估值高估或低估的问题，但它本质上是一种分段常数 (Piecewise Constant) 函数。也就是说，同一个桶中的所有样本都会被映射到相同的校准值：

$$f(p) = c_k \quad (12.149)$$

因此在桶边界附近容易出现明显的跳变现象。例如两个原始分数非常接近的样本，仅仅因为落在不同桶中，就可能获得差异较大的校准结果。

为了解决这一问题，阿里妈妈团队在 2018 年提出了 SIR (Smoothed Isotonic Regression) 方法。SIR 的核心思想是：



笔记 在保序回归得到的单调映射函数基础上进一步进行平滑处理 (Smoothing)，从而获得连续且更加稳定的校准函数。

假设保序回归得到相邻两个校准点 (p_k, r_k) 与 (p_{k+1}, r_{k+1}) ，则对于区间内部任意一个预测值：

$$p_k \leq p \leq p_{k+1} \quad (12.150)$$

，SIR 会采用线性插值构造平滑校准函数：

$$f(p) = r_k + \frac{p - p_k}{p_{k+1} - p_k} (r_{k+1} - r_k) \quad (12.151)$$

这样得到的校准函数不再是阶梯函数，而是连续的分段线性函数。

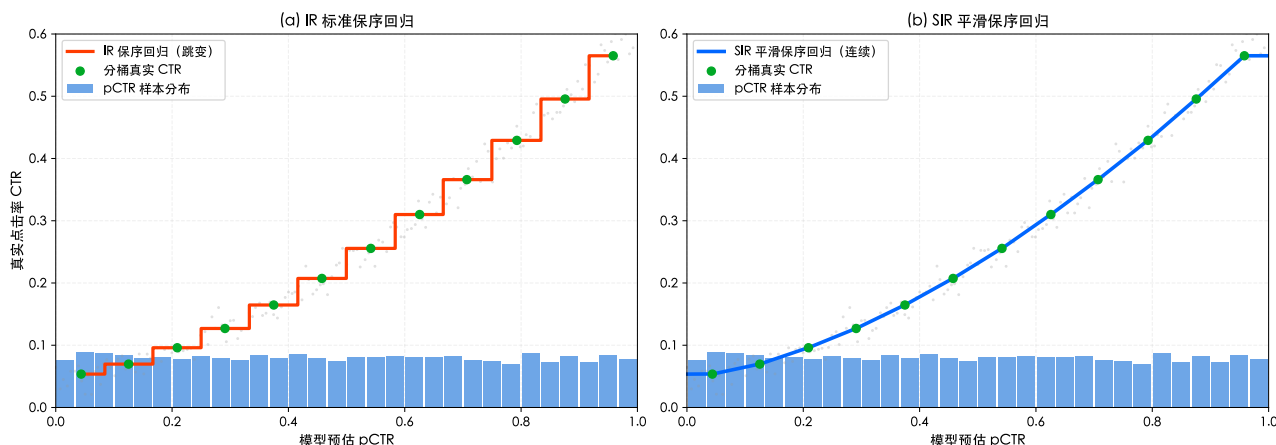


图 12.18: 保序回归于带平滑的保序回归方法对比。

SIR 与 IR 的校准效果对比如图 12.18 所示。从图中可以看出，真实 CTR 本质上是基于历史数据统计得到的离散后验值，因此在不同预测分桶上通常表现为一系列离散采样点。在图 12.18(a) 中，传统 Isotonic Regression (IR) 方法通过分段常数函数对后验 CTR 进行拟合。虽然能够保证校准结果满足单调性约束，但由于每个区间内采用相同的校准值，因此在相邻区间之间往往会产生明显的阶梯状跳变。这种不连续的映射关系在实际线上系统中容易导致排序分数出现局部突变，从而影响排序稳定性。相比之下，图 12.18(b) 展示的 Smoothed Isotonic Regression (SIR) 方法在 IR 的基础上进一步引入了线性插值平滑处理。通过对相邻分桶之间的校准结果进行连续化建模，SIR 能够得到更加平滑的校准曲线，使后验 CTR 随预测 CTR 的变化呈现连续增长趋势。这样不仅保留了保序回归的单调性约束，还显著提升了校准函数的平滑性和泛化能力。

因此，在工业界推荐系统和广告系统中，SIR 通常比传统 IR 更为常用。一方面能够有效缓解分桶统计噪声带来的局部震荡问题，另一方面也能够避免校准函数出现过大的阶梯跳变，从而提升后续排序与竞价环节的稳定性。总结一下，相比传统 IR 算法，SIR 算法具有以下几个优点：

- 保留了保序回归天然的单调性约束；
- 消除了桶边界处的分数跳变问题；
- 校准曲线更加平滑，线上表现更加稳定；
- 对于广告 CTR、CVR 等需要高精度校准的场景效果更好。

从工业实践的角度来看，IR 更像是一种非参数化 (Non-parametric) 的概率校准方法，而 SIR 则是在 IR 基础上的工程增强版本。目前在广告、搜索、推荐以及混排等场景中，SIR 仍然是使用最广泛的分数校准方案之一。对于混排模块而言，各业务模型首先通过 IR 或 SIR 完成业务价值分校准，然后再进行归一化与跨业务排序，从而保证不同业务能够在统一且可信的价值空间内参与后续流量竞争。在实际工程落地中，SIR 往往以离线天级任务的形式运行，通过 Spark 等分布式计算框架对海量样本进行统计与拟合；工程师通常会在校准逻辑封装为 Python 脚本或 UDF，并嵌入 Hive SQL 或 Spark Pipeline 中定期执行，最终生成校准映射表供在线服务加载使用。

除此之外，在多个业务分校准的过程中，经过保序回归 (Isotonic Regression, IR) 对 pXTR 进行分桶之后，由于真实后验反馈数据通常较为稀疏，某些 pXTR 对应分桶内的样本量可能非常有限，从而导致后验 CTR、CVR 等统计指标出现较大的随机波动。这种由小样本统计带来的高方差 (High Variance) 问题，是工业界推荐系统、

广告系统以及搜索系统中普遍存在的现象。如果直接使用这些高噪声统计结果作为校准值，往往会导致校准映射表不稳定，进而影响后续混排与流量分配效果。因此，在实际工程中，除了利用 IR 保证校准曲线的单调性之外，通常还会进一步引入贝叶斯平滑（Bayesian Smoothing）等方差缩减（Variance Reduction）方法，以降低小样本分桶带来的统计噪声。

表 12.6: 为什么要做贝叶斯平滑的 pCTR 校准示例。

pCTR 分桶	PV	Click	CTR
[0.1, 0.2]	100000	12000	0.12
[0.2, 0.3]	80000	18000	0.225
[0.3, 0.4]	50	20	0.40

以表 12.6 为例，pCTR 取值在 [0.3, 0.4] 区间对应的分桶仅包含 50 次曝光和 20 次点击行为，其经验 CTR 为 $CTR = \frac{20}{50} = 0.4$ ，即真实后验 CTR 高达 40%。然而由于该分桶样本量极小，这一结果很可能只是随机波动导致的统计偏差。如果直接将其作为校准结果，则会使该区间的校准值明显偏离真实水平。因此需要利用历史统计信息对当前观测结果进行约束，使其向更加可信的全局均值回归，从而获得更加稳定的后验估计。

贝叶斯平滑的核心思想是：

 **笔记** 当前观测数据与历史先验知识共同决定最终 CTR 的取值。当观测样本较多时，模型更相信当前数据；当观测样本较少时，则更多依赖历史先验信息。

由于 CTR 本质上表示一次曝光产生点击的概率，其取值范围位于 [0, 1] 区间，因此通常使用 Beta 分布作为 CTR 的先验分布：

$$CTR \sim Beta(\alpha, \beta) \quad (12.152)$$

，其中 α 和 β 分别表示 Beta 分布的两个参数，它们可以通过历史大盘数据、业务分桶统计数据或者矩估计（Method of Moments）、极大似然估计（MLE）等方式获得。对于给定分桶中的曝光与点击数据，假设点击次数服从二项分布：

$$Click \sim Binomial(PV, CTR) \quad (12.153)$$

，即 $P(Click|CTR)$ 表示在 CTR 已知的情况下，观察到对应点击次数的概率。

由于 Beta 分布与 Binomial 分布互为共轭先验（Conjugate Prior），根据贝叶斯定理可知，观测数据更新后的后验分布仍然服从 Beta 分布：

$$CTR|Data \sim Beta(\alpha + Click, \beta + PV - Click) \quad (12.154)$$

因此，最终后验 CTR 的期望值可以写为：

$$E[CTR] = \frac{Click + \alpha}{PV + \alpha + \beta} \quad (12.155)$$

这也是工业界贝叶斯平滑最常使用的计算公式。

从该公式可以看出，当分桶样本量较大时， $PV \gg \alpha + \beta$ ，后验 CTR 会逐渐逼近经验 CTR：

$$CTR \approx \frac{Click}{PV} \quad (12.156)$$

此时模型更加相信观测数据；而当分桶样本量较小时，先验参数 α, β 对结果影响更大，后验 CTR 会向历史均值回归，从而有效抑制小样本带来的统计波动。

总结起来，贝叶斯平滑本质上是一种利用先验知识约束后验估计的方差缩减（Variance Reduction）技术。它与保序回归并不冲突，而是对 IR 校准结果的重要补充：IR 负责保证校准曲线的单调性，贝叶斯平滑负责降低小样本分桶带来的统计噪声，而线性插值则用于消除分桶边界带来的跳变，使校准曲线更加平滑和连续。这三种技术有机结合后，便构成了工业界广泛使用的 Bayesian-SIR 校准方案。此外，贝叶斯平滑的思想并不仅限于分数校准场景。在推荐系统中，粗排、精排模型的 Label 纠偏、CTR/CVR 统计估计以及各类低频特征的平滑处理中，都可以利用贝叶斯平滑降低噪声干扰，从而获得更加稳定和可靠的统计结果。因此，贝叶斯平滑也被认为

是推荐系统领域一个非常经典且实用的工程技巧（Engineering Trick）。

12.6.1.2 业务分归一化方法

在混排系统中，各业务模型的业务分经过 SIR 等校准方法完成校准之后，通常还需要进一步进行归一化（Normalization）处理。归一化的主要目的是将不同业务的竞价分（模型预估值或由多个预估指标组合得到的 Score）映射到统一的量纲和数值范围下，从而使不同业务能够在相对公平的条件参与后续流量竞争。

造成归一化需求的主要原因在于：**不同业务的目标函数、训练样本以及模型结构往往存在较大差异**。例如广告业务的 eCPM 分数可能分布在数百到数千之间，而推荐业务的 pCTR 或 pVTR 可能仅分布在 [0, 1] 区间内。即使经过校准，不同业务分的数值分布仍然可能存在明显差异，因此需要进一步进行归一化处理。

工业界常见的归一化方法主要包括 Min-Max 归一化、Log Min-Max 归一化、Z-Score 归一化以及基于标准高斯分布的归一化等。Min-Max 归一化将数据线性映射到固定区间（通常为 [0, 1]）：

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}} \quad (12.157)$$

，其中 x_{min} 和 x_{max} 分别表示业务分的最小值和最大值。该方法实现简单，计算开销低，因此在工程实践中应用较为广泛。

当业务分分布具有明显长尾特征时，通常会先进行对数变换，再执行 Min-Max 归一化：

$$x' = \frac{\log(x) - \log(x_{min})}{\log(x_{max}) - \log(x_{min})} \quad (12.158)$$

这种方式能够有效压缩极端大值带来的影响，提高归一化结果的稳定性。

Z-Score 归一化则利用均值和标准差对数据进行标准化处理：

$$x' = \frac{x - \mu}{\sigma} \quad (12.159)$$

，其中 μ 表示均值， σ 表示标准差。经过归一化后，数据均值为 0，标准差为 1。相比 Min-Max 方法，Z-Score 对数据整体分布刻画更加充分，在业务分动态变化较大的场景中较为常见。

对于分布近似服从高斯分布的业务分，还可以进一步利用标准正态分布的累积分布函数（CDF）完成概率空间映射：

$$x' = \Phi\left(\frac{x - \mu}{\sigma}\right) \quad (12.160)$$

，其中 $\Phi(\cdot)$ 表示标准高斯分布的累积分布函数。经过变换后，业务分被映射到 [0, 1] 区间，同时能够较好地保留原始排序关系，因此在部分大规模混排系统中也有广泛应用。

在工业界实际应用中，归一化技术并不能替代校准。校准解决的是业务分与真实价值之间的一致性问题，而归一化解决的是不同业务之间量纲和分布不一致的问题。在工业界混排系统中，通常先进行 SIR 等校准处理，再进行归一化，最后完成跨业务排序与流量分配。

此外，还需要注意的是，归一化虽然能够统一不同业务竞价分的量纲和取值范围，但也可能改变原始业务分的分布特征。如果后续多业务融合采用线性加权等方式进行排序，那么“归一化 + 线性融合”的组合往往会使最终排序结果对竞价分的数值变化非常敏感。这种敏感性通常会给模型迭代带来额外风险。例如，当模型出现预估偏高或预估偏低的问题时，如果缺少相应的校准环节，归一化过程可能进一步放大这种偏差，从而导致线上排序效果下降。因此，无论是在粗排、精排阶段的多目标融合，还是在混排阶段的跨业务排序，归一化操作都有可能对竞价分的真实价值表达产生影响。

正因如此，在实际工程中，上游模块输出的竞价分并不一定直接采用模型原始预估值，而往往会结合业务需求进行一定的分布塑形（Score Shaping）。常见做法包括对预估分进行非线性变换、分段映射，甚至将 Top-K 内容的分数统一置为 1 等方式，从而减弱归一化带来的影响，保证业务希望重点透出的内容能够获得更稳定的曝光机会。

12.6.2 序列生成

混排阶段的序列生成与重排阶段类似，本质上都是从候选物料集合中搜索可能的排序序列。然而，两者最大的区别在于候选规模。重排阶段通常仍然需要面对数十甚至上百个候选 Item，因此往往需要借助 Beam Search 等启发式搜索方法；而在混排阶段，各业务候选数量已经非常有限，此时完全可以直接枚举所有合法序列。

例如，假设当前短视频候选固定为 6 个，广告候选最多为 3 个，直播候选最多为 1 个。此时混排问题实际上转化为：在已经确定的视频序列中，枚举广告和直播可能插入的位置。当广告数量为 3 个时，仅存在一种广告组合。若存在直播，则最终序列长度为 10，需要从 10 个位置中选择 4 个位置放置广告和直播，对应的候选序列数为： $C(10, 4) = 210$ ；若不存在直播，则序列长度为 9，对应候选序列数为： $C(9, 3) = 84$ ，因此该场景总共可以生成： $210 + 84 = 294$ 个候选序列。类似地，当广告数量分别为 2、1、0 时，对应候选序列数分别为： $3(C(9, 3) + C(8, 2)) = 336$ ， $3(C(8, 2) + C(7, 1)) = 105$ ， $C(7, 1) + C(6, 0) = 8$ 。最终所有合法候选序列数量为： $294 + 336 + 105 + 8 = 743$ 。

可以看到，对于上述例子，即使考虑广告和直播的各种插入组合，总候选序列数仍然只有数百级别。因此在许多工业界混排系统中，如果多业务的候选 item 数量不多，往往可以直接枚举所有候选序列，再利用序列评估模型进行打分和选择，而无需采用复杂的搜索算法。

读者需要从上述示例中认识到：

定义 12.2

当多业务组合产生的候选序列规模较小时，混排阶段的序列生成过程完全可以通过暴力枚举的方式实现，并不一定需要引入复杂的序列生成模型或搜索策略。因此，序列生成模型本身并不是混排系统的必要组成部分，而是解决大规模组合优化问题的一种工程手段。



然而，当业务规模逐渐扩大时，混排系统输入的候选队列会显著增多。例如，除了广告、直播之外，还可能引入图文、电商、本地生活、知识内容等业务；同时，每个业务的候选 Item 数量也会持续增长。在这种情况下，合法候选序列的数量将呈现组合数级别的增长，甚至出现组合爆炸（Combinatorial Explosion）问题。此时，一方面暴力枚举本身的时间复杂度会迅速升高；另一方面，大量候选序列还需要经过后续的序列评估模型进行打分，从而进一步增加系统的算力消耗和线上延迟。在工业级推荐系统中，这种计算开销和响应耗时往往是不可接受的。

正因如此，混排系统通常会在序列生成阶段引入各种候选序列召回策略与序列生成模型，从全量候选序列空间中快速筛选出少量高质量候选。例如，利用 Beam Search、启发式搜索（Heuristic Search）、规则剪枝（Rule-based Pruning）、强化学习（Reinforcement Learning）、序列生成模型（Sequence Generation Model）等方法，对候选空间进行有效压缩。其本质目标并不是寻找绝对最优解，而是在排序效果、计算复杂度和线上时延之间取得合理平衡。

从工程角度来看，混排序列生成阶段与传统召回模块具有一定相似性：二者都不直接追求最终最优结果，而是通过较低的计算成本从庞大的候选空间中快速筛选出一批优质候选，再交由后续更加精细的评估模型完成最终决策。因此，混排中的序列生成模型本质上也可以看作是一种面向序列空间的“召回模型”。

12.6.2.1 序列生成约束规则

在混排序列生成过程中，除了需要考虑序列评估模型的得分之外，还需要满足大量业务约束（Business Constraints）。从工程实践角度来看，混排本质上是一个带约束条件的序列优化问题（Constrained Sequence Optimization）。因此，序列生成阶段除了负责寻找高价值候选序列之外，还需要保证生成的序列满足各种业务规则 and 用户体验规则。

广告插入规则是混排系统中最常见的一类约束。例如，为了降低广告对用户体验的负面影响，广告通常不能连续出现，相邻广告之间必须间隔一定数量的非广告内容，这种约束称为刷次内 Gap 约束（Intra-Feed Gap）。

Constraint)。例如要求任意两个广告之间至少间隔 3 个自然内容：

$$ad \rightarrow item \rightarrow item \rightarrow item \rightarrow ad \quad (12.161)$$

。如果广告连续出现：

$$ad \rightarrow ad \quad (12.162)$$

，则该候选序列会被直接过滤。

除此之外，还存在**跨刷次 Gap 约束 (Inter-Feed Gap Constraint)**。例如用户上一刷的最后一个 Item 是广告，则下一刷的首位通常不允许再次出现广告，否则用户会感知到广告过于密集。此时混排系统在生成当前刷次序列时，需要同时参考历史曝光序列信息。

除了广告之外，直播业务也经常存在类似约束。例如对于直播偏好较低的人群，直播内容可能不允许出现在首刷位置；而对于直播重度用户，则允许直播出现在首刷甚至首位位置。类似地，电商、本地生活等强商业化内容也可能存在不同程度的位置约束。

在实际工业系统中，常见的混排约束还包括：

- **位置约束 (Position Constraint)**：某类内容只能出现在指定位置范围内；
- **频控约束 (Frequency Constraint)**：同一业务或同一作者在固定窗口内不能曝光过多次；
- **多样性约束 (Diversity Constraint)**：避免连续出现相同类别、相同作者或相同主题内容；
- **保量约束 (Quota Constraint)**：保证特定业务获得最低曝光量；
- **探索约束 (Exploration Constraint)**：为新内容预留一定曝光机会；
- **商业化约束 (Commercial Constraint)**：保证广告、电商等业务达到既定收益目标。

因此，混排中的序列生成过程通常并不是简单地按照分数排序，而是在生成候选序列时不断检查各种约束条件。一个典型的约束过滤伪代码如下：

代码 12.10: 带约束规则的混排序列生成伪代码

```
for sequence in candidate_sequences:
    if violate_ad_gap(sequence):
        continue
    if violate_live_position(sequence):
        continue
    if violate_diversity(sequence):
        continue
    if violate_frequency(sequence):
        continue
    valid_sequences.append(sequence)
```

更进一步，在 Beam Search 等搜索算法中，约束通常会提前融入搜索过程。当某个部分序列已经违反约束时，系统会直接剪枝 (Pruning)，而不会继续扩展后续节点：

代码 12.11: 带约束规则的混排 beam search 伪代码

```
for partial_sequence in beam:
    next_items = generate_candidates()
    for item in next_items:
        if violate_constraints(partial_sequence, item):
            continue
        expand(partial_sequence, item)
```

这样能够显著减少无效搜索空间，提高序列生成效率。

从工业界实践来看，随着业务数量不断增加，混排系统往往需要同时满足数十甚至上百条约束规则。因此，约束管理本身也逐渐演变为独立的规则引擎（Rule Engine）或约束求解模块（Constraint Solver）。很多时候，一个混排系统的复杂度并不来自排序模型本身，而是来自这些相互耦合且不断变化的业务约束。

12.6.2.2 业务预竞价与全竞价架构

在混排序列生成过程中，一个经常被忽略但非常重要的问题是：

 **笔记** 究竟应该由上游业务系统决定是否参与竞价，还是由混排系统统一决定最终的流量分配？

这个问题会直接影响到混排模块序列生成这一部分的工程实现。围绕这个问题，工业界通常存在两种不同的架构设计思路：**业务预竞价（Pre-Auction）**和**全竞价（Full Auction）**。

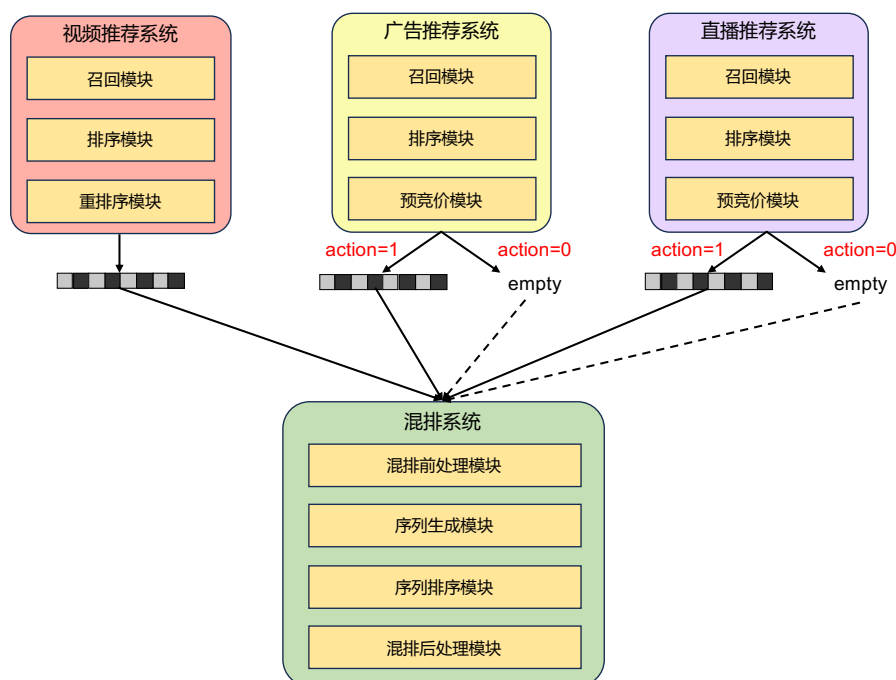


图 12.19: 业务预竞价的系统架构。

如图 12.19 所示，在业务预竞价（Pre-Auction）架构中，广告、直播、电商等业务系统会首先在各自业务内部完成排序与竞价过程，并提前决定哪些内容具备参与混排的资格。对于不满足竞价条件的候选 Item，业务侧会直接进行过滤，不再传递给混排模块。例如，广告系统可能结合 eCPM、预算约束、频次控制等因素进行内部决策，最终仅输出最多 3 个广告候选。此时预竞价模块实际上完成了一次是否参与混排的判断：当输出动作为 1 时，表示广告参与后续混排；当输出动作为 0 时，则不向混排系统下发广告候选。类似地，直播系统也可以根据用户画像、直播偏好以及业务目标等信息，决定是否向混排系统下发直播候选。经过业务侧预竞价之后，混排模块接收到的候选规模通常已经被显著压缩，因此后续序列生成过程更多是在少量候选内容之间进行位置组合与排序优化，而无需面对庞大的候选空间。

这种架构的优点主要体现在以下几个方面：

- 候选规模较小，序列空间有限。尤其是在广告、直播等业务未参与竞价时，混排阶段序列生成的搜索空间会进一步地缩小；
- 混排复杂度较低，计算成本和在线时延更容易控制；
- 各业务能够独立维护自身竞价逻辑，并充分利用业务侧特有的特征、约束以及优化目标进行决策；
- 系统职责划分清晰，模块边界明确，工程实现和迭代维护相对简单。

然而，这种架构也存在明显的局限性。由于业务侧已经提前完成了候选筛选和流量决策，混排系统实际上丧失了部分全局优化能力。例如，当直播推荐系统已经决定输出 1 个广告候选时，混排阶段只能优化这 1 个直播的插入位置，而无法进一步探索“插入 2 个直播”或“插入 3 个直播”等其他可能的序列方案。从本质上来看，业务

预竞价将部分流量分配决策前置到了各业务推荐系统内部，从而缩小了混排系统的决策空间。虽然这种方式能够显著降低系统复杂度，但也可能限制混排模块对用户体验、商业收益以及长期价值进行全局权衡的能力，因此容易与理论上的全局最优解存在一定差距。

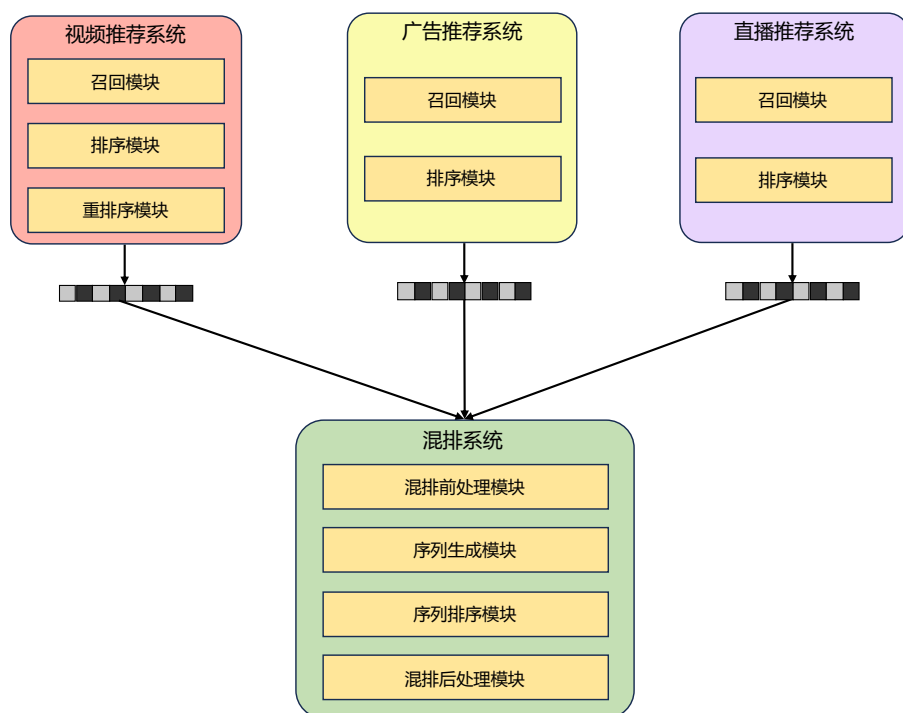


图 12.20: 业务全竞价的系统架构。

与之相对应的是全竞价架构。如图12.20所示，在全竞价模式下，广告、直播、电商等业务系统不会提前决定本次请求是否参与竞价，而是将排序后需要参与竞价的候选列表整体传递给混排系统。例如，广告系统返回 Top5 广告候选，直播系统返回 Top5 直播候选，短视频系统返回 Top8 视频候选。最终由混排系统统一决定：

- 是否插入广告；
- 插入多少广告；
- 是否插入直播；
- 插入哪些直播；
- 各业务内容的具体排序位置。

在这种架构下，混排模块不仅能够生成包含广告和直播的序列，也能够生成完全不包含广告、完全不包含直播的序列，并通过序列评估模型统一比较各种方案的收益。从理论上讲，全竞价架构更容易获得全局最优解，因为所有业务都处于同一个竞争空间之中，流量分配决策统一由混排系统完成。

然而，全竞价架构也会带来新的挑战。首先，随着业务种类和候选 Item 数量的不断增加，合法候选序列的数量将呈现组合数级增长，从而产生严重的组合爆炸（Combinatorial Explosion）问题。此时无论是序列生成阶段还是后续序列评估阶段，都将面临巨大的算力消耗和时延压力。尤其是在工业级推荐系统中，混排模块通常运行在毫秒级延迟约束下，因此不可能对海量候选序列进行充分评估。其次，全竞价会将大量业务决策逻辑集中到混排模块。例如，业务是否参与竞价、参与竞价的候选规模以及最终曝光数量等问题，都需要在混排序列生成阶段完成决策。随着业务数量增加，混排模块逐渐演变为一个统一的流量调度中心，其需要同时兼顾用户体验、商业收益、内容生态以及长期价值等多个优化目标。

然而，混排模块天然存在信息边界问题。相比于广告、直播、电商等业务推荐系统，混排模块通常只能获取经过加工后的候选结果和有限的特征信息，而难以访问各业务内部更加丰富的用户特征、内容特征以及业务状态信息。因此，在相同算力约束下，混排模块对于“是否参与竞价”、“参与多少候选”的判断，未必能够达到业务侧预竞价模块的决策精度。从另一个角度来看，业务预竞价实际上也是一种有效的搜索空间剪枝（Pruning）机

制。通过在前链路提前完成部分业务决策，可以显著缩小混排阶段需要考虑的候选空间，从而降低序列生成的复杂度。同时，候选序列规模的下降还能够释放更多算力预算，使系统能够采用更加复杂、更高精度的序列评估模型进行最终打分。

在工业实践中，业务预竞价与全竞价并不存在绝对优劣。业务预竞价更强调系统效率、模块解耦以及工程可控性，而全竞价更强调全局最优和统一流量分配能力。许多大型推荐系统往往采用折中方案：由业务侧完成第一轮粗粒度筛选和预竞价，再由混排系统在有限候选空间内进行统一决策，从而在排序效果、系统复杂度以及在线时延之间取得平衡。

从系统架构与设计的角度来看，业务预竞价与全竞价本质上体现的是流量决策权在业务侧与混排侧之间的不同划分方式。前者更倾向于分布式决策，而后者更倾向于集中式决策。在分布式决策的预竞价框架下，业务侧承担更多决策责任时，混排系统更接近排序器（Ranker）；而在集中式决策的全竞价框架下，混排模块则承担了统一的流量分配职责，这时的混排系统更接近全局资源调度器（Global Allocator）。上述这两种架构各有适用场景，其选择往往取决于业务复杂度、算力预算以及系统对全局最优性的追求程度。

12.6.3 序列排序

混排模块的序列排序阶段主要包括序列评估（Sequence Evaluation）和序列排序（Sequence Ranking）两个部分。其中，序列评估阶段与重排序模块中的序列评估过程较为类似，其目标是对序列生成阶段产生的多个候选序列进行价值估计；而序列排序阶段则根据评估结果对所有候选序列进行排序，并最终选择价值最高的序列作为最终曝光结果。

在实际工程中，序列评估可以采用两种方式完成：

- 直接利用序列评估模型直接预测序列整体价值；
- 将序列评估模型输出与业务价值分进行融合，构建最终的序列价值分。

因此，一个常见的序列融合分计算公式可以表示为：

$$Score_{seq} = \alpha \cdot Score_{model} + \beta \cdot Score_{business} \quad (12.163)$$

，其中：

- $Score_{model}$ 表示序列评估模型预测的整体收益；
- $Score_{business}$ 表示广告收益、时长收益、电商收益等业务目标的加权组合；
- α, β 表示不同目标之间的权重参数。

对于多目标混排场景，还可以进一步扩展为：

$$Score_{seq} = \sum_{k=1}^N w_k \cdot V_k \quad (12.164)$$

，其中 V_k 表示第 k 个业务目标对应的价值指标，例如广告收入、观看时长、用户留存、直播成交额等，而 w_k 则表示对应目标的重要程度。

然而，仅仅对序列中的 Item 分数进行简单求和往往是不合理的，因为用户对于不同位置内容的关注程度存在显著差异。通常情况下，靠前坑位获得的曝光概率和点击概率远高于靠后坑位，这种现象被称为**位置偏差（Position Bias）**。因此，在计算序列价值时，工业界通常会引入**位置衰减因子（Position Discount Factor）**：

$$Score_{seq} = \sum_{i=1}^L \gamma_i \cdot Score_i \quad (12.165)$$

，其中：

- $Score_i$ 表示第 i 个位置 Item 的价值分；
- γ_i 表示位置权重；
- L 表示序列长度。

一种常见的位置衰减方式为指数衰减：

$$\gamma_i = e^{-\lambda(i-1)} \quad (12.166)$$

，其中 λ 为衰减系数。例如： $\lambda = 0.2$ 时：

$$\gamma_1 = 1.0, \gamma_2 = 0.82, \gamma_3 = 0.67, \gamma_4 = 0.55 \quad (12.167)$$

从而体现首屏内容远比尾部内容更重要的特点。

除此之外，在混排系统中，还经常会遇到候选序列长度不一致的问题。例如：

- 序列 A 包含 6 个短视频和 1 个直播，总长度为 7；
- 序列 B 仅包含 6 个短视频，总长度为 6。

如果直接对序列中所有 Item 的价值分进行累加，则长度更长的序列往往会获得更高的总价值分，从而在最终排序过程中占据天然优势。然而，这种优势并非来源于序列质量本身，而是由序列长度差异所导致的统计偏差，因此会影响不同候选序列之间的公平比较。

针对这一问题，工业界通常采用以下两种解决方案：

- 对序列总价值进行长度归一化（Length Normalization）；
- 引入虚拟 Item（Mock Item）对较短序列进行长度补齐。

其中，第二种方案在实际业务场景中更为常见。

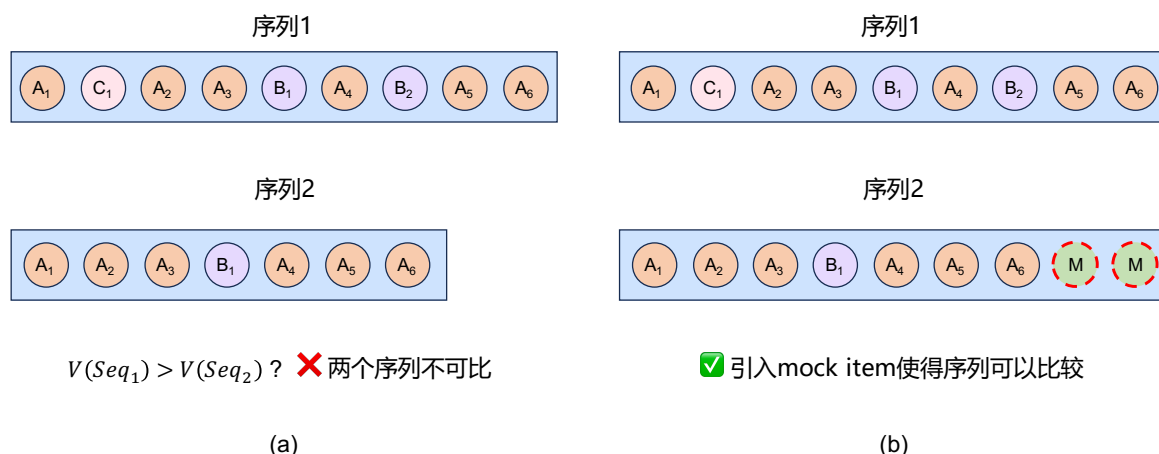


图 12.21: 混排序列排序中的虚拟 Item 补齐示意图。

如图 12.21(a) 所示，设 $A_1, A_2, \dots, A_6$ 表示短视频内容， B_1, B_2 表示广告内容， C_1 表示直播内容， M 表示虚构的内容。此时，序列 1 的长度为 9，而序列 2 的长度仅为 7。若直接将两个序列中所有 Item 的价值分进行求和，由于序列 1 包含更多的 Item，其总价值分可能天然高于序列 2，从而使排序策略过度偏向长度更长的候选序列。然而，不同长度的序列在统计意义上并不具备直接可比性，因此这种排序方式并不合理。

为了解决这一问题，可以采用 Mock Item 对序列进行长度对齐。如图 12.21(b) 所示，在序列 2 末尾补充 2 个虚拟 Item，使其长度与序列 1 保持一致。经过长度补齐后，不同候选序列的总价值分便能够在统一标准下进行比较，从而提高序列排序的公平性与合理性。在工业实践中，Mock Item 并不对应真实内容，其价值分通常设置为当前业务类型下内容价值分的均值，或者依据历史统计数据估计，以尽可能减小其对序列整体价值评估结果的影响。

当所有候选序列完成价值评估后，混排系统根据序列融合分（Sequence Score）进行排序：

$$Seq^* = \arg \max_{Seq} Score_{seq} \quad (12.168)$$

即从所有候选序列中选择融合价值最高的 Top-1 序列作为最终输出结果。

12.6.3.1 序列价值评估体系

需要注意的是，从算法实现角度来看，序列排序本身并不复杂，其核心挑战更多来自序列价值函数的设计。位置偏差建模、多目标融合以及长度归一化等问题，往往比排序过程本身更加重要。很多时候，一个优秀的混排系统并不是因为使用了复杂的排序算法，而是因为构建了更加准确的序列价值评估体系。

而在序列价值评估体系构建过程中，一个非常关键且具有挑战性的问题在于：

 **笔记** 不同业务之间的价值究竟应该如何进行统一衡量？

在混排序列价值计算过程中，每个坑位对应的 **Item** 可能来自完全不同的业务。例如，短视频内容的核心价值通常体现在用户消费时长、完播率或用户留存等指标上；直播内容的价值可能更多来源于打赏收入、商品成交额以及用户互动行为；广告内容则主要体现为营销收入或 **eCPM** 收益。不同业务的价值定义、统计口径以及数值分布均存在显著差异，因此这些业务分天然不具备直接比较的能力。

最简单的处理方式是通过归一化方法将不同业务的价值分映射到统一范围，例如采用 **Min-Max** 归一化、**Z-Score** 标准化等方式完成数值对齐。然而，这种方法本质上只是对数据进行了缩放处理，并没有真正解决不同业务价值口径不一致的问题。在工业实践中，更常见的做法是通过统计分析和长期实验建立**不同业务价值之间的兑换关系 (Exchange Rate)**。例如，通过历史数据分析评估广告收入、直播打赏收入以及用户消费时长之间的关联关系，从而得到某种统一的价值衡量标准。这样一来，直播产生的打赏收益、广告带来的营销收入以及其他商业化收益都可以折算为统一的时长价值、留存价值或者平台价值，最终归约到同一个价值体系下进行计算。

下面通过一个简化示例来说明业务价值统一的过程。假设平台经过长期实验分析发现，不同用户行为对于平台整体价值的贡献关系如表 12.7 所示。

表 12.7: 不同业务价值的兑换关系示例

用户行为	平台收益	折算时长价值
用户消费 1 分钟短视频	0.05 元	1 分钟
广告收入 1 元	1 元	20 分钟
直播打赏 1 元	0.5 元	10 分钟
电商 GMV 1 元	0.1 元	2 分钟

从表中可以看到，平台选择将「用户消费时长」作为统一价值单位。由于：

$$1\text{分钟时长价值} = 0.05\text{元} \quad (12.169)$$

，因此：

$$1\text{元平台收益} = 20\text{分钟时长价值} \quad (12.170)$$

，从而可以得到各业务对应的兑换关系：

$$Value_{ad} = 20 \times Revenue \quad (12.171)$$

$$Value_{live} = 10 \times Gift \quad (12.172)$$

$$Value_{ecommerce} = 2 \times GMV \quad (12.173)$$

这样不同业务的价值便可以映射到同一个价值空间之中。

例如，当前有如下候选 **Item**，如表 12.8 所示。经过统一价值映射之后，混排系统看到的不再是广告收入、直播打赏金额或者电商成交额，而是统一价值分：

$$Value(item) = 3, 4, 5, 3 \quad (12.174)$$

表 12.8: 统一价值计算示例

Item	业务类型	原始价值	统一价值分
Video-A	短视频	预计观看 3 分钟	3
Ad-B	广告	eCPM=0.20 元	4
Live-C	直播	预计打赏 0.50 元	5
Ecom-D	电商	GMV=1.50 元	3

。此时，不同业务的内容终于具备了直接比较的能力。进一步地，假设当前位置存在位置偏差（Position Bias），对应的位置权重如下：

表 12.9: 位置衰减权重示例

位置	权重
1	1.0
2	0.8
3	0.6
4	0.4

此时，两个候选序列：

$$Seq_1 = [Live-C, Ad-B, Video-A] \quad (12.175)$$

$$Seq_2 = [Ad-B, Live-C, Video-A] \quad (12.176)$$

对应的序列价值分别为：

$$Score(Seq_1) = 1.0 \times 5 + 0.8 \times 4 + 0.6 \times 3 = 10.0 \quad (12.177)$$

$$Score(Seq_2) = 1.0 \times 4 + 0.8 \times 5 + 0.6 \times 3 = 9.8 \quad (12.178)$$

。因此：

$$Score(Seq_1) > Score(Seq_2) \quad (12.179)$$

。最终，混排系统会选择 Seq_1 作为排序结果。

需要注意的是，上述示例仅用于说明业务价值统一的基本思想，具体数据并不一定合理。在实际工业系统中，兑换关系往往并不是固定常数，而是通过长期 AB 实验观测、ROI 分析、LTV（Life Time Value）建模以及因果推断等方法动态估计得到的。随着业务目标和平台阶段的变化，不同业务之间的价值兑换比例也会持续调整。

进一步地，一些成熟的推荐系统还会构建**统一价值模型（Unified Value Model）**，直接为每个 Item 计算综合价值分。该价值分同时考虑用户体验、商业收益、内容生态以及长期价值等多个目标，使不同业务的内容能够在同一个价值空间中进行公平竞争。在统一价值体系的基础上，序列价值函数可以表示为：

$$Score_{seq} = \sum_{i=1}^L \gamma_i \cdot Value(item_i) \quad (12.180)$$

其中：

- $Value(item_i)$ 表示统一价值模型计算得到的 Item 价值；
- γ_i 表示对应位置的曝光衰减权重；
- L 表示序列长度。

此时，无论当前位置展示的是短视频、直播、广告还是其他业务内容，都能够通过统一价值函数参与序列收益计算。

从本质上来看,混排系统最终优化的并不是广告价值、直播价值或者时长价值本身,而是**平台整体价值(Platform Value)**。只有当所有业务被映射到统一的价值空间之后,序列评估模型才能真正比较不同序列之间的优劣,混排系统的流量分配也才能更加科学、合理,并最终实现用户体验、商业收益以及内容生态的长期平衡与共同繁荣。

12.6.4 混排后处理

当序列排序完成后,系统将获得融合价值最高的 Top-1 候选序列。最直接的做法是将该序列直接下发至服务端,由服务端完成必要的数据处理后返回客户端,并最终曝光给用户。然而,在真实工业场景中,完全依赖序列排序结果往往存在较大的风险。

一方面,序列生成、序列评估以及序列排序等环节所依赖的模型均可能出现异常。例如模型训练任务中断、特征服务故障、实时数据流出现脏数据、数据分布发生漂移(Data Drift)、模型参数出现 Nan 值等问题,都可能导致部分业务的价值评估结果出现明显偏差。另一方面,即使模型本身运行正常,也难以保证其能够覆盖所有极端场景。当排序模型发生误判时,最终输出的 Top-1 序列可能并非真正符合业务目标的最优结果。例如广告价值被异常放大、直播内容被过度推荐,甚至某些业务流量在短时间内出现剧烈波动,从而对用户体验和平台生态产生负面影响。

因此,在工业级推荐系统中,混排模块通常不会将排序结果直接下发,而是会在最终曝光之前增加一道后处理(Post-processing)环节,对输出结果进行最后的安全校验和业务修正,以保证系统整体运行的稳定性与可控性。

12.6.4.1 Load 调控

Load 调控(Load Control)是推荐系统混排后处理阶段的核心功能模块,其核心价值在于推荐链路的末端环节,实现多业务流量的实时闭环调控,从机制上规避上游召回、排序、混排模型异常波动、模型迭代偏差、内容突发热点等因素引发的流量分配失衡问题,保障平台生态与用户体验的稳定性。

在多业务混排的工业场景中,混排模型通过学习广告、直播、普通内容等多类型物料的价值分布,完成内容的初步混合排序,但模型在实际在线运行过程中难以避免动态波动。例如,广告预估价值被异常高估、直播内容预测点击率突增、热点内容挤压普通内容曝光空间等场景,若完全依赖模型原始排序结果,会导致单一业务流量短时间内急剧膨胀,直接降低用户体验,甚至影响平台长期留存与生态平衡。

基于此,工业界普遍为各业务设定流量占比目标与安全波动区间,通过滑动时间窗口实时统计各业务的实际曝光占比,基于实际占比与目标占比的偏差动态修正内容排序分数或曝光概率,最终实现流量的稳态控制。典型的业务流量约束配置可表述为:广告流量占比约束在 10% ~ 15% 区间,直播流量占比约束在 5% ~ 8% 区间,普通内容流量占比设置不低于 80% 的兜底约束。

在流量闭环控制的技术方案中,PID(Proportional-Integral-Derivative)控制器作为经典控制理论的核心算法,凭借无训练数据依赖、结构简洁、可解释性强、鲁棒性优异、实时响应快等特性,成为推荐、广告、搜索系统流量 Load 调控的主流方案。PID 控制器以目标流量占比与实际流量占比的偏差为核心输入,通过比例、积分、微分三项的协同作用,输出精准的调节量,实现流量的动态纠偏。定义 t 时刻目标流量占比为 r_t ,当前时间窗口内统计得到的实际流量占比为 y_t ,二者的偏差构成控制误差,其数学表达为:

$$e_t = r_t - y_t \quad (12.181)$$

该误差是 PID 控制器的调节依据,当误差为正时,代表实际流量占比低于目标值,需要提升对应业务的曝光权重;当误差为负时,代表实际流量占比高于目标值,需要降低对应业务的曝光权重;误差为零时则无需调节。PID 控制器的输出由比例项、积分项、微分项线性叠加构成,完整数学表达式为:

$$u_t = K_P e_t + K_I \sum_{i=1}^t e_i + K_D (e_t - e_{t-1}) \quad (12.182)$$

其中 K_P 、 K_I 、 K_D 分别为比例系数、积分系数、微分系数,三项组件各司其职且协同互补。比例项直接映射当

前控制误差，实现对流量偏差的即时响应，误差越大调节力度越强，是控制器快速纠偏的核心；积分项对历史误差进行累加运算，能够消除长期稳态误差，确保实际流量占比最终精准收敛至目标值，弥补纯比例调节的固有缺陷；微分项通过误差的变化率预测调节趋势，对偏差的突变进行提前抑制，有效降低调节过程中的震荡与超调，提升系统稳定性。

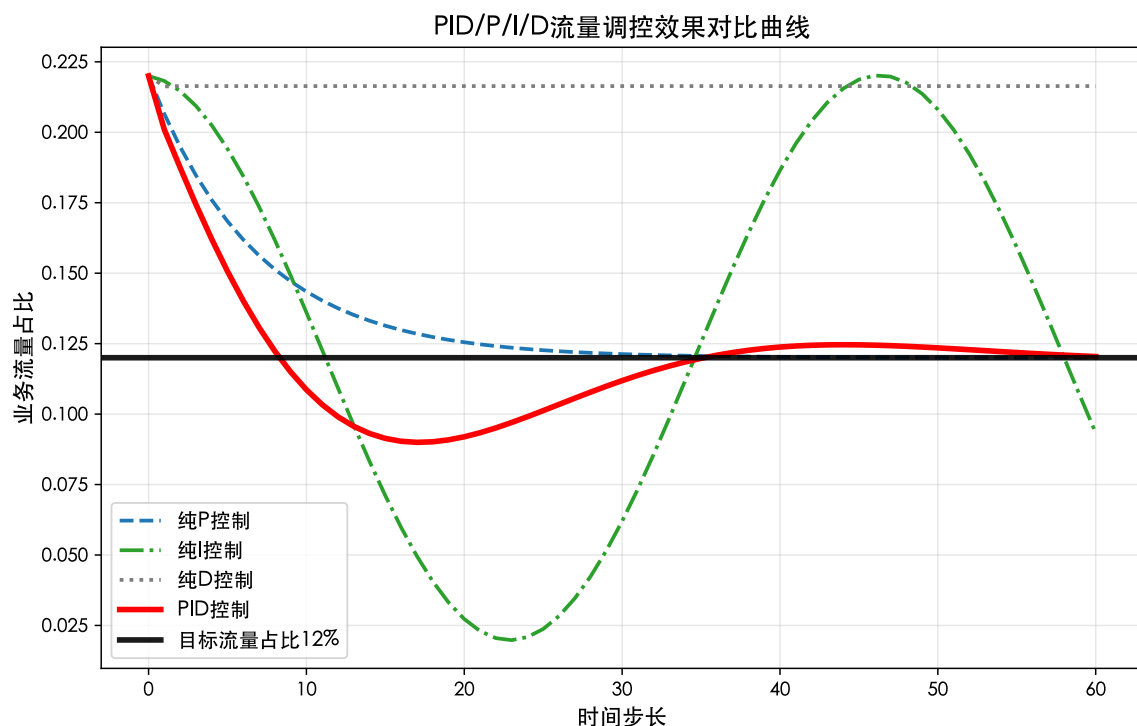


图 12.22: PID 控制与 P、I、D 控制对比示意图。

PID 控制的示意图如图 12.22 所示，一开始假设业务初始流量占比 22%、目标稳态占比 12%，分别构建仅启用比例、仅启用积分、仅启用微分、三项联合 PID 这四类调控逻辑。仅采用纯 P 控制时，流量可以快速向着目标值回落，但曲线最终停滞在高于目标线的位置，始终存在固定稳态残差，无法精准贴近 12% 基准线，这正是缺少积分项无法消除累积静差的典型表现。纯 I 控制依托误差累积的持续修正能力可以消除稳态误差、最终收敛到目标占比，但前期收敛速度缓慢，调节进程滞后，短时间内难以快速压制过高的流量占比。纯 D 控制仅感知误差变化速率，无法自主消除固定偏差，整条曲线几乎无法向目标值收敛，无法独立完成流量闭环调控。

而完整 PID 控制融合三项优势，依靠比例项实现快速下行纠偏、积分项根除稳态误差、微分项抑制曲线震荡超调，曲线平稳、快速地收敛至目标流量基准线，兼顾收敛速度与稳态精度，这也是该算法在工业流量负载管控中大规模落地的关键原因。从工程落地视角来看，三种单一项控制各自存在明显短板，无法单独满足线上多业务流量平稳约束的需求，PID 的组合形式补齐各项缺陷，完美适配混排后实时流量熔断与稳态调控的业务诉求。

在推荐系统 Load 调控的工程落地中，PID 控制器的输出 u_t 可以直接作用于内容排序分数的修正环节，将原始混排分数叠加调节量后得到最终排序分数，即

$$score_{final} = score_{raw} + u_t \quad (12.183)$$

，基于修正后的分数重新完成内容排序。通过实时统计曝光数据、计算控制误差、输出调节量、修正排序分数的闭环流程，实现流量占比的持续稳态控制。例如，广告业务目标流量占比为 12%，若实时统计占比达到 15%，此时误差 $e_t = 12\% - 15\% = -3\%$ ，PID 控制器输出负向调节量，降低广告内容的排序分数，减少其曝光概率；若广告实际占比低于目标值，则输出正向调节量提升曝光权重。

相较于强化学习、多目标优化等复杂调控策略，PID 控制器无需标注数据与模型训练，不依赖上游模型参数，在模型异常、流量突增等极端场景下仍能稳定工作，具备极强的工程实用性。从系统架构层面，Load 调控本质是推荐系统的熔断保护机制，能够独立于上游模块工作，即便召回、排序等核心模型出现异常，也能约束

各业务流量在安全区间内，避免流量剧烈波动，维持平台整体生态的稳定运行。

Load 调控的工程落地高度依赖对真实流量分布的持续观测，工业场景中普遍采用滑动时间窗口或固定计数窗口完成流量占比统计。系统会对每一次用户请求返回的曝光结果进行实时采集，将最近 N 次曝光记录纳入统计窗口，动态计算目标业务的实际流量占比，这一统计结果作为 PID 控制器的输入，形成完整的流量反馈闭环。

基于窗口统计的 PID 控制器能够有效规避瞬时波动带来的调节震荡，使流量调控更贴合业务的真实分布状态。在调节机制上，比例项依据当前窗口内的流量偏差提供即时纠偏能力，积分项通过累积历史窗口误差消除稳态偏差，微分项则根据误差变化趋势抑制调节超调，三者协同使流量占比平滑收敛至目标区间。为适配在线系统的稳定性要求，工程实现中通常引入积分限幅与调节量裁剪机制，避免积分累积饱和与分数剧烈波动，确保系统在高并发、高动态的流量场景中始终保持鲁棒运行。这种基于滑动窗口的 PID 流量调控方案，无需模型训练、不依赖上游模型状态，能够在模型异常、流量突变等场景下独立完成流量熔断与稳态保护，成为推荐系统流量治理体系中不可或缺的核心环节。

下面，我们给出一个工业界场景下基于 N 次请求窗口的 PID 控制的 C++ 类的具体实现逻辑。

代码 12.12: 带请求窗口的 PID 控制示例代码

```
#include <vector>
#include <mutex>
#include <cmath>
#include <algorithm>

class PIDLoadController {
private:
    struct Window {
        uint64_t total = 0;
        uint64_t exposed = 0;
    };

    float target_load_;
    float kp_, ki_, kd_;
    float last_error_{0.0f};
    float integral_{0.0f};
    float max_integral_{1.0f};
    float max_adjust_{0.5f};

    int window_num_;
    int window_size_;

    std::vector<Window> windows_;
    int current_window_idx_{0};
    uint64_t req_count_in_win_{0};

    mutable std::mutex mtx_;

    float unsafe_get_current_load() const {
        uint64_t total = 0, exposed = 0;
        for (const auto& w : windows_) {
            total += w.total;
            exposed += w.exposed;
        }
    }
};
```

```

    return (total == 0) ? 0.0f : static_cast<float>(exposed) / total;
}

public:
    PIDLoadController(float target_load, float kp, float ki, float kd,
                      int window_num, int window_size,
                      float max_adjust = 0.5f, float max_integral = 1.0f)
        : target_load_(target_load), kp_(kp), ki_(ki), kd_(kd),
          window_num_(window_num), window_size_(window_size),
          max_adjust_(max_adjust), max_integral_(max_integral)
    {
        windows_.resize(window_num_);
    }

    void report(bool has_ad) {
        std::lock_guard<std::mutex> lock(mtx_);
        if (req_count_in_win_ >= window_size_) {
            current_window_idx_ = (current_window_idx_ + 1) % window_num_;
            req_count_in_win_ = 0;
            windows_[current_window_idx_] = {};
        }
        auto& win = windows_[current_window_idx_];
        win.total++;
        if (has_ad) win.exposed++;
        req_count_in_win_++;
    }

    float get_current_load() const {
        std::lock_guard<std::mutex> lock(mtx_);
        return unsafe_get_current_load();
    }

    float compute_adjustment() {
        std::lock_guard<std::mutex> lock(mtx_);
        float current = unsafe_get_current_load();
        float error = target_load_ - current;

        float P = kp_ * error;
        integral_ = std::clamp(integral_ + error, -max_integral_, max_integral_);
        float I = ki_ * integral_;
        float D = kd_ * (error - last_error_);
        last_error_ = error;

        return std::clamp(P + I + D, -max_adjust_, max_adjust_);
    }
};

```

接下来是具体的主函数入口和模拟请求的调用，代码如下所示。需要注意的是，在 main 函数的 *has_ad* 判断的时候，出于样例代码的考虑，这里采用随机的方式进行生成。实际在工业界推荐系统中，*has_ad* 的判断完

全是由序列生成和序列排序阶段决定的。最终序列排序阶段排出来的 `top1` 序列中没有广告，则 `has_ad` 为 0，反之为 1。这与示例代码中是完全不同的。

代码 12.13: 带请求窗口的 PID 控制主函数代码

```
#include <iostream>
#include <iomanip>
#include <vector>
#include <cstdlib>
#include <ctime>
#include <thread>
#include <atomic>
#include "PIDLoadController.h"

using namespace std;

struct Item {
    bool is_ad;
    float raw_score;
};

// 全局共享：线程安全 PID 控制器
PIDLoadController pid(0.12f, 1.8f, 0.35f, 0.45f, 10, 500);

// 全局统计（多线程安全）
atomic<uint64_t> total_reqs{0};
atomic<bool> stop_flag{false}; // 新增全局退出标志
const int PRINT_INTERVAL = 500;
const uint64_t MAX_TOTAL = 10000; // 最大总轮次

// 单个工作线程：无限循环并发上报
void worker_thread() {
    while (!stop_flag.load()) { // 判断停止标记
        float adjust = pid.compute_adjustment();

        float ad_base_score = 0.20f + adjust;
        float prob = clamp(ad_base_score * 100.0f, 0.0f, 99.0f);
        bool has_ad = (rand() % 100) < prob;

        vector<Item> items;
        if (has_ad) items.push_back({true, 0.78f});
        items.push_back({false, 0.81f});
        items.push_back({false, 0.79f});
        items.push_back({false, 0.75f});

        bool ad_in_request = false;
        for (const auto& item : items) {
            if (item.is_ad) {
                ad_in_request = true;
            }
        }
    }
}
```

```

        break;
    }
}

pid.report(ad_in_request);

// 全局计数 + 定时打印
uint64_t now = ++total_reqs;
if (now % PRINT_INTERVAL == 0) {
    float current_load = pid.get_current_load();
    cout << now << "\t\t"
         << current_load << "\t\t"
         << 0.12f << "\t\t"
         << adjust << "\n";
}

// 到达阈值，触发全局停止
if (now >= MAX_TOTAL) {
    stop_flag = true;
}
}
}

int main() {
    cout << fixed << setprecision(4);
    cout << "总请求数\t当前广告占比\t目标占比\tPID调节量\n";
    cout << "-----" << endl;
    // 启动 8 个并发工作线程
    vector<thread> threads;
    int thread_num = 8;

    for (int i = 0; i < thread_num; ++i) {
        threads.emplace_back(worker_thread);
    }
    // 等待所有线程跑完退出
    for (auto& t : threads) {
        t.join();
    }
    cout << "\n全部运行结束，总请求：" << total_reqs << endl;
    return 0;
}

```

根据这个示例的多线程 PID 调控示例程序，模拟工业界并发请求的环境下的广告 load 调控过程，最终可以打印出广告的 Load 变化情况以及 PID 调节量的大小，如下表 12.10 所示。本次模拟验证以目标负载系数 $r = 0.12$ 为被控设定值，采用离散 PID 控制算法对广告曝光占比实施闭环调控，模拟产生的总样本量 10000，等间隔采样得到被控量与控制量序列。从下表中的实验数据可以看出：基于 PID 的 Load 调控系统经动态调节后，被控输出稳态运行区间为 $[0.1170, 0.1280]$ ，被控量围绕设定值小幅往复振荡，稳态偏差被限制在 ± 0.008 以内；控制器输出可依据系统偏差极性实现双向控制：实际负载高于设定值时控制量为负，抑制广告投放概率；实际负载低

于设定值时控制量为正，抬升广告投放概率。

总请求数	当前广告占比	目标占比	PID 调节量
500	0.1280	0.1200	-0.2806
1000	0.1210	0.1200	0.0406
1500	0.1246	0.1200	-0.2361
2000	0.1194	0.1200	-0.2379
2500	0.1228	0.1200	-0.2152
3000	0.1220	0.1200	-0.1053
3500	0.1206	0.1200	0.0931
4000	0.1170	0.1200	-0.0944
4500	0.1195	0.1200	-0.1642
5000	0.1232	0.1200	-0.0379
5500	0.1170	0.1200	0.0065
6000	0.1179	0.1200	-0.1822
6500	0.1233	0.1200	-0.0700
7000	0.1168	0.1200	-0.0438
7500	0.1192	0.1200	-0.2515
8000	0.1226	0.1200	-0.0466
8500	0.1186	0.1200	0.0061
9000	0.1200	0.1200	-0.2029
9500	0.1250	0.1200	-0.0447
10000	0.1233	0.1200	0.0181

表 12.10: 多线程 PID 广告 Load 调控数据表

综上所述，示例代码中所整定 PID 参数可实现被控量对目标值的有效跟踪，系统动态收敛特性与稳态控制精度满足预设控制指标，闭环控制系统具备优良的稳态调节性能。在工业界具体实践中，通常我们也会在线上实时监控 Load 调控的曲线，从而判断 PID 调控中的超参数设置是否合理，如果 PID 调控的收敛速度很慢，则需要手动调参，进而提升 PID 调控的灵敏度与调控力度。

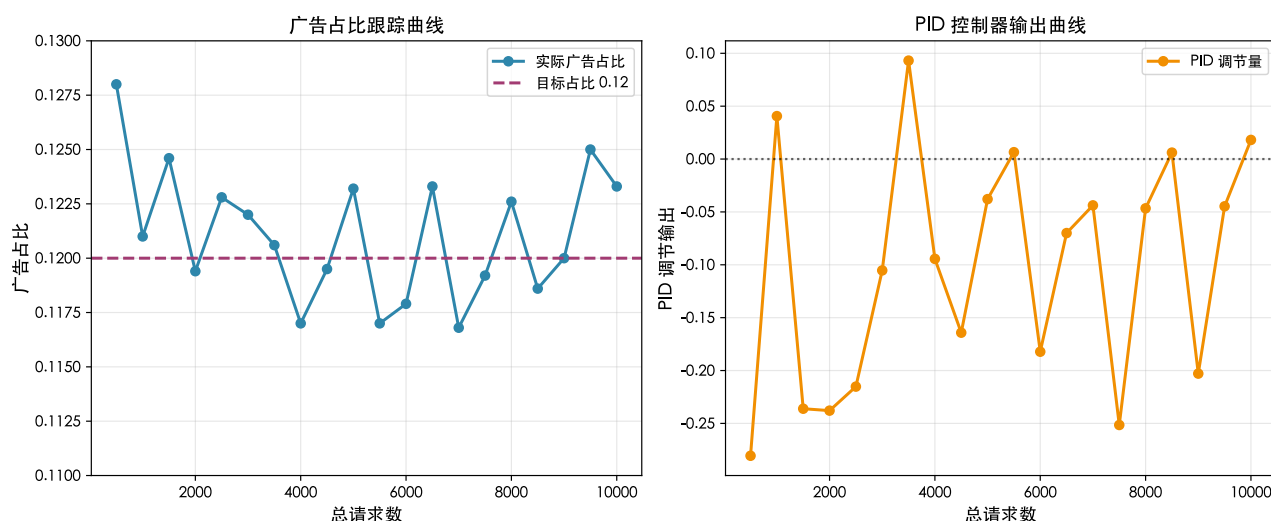


图 12.23: 广告 Load 占比及 PID 调控量监控曲线。

下面给出上述模拟示例中广告 Load 在 PID 调控过程中的监控曲线，以及对应的 PID 调控量变化情况，如图 12.23 所示。在工业界推荐系统的混排模块中，类似于图 12.23(a) 所示的广告 Load 占比监控是非常重要的线上监控指标。通常情况下，系统会为广告 Load 设置目标区间以及上下界阈值，并配置相应的报警策略。当广告 Load 超过设定上限，或者低于设定下限时，监控系统会自动触发告警。此时，工程师需要及时排查推荐链路中的各个环节，包括召回、排序、混排以及流量调控模块，以确认是否存在模型异常、特征缺失、数据延迟或系统故障等问题，从而避免线上流量分配失衡导致用户体验下降或业务收益受损。除了广告业务之外，直播、电商、

图文、短剧等业务的流量占比通常也会在混排模块进行实时监控。对于大型推荐系统而言，这些业务的 Load 监控往往构成了核心的业务健康度指标，是保障平台生态稳定运行的重要基础设施。

相比之下，图 12.23(b) 所示的 PID 调控量监控曲线虽然同样重要，但其关注重点与 Load 占比监控有所不同。Load 占比监控主要用于观察最终业务流量是否处于合理区间，而 PID 调控量监控则更多用于分析控制器本身的运行状态。通过观察 PID 输出值的变化趋势，工程师可以判断 PID 控制器是否正常生效，以及当前调控力度是否合理。例如，当业务 Load 持续偏离目标值时，若 PID 调控量始终接近于零，则可能意味着控制器参数配置存在问题；而当 PID 调控量出现剧烈震荡时，则可能表明控制器过度调节，影响系统稳定性。

因此，在实际生产环境中，Load 占比监控与 PID 调控量监控通常需要结合使用：前者用于监控业务结果，后者用于监控控制过程。二者共同构成了混排流量调控模块的重要观测手段，从而保证 PID 控制策略能够长期稳定地运行，并持续发挥业务流量治理的作用。

12.6.4.2 运营强插

除了流量治理之外，混排后处理阶段还承担着产品运营策略落地的重要职责。在实际业务中，平台经常会举办各类运营活动，例如大型直播活动、节日营销活动、新业务推广活动等。此时运营人员通常希望某些指定内容能够获得确定性的曝光机会，而不仅仅依赖模型排序结果。

因此，系统通常会提供强插（Force Insertion）能力，允许运营人员通过后台配置规则，在特定用户群体、特定刷次以及特定坑位插入指定内容。例如：

- 指定白名单创作者的视频强插至第 5 个位置；
- 指定主播的直播间强插至用户首次刷新页面时的第 3 个位置；
- 针对特定人群包插入活动广告内容。

对于影响力极大的运营活动，例如头部明星直播、大型电商促销活动等，平台甚至可能采用全局强插策略，对所有用户统一插入指定内容，以保证活动获得充足曝光。从系统实现角度来看，强插通常发生在最终曝光前的最后阶段，其优先级往往高于混排排序结果，因此能够确保运营策略得到稳定执行。

12.6.4.3 保量与降权策略

除了流量治理和运营强插之外，混排后处理阶段还承担着平台生态调控的重要职责。在推荐系统的发展过程中，平台往往需要扶持某些新兴业务或内容生态。例如短剧、AI 漫剧、知识内容、本地生活等业务在发展初期通常缺乏足够的数据积累，如果完全依赖模型排序，可能很难获得充足曝光。

因此，平台通常会在混排后处理阶段增加保量（Quota Guarantee）机制。例如：

- 每 100 次曝光至少包含一定比例的短剧内容；
- 新创作者内容获得最低曝光保障；
- AIGC 内容进行适当的降权打压；
- AI 漫剧内容获得额外曝光加权。

与保量策略相对应的是降权（Demotion）策略。对于低质内容、重复内容、高营销感内容或用户反馈较差的内容，可以在最终曝光前进一步降低其排序分数，从而减少其曝光机会。本质上，保量与降权都是平台生态治理的重要手段。其目标并非单纯优化短期点击率或观看时长，而是在更长周期内促进内容生态的健康发展，实现平台、创作者与用户之间的长期平衡。

综上所述，混排后处理并不追求更复杂的排序模型，而更关注系统的稳定性、安全性与业务可控性。通过 Load 调节、运营强插以及保量降权等机制，推荐系统能够在复杂业务环境下保持稳定运行，并确保平台战略目标得到有效落实。

12.7 生成式推荐模块

随着生成式模型（Generative Model）以及大语言模型（Large Language Model, LLM）的快速发展，推荐系统领域近年来也开始积极探索将生成式建模思想引入传统推荐架构之中。特别是在 Scaling Law、Transformer 架构以及 Next Token Prediction（NTP）训练范式取得巨大成功之后，越来越多的研究工作开始思考：推荐问题是否也能够像自然语言生成一样，被统一建模为一个序列生成问题。

传统推荐系统通常采用“召回-粗排-精排-重排”的级联式架构，通过多个模块逐步缩小候选集合并提升排序精度。而生成式推荐（Generative Recommendation）的核心思想则是：

定义 12.3

将推荐任务直接转化为一个序列生成任务：给定用户历史行为序列、上下文环境以及用户画像等信息，由模型直接生成用户未来最可能感兴趣的内容标识（Item ID），从而完成推荐决策。



然而，与自然语言中的词汇 Token 不同，推荐系统中的 Item 规模往往达到百万甚至亿级别，直接将 Item ID 作为生成目标会导致词表规模过大，训练和推理成本极高。因此，目前主流的生成式推荐工作通常会首先对 Item 进行语义离散化编码（Semantic Tokenization），将每个 Item 映射为一个或多个语义 ID（Semantic ID）。这些语义 ID 通常通过 Residual Quantization（RQ）[21]、VQ-VAE [34]、Product Quantization（PQ）[17] 等向量量化技术从 Item Embedding 中学习得到。例如，一个视频内容可能被编码为：

$$Item_A \rightarrow [s_1, s_2, s_3, s_4] \quad (12.184)$$

，其中 s_i 表示不同层级的语义编码。这样，原本庞大的 Item 空间便被映射到一个相对紧凑的语义 Token 空间之中。

在训练阶段，系统将用户行为序列转换为语义 ID 序列：

$$[s_1, s_2, s_3] \rightarrow [s_4] \quad (12.185)$$

或者：

$$Item_1 \rightarrow Item_2 \rightarrow Item_3 \rightarrow ? \quad (12.186)$$

并采用与大语言模型类似的 Next Token Prediction（NTP）目标进行训练。模型通过学习用户历史消费序列中的模式，预测用户下一步最可能感兴趣的语义 Token。训练完成后，模型便具备了根据用户行为历史直接生成未来兴趣内容的能力。

在线推理阶段，生成式推荐模型接收用户最近浏览的视频、点击行为、停留时长、搜索记录以及上下文特征等信息作为输入，通过自回归生成（Autoregressive Generation）的方式逐步生成语义 ID 序列。例如：

$$[s_1, s_2] \rightarrow [s_3] \rightarrow [s_4] \quad (12.187)$$

。当生成完整语义 ID 后，系统再通过预先构建的 Semantic ID 到 Item ID 映射表完成检索，从而获得对应的推荐内容集合。

12.7.1 生成式推荐架构

从系统架构角度来看，生成式推荐模块在工业级推荐系统中的位置如图 12.24 所示。与传统推荐架构不同，生成式推荐并不严格遵循“召回-粗排-精排”的逐层筛选流程，而是利用大模型的生成能力直接产生候选内容，因此其在整体架构中通常作为传统推荐链路之外的一条独立推荐通路存在。

目前工业界主要存在两种典型的部署方式。第一种方式是并行候选生成架构（Parallel Candidate Generation Architecture）。在这种模式下，生成式推荐模块与传统的“召回-粗排-精排”模块并行运行。传统推荐链路负责产

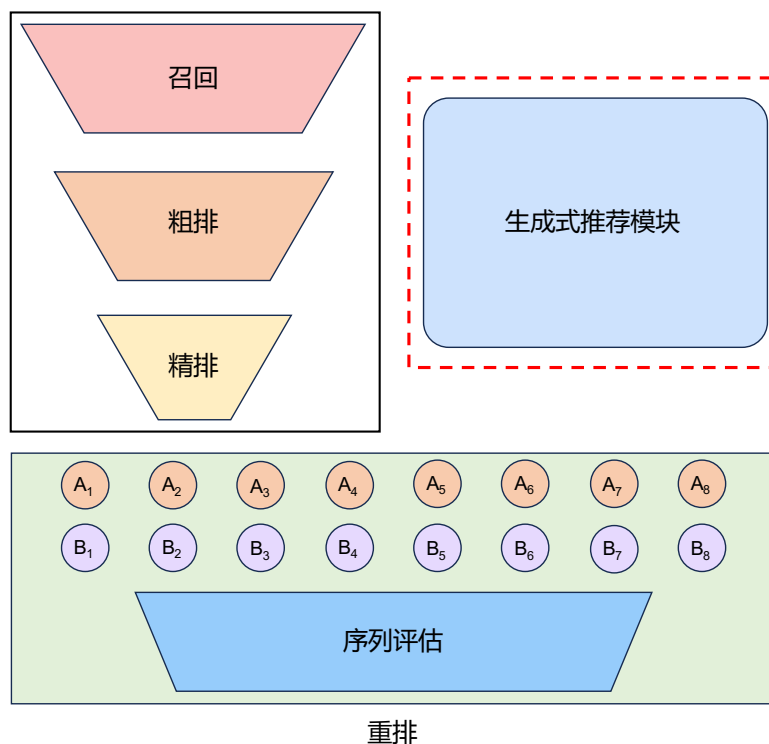


图 12.24: 生成式推荐系统架构图。

生大规模高覆盖率候选集，而生成式推荐模块则利用大模型直接生成一批高质量候选内容。两部分候选集合最终汇合进入重排模块进行统一排序。此时，生成式推荐模块本质上可以被视为一条特殊的召回通路。与传统基于 **Embedding** 向量召回、图网络召回等召回策略相比，生成式推荐能够直接利用用户行为序列中的长期兴趣和复杂上下文信息生成候选内容，因此通常被认为是一条质量较高的“直通重排召回”。这种传统推荐与生成式推荐并行执行的架构具有较好的工程兼容性。一方面可以充分利用生成式模型的表达能力；另一方面即使生成式模型出现异常，传统推荐链路仍然能够作为兜底方案保证系统正常运行，因此目前许多工业界落地案例都采用这种方式进行部署。

第二种方式是**流量隔离架构 (Traffic Isolation Architecture)**。在这种模式下，平台会从整体流量中划分出一定比例的实验流量，例如：5% ~ 30% 作为生成式推荐流量。当用户命中生成式推荐实验流量时，请求将直接进入生成式推荐模块，不再执行传统的召回、粗排和精排流程，而是由生成式模型直接生成候选 **Item** 集合，并送入重排模块完成最终排序。在这种情况下，整个推荐流程可以表示为：

$$User \rightarrow Generative Recommendation \rightarrow Reranking \rightarrow Result \quad (12.188)$$

。而对于未命中实验流量的用户，则仍然采用传统级联架构：

$$User \rightarrow Retrieve \rightarrow Coarse Ranking \rightarrow Fine Ranking \rightarrow Reranking \rightarrow Result \quad (12.189)$$

这种部署方式的优势在于能够更加准确地评估生成式推荐对整体推荐效果的增益情况。通过 **AB** 实验，平台可以直接比较生成式推荐架构与传统级联架构在 **CTR**、观看时长、用户留存以及商业化指标上的差异，从而评估生成式推荐的实际业务价值。

需要注意的是，即使在采用生成式推荐架构的场景下，大多数工业系统仍然会保留重排 (**Re-ranking**) 模块作为最后一道决策环节。原因在于生成式模型虽然能够生成用户可能感兴趣的内容，但仍然难以同时兼顾广告收益、直播曝光、多样性约束、生态治理以及流量调控等复杂业务目标。因此，重排模块仍然承担着多目标优化与最终决策的重要职责。

从当前 **LLM4Rec** 研究的发展趋势来看，生成式推荐更像是在传统推荐架构之上增加了一层强大的候选生成

能力，而不是完全替代整个推荐链路。未来推荐系统的发展方向，很可能是生成式模型与传统级联架构长期共存，并逐步形成融合式推荐范式（Hybrid Recommendation Architecture），从而同时兼顾模型能力、系统稳定性以及复杂业务目标的优化需求。

12.7.2 生成式推荐讨论

虽然生成式推荐（Generative Recommendation）近年来受到了学术界和工业界的广泛关注，并在推荐系统中取得了一系列令人瞩目的进展，但与此同时，一个值得深入思考的问题是：

 **笔记** 对于推荐系统这样一个复杂的 AI 驱动系统工程而言，One-Model 范式真的能够完全替代传统级联式推荐架构吗？

事实上，类似的讨论也出现在近年来快速发展的具身智能（Embodied AI）领域。随着 VLA（Vision-Language-Action）[2, 18] 等端到端模型的兴起，越来越多的研究尝试利用单一模型同时完成环境感知、语义理解、任务规划以及动作决策等多个环节，希望通过统一建模实现机器人系统的端到端优化。然而，对于具有实际工业模型训练经验的研究者而言，一个模型同时承担多个目标往往意味着需要在优化过程中兼顾多种甚至相互冲突的任务需求。

从优化视角来看，当一个模型需要同时最小化多个目标函数时，不同任务之间往往会产生梯度冲突（Gradient Conflict）。某一目标的优化方向可能会削弱另一目标的性能，从而导致模型在训练过程中面临复杂的 Trade-off 问题，并最终收敛到某种折中的局部最优解。虽然近年来出现了诸如 PCGrad [38]、GradNorm [5] 等多任务优化方法来缓解梯度冲突问题，但在复杂系统工程场景下，多目标协同优化依然是一个开放性研究课题。

推荐系统同样属于典型的复杂多目标优化系统。一个工业级推荐平台不仅需要最大化用户体验相关指标，例如点击率（CTR）、观看时长（Watch Time）、用户留存（Retention）等，同时还需要兼顾内容生态健康、新作者成长、内容多样性，以及广告、直播、电商等商业化业务的发展需求。这些目标之间并非完全一致，甚至在许多情况下存在天然冲突。例如，提高广告曝光比例可能有利于商业收益，却可能损害用户体验；过度追求短期点击率则可能影响内容生态的长期健康发展。

更重要的是，这些复杂的系统目标往往难以被统一表达为单一的损失函数。因此，从工程实践角度来看，推荐系统不仅仅是一个模型优化问题，更是一个涉及业务目标管理、流量治理、生态调控以及风险控制的复杂系统工程问题。

不可否认的是，生成式推荐的发展为推荐系统带来了新的可能性。借助大语言模型（LLM）强大的序列建模能力，生成式推荐已经在候选召回、粗排排序以及用户兴趣建模等场景展现出显著优势，甚至在部分场景下实现了召回与粗排阶段的一体化建模。这种端到端建模方式有效减少了传统推荐链路中的人工特征工程和模块间的信息损失，为推荐系统架构演进提供了新的方向。

然而，从当前工业实践来看，生成式推荐更有可能成为传统推荐架构的重要补充，而非完全替代者。一方面，复杂的多目标优化需求决定了推荐系统仍然需要大量业务规则、流量治理策略以及生态调控机制参与决策过程；另一方面，工业级推荐系统对于稳定性、可解释性以及容灾能力有着极高要求。传统级联架构中的召回、排序、混排以及后处理等模块天然形成了多层防护机制，即使某一模块出现异常，其影响范围通常也能够被限制在可控范围内。而在 One-Model 架构下，系统决策高度集中于单一模型，一旦模型出现训练异常、数据漂移、服务故障或预测失真等问题，其影响往往会被放大至整个推荐链路。

因此，从目前的发展阶段来看，生成式推荐所代表的端到端建模思想无疑是推荐系统未来的重要发展方向，但其更可能以“生成式能力融入级联架构”的形式逐步落地，而非完全取代传统推荐系统。对于推荐系统初学者、从业工程师以及学术研究者而言，在关注新技术突破的同时，也需要充分认识到工业级推荐系统背后所蕴含的复杂系统工程属性。传统级联架构能够在工业界长期存在并持续演进，其背后不仅是历史积累的结果，更体现了对多目标优化、系统稳定性以及业务可控性等问题的深刻权衡。

12.8 参考文献

- [1] Jinze Bai et al. “Qwen-VL: A Versatile Vision-Language Model for Understanding, Localization, Text Reading, and Beyond”. In: *arXiv preprint arXiv:2308.12966* (2023).
- [2] Anthony Brohan et al. “Rt-2: Vision-language-action models transfer web knowledge to robotic control, 2023”. In: *URL <https://arxiv.org/abs/2307.15818>* 1 (2024), p. 2.
- [3] Jaime Carbonell and Jade Goldstein. “The use of MMR, diversity-based reranking for reordering documents and producing summaries”. In: *Proceedings of the 21st annual international ACM SIGIR conference on Research and development in information retrieval*. 1998, pp. 335–336.
- [4] Laming Chen, Guoxin Zhang, and Eric Zhou. “Fast greedy map inference for determinantal point process to improve recommendation diversity”. In: *Advances in neural information processing systems* 31 (2018).
- [5] Zhao Chen et al. “Gradnorm: Gradient normalization for adaptive loss balancing in deep multitask networks”. In: *International conference on machine learning*. PMLR. 2018, pp. 794–803.
- [6] Paul Covington, Jay Adams, and Emre Sargin. “Deep neural networks for youtube recommendations”. In: *Proceedings of the 10th ACM conference on recommender systems*. 2016, pp. 191–198.
- [7] Jan De Leeuw, Kurt Hornik, and Patrick Mair. “Isotone optimization in R: pool-adjacent-violators algorithm (PAVA) and active set methods”. In: *Journal of statistical software* 32 (2010), pp. 1–24.
- [8] Chao Deng et al. “Calibrating user response predictions in online advertising”. In: *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*. Springer. 2020, pp. 208–223.
- [9] Alexey Dosovitskiy. “An image is worth 16x16 words: Transformers for image recognition at scale”. In: *arXiv preprint arXiv:2010.11929* (2020).
- [10] A Fielding. *Statistical inference under order restrictions. the theory and application of isotonic regression*. 1974.
- [11] Markus Freitag and Yaser Al-Onaizan. “Beam search strategies for neural machine translation”. In: *arXiv preprint arXiv:1702.01806* (2017).
- [12] Jennifer Gillenwater, Alex Kulesza, and Ben Taskar. “Near-optimal map inference for determinantal point processes”. In: *Advances in Neural Information Processing Systems* 25 (2012).
- [13] Kaiming He et al. “Deep residual learning for image recognition”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 770–778.
- [14] Andrew G Howard et al. “Mobilenets: Efficient convolutional neural networks for mobile vision applications”. In: *arXiv preprint arXiv:1704.04861* (2017).
- [15] Yanhua Huang et al. “Sliding spectrum decomposition for diversified recommendation”. In: *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*. 2021, pp. 3041–3049.
- [16] Michael Janner, Qiyang Li, and Sergey Levine. “Offline reinforcement learning as one big sequence modeling problem”. In: *Advances in neural information processing systems* 34 (2021), pp. 1273–1286.
- [17] Herve Jegou, Matthijs Douze, and Cordelia Schmid. “Product quantization for nearest neighbor search”. In: *IEEE transactions on pattern analysis and machine intelligence* 33.1 (2010), pp. 117–128.
- [18] Moo Jin Kim et al. “Openvla: An open-source vision-language-action model”. In: *arXiv preprint arXiv:2406.09246* (2024).
- [19] Alex Kulesza, Ben Taskar, et al. “Determinantal point processes for machine learning”. In: *Foundations and Trends® in Machine Learning* 5.2–3 (2012), pp. 123–286.

- [20] Alex Kulesza and Ben Taskar. “Learning determinantal point processes”. In: *Learning* 7 (2011), pp. 1–2011.
- [21] Doyup Lee et al. “Autoregressive image generation using residual quantization”. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2022, pp. 11523–11532.
- [22] Junnan Li et al. “Blip: Bootstrapping language-image pre-training for unified vision-language understanding and generation”. In: *International conference on machine learning*. PMLR. 2022, pp. 12888–12900.
- [23] Bruce P Lowerre and B Raj Reddy. “Harpy, a connected speech recognition system”. In: *The Journal of the Acoustical Society of America* 59.S1 (1976), S97–S97.
- [24] Jiaqi Ma et al. “Modeling task relationships in multi-task learning with multi-gate mixture-of-experts”. In: *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 2018, pp. 1930–1939.
- [25] Jiaqi Ma et al. “Snr: Sub-network routing for flexible parameter sharing in multi-task learning”. In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 33. 01. 2019, pp. 216–223.
- [26] Xiao Ma et al. “Entire space multi-task model: An effective approach for estimating post-click conversion rate”. In: *The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval*. 2018, pp. 1137–1140.
- [27] Odile Macchi. “The coincidence approach to stochastic point processes”. In: *Advances in Applied Probability* 7.1 (1975), pp. 83–122.
- [28] Yu A Malkov and Dmitry A Yashunin. “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs”. In: *IEEE transactions on pattern analysis and machine intelligence* 42.4 (2018), pp. 824–836.
- [29] Clara Meister, Tim Vieira, and Ryan Cotterell. “If beam search is the answer, what was the question?” In: *arXiv preprint arXiv:2010.02650* (2020).
- [30] Alec Radford et al. “Learning transferable visual models from natural language supervision”. In: *International conference on machine learning*. PmLR. 2021, pp. 8748–8763.
- [31] Steven M Rubin and Raj Reddy. “The LOCUS Model of Search and its Use in Image Interpretation.” In: *IJCAI*. Vol. 2. 1977, pp. 590–595.
- [32] Mingxing Tan and Quoc Le. “Efficientnet: Rethinking model scaling for convolutional neural networks”. In: *International conference on machine learning*. PMLR. 2019, pp. 6105–6114.
- [33] Hongyan Tang et al. “Progressive layered extraction (ple): A novel multi-task learning (mtl) model for personalized recommendations”. In: *Proceedings of the 14th ACM conference on recommender systems*. 2020, pp. 269–278.
- [34] Aaron Van Den Oord, Oriol Vinyals, et al. “Neural discrete representation learning”. In: *Advances in neural information processing systems* 30 (2017).
- [35] Hao Wang et al. “ESCM2: Entire space counterfactual multi-task model for post-click conversion rate estimation”. In: *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2022, pp. 363–372.
- [36] Sam Wiseman and Alexander M Rush. “Sequence-to-sequence learning as beam-search optimization”. In: *arXiv preprint arXiv:1606.02960* (2016).
- [37] Dongbo Xi et al. “Modeling the sequential dependence among audience multi-step conversions with multi-task learning in targeted display advertising”. In: *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 2021, pp. 3745–3755.

-
- [38] Tianhe Yu et al. “Gradient surgery for multi-task learning”. In: *Advances in neural information processing systems* 33 (2020), pp. 5824–5836.
 - [39] Ruohan Zhan et al. “Deconfounding duration bias in watch-time prediction for video recommendation”. In: *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*. 2022, pp. 4472–4481.